University of Warsaw
Faculty of Mathematics, Informatics and Mechanics

Piotr Styczyński
Student no. 386038

An integrated and practical database solution for storage and efficient processing of large genomic datasets

Master's thesis
in COMPUTER SCIENCE

Warsaw, 2026-07-28

Abstract

Genomic workflows commonly connect specialized programs through files. This arrangement separates sequences from relational metadata, repeats parsing and indexing work, and prevents the database planner from representing the cost of alignment. This thesis presents BioSeqDB, a PostgreSQL extension that stores compact biological sequences and runs a minimap2-derived seeding, chaining, and extension pipeline during database scans.

BioSeqDB provides alphabet-generic packed sequence views, scalar and single instruction, multiple data (SIMD) minimizer extraction, and Vulkan implementations for selected sketching and chaining operations. Bounded CPU dynamic-programming kernels and GPU alignment routes perform final extension. Optional SneakySnake and GateKeeper filters reject unsuitable candidate regions before that stage. The database layer adds persistent minimizer indexes, bounded ingestion, external-provider relation definitions, coordinated caching, and generated PostgreSQL boundary declarations.

Alignment is not evaluated as an independent scalar function for every row. Planner support introduces custom paths whose execution nodes batch sequences, execute the explicitly selected implementation when the input and hardware pass admission, and place alignment results into planned expressions. The same operation can therefore participate in joins, filtering, metadata projection, and persistent storage. The thesis describes how the mapper's data and control flow become a database execution primitive while retaining explicit resource bounds and support for heterogeneous processors.

Keywords

sequence alignment, bioinformatics, minimap2, PostgreSQL, databases, genomic data management, SIMD, GPU computing

Thesis domain (Socrates-Erasmus subject area codes)

11.3 Informatics and Computer Science

Subject classification

Applied computing, life and medical sciences, bioinformatics

Tytuł pracy w języku polskim

Zintegrowane i praktyczne rozwiązanie bazodanowe do przechowywania i wydajnego przetwarzania dużych zbiorów danych genomicznych

Contents

Introduction . . . 7

1. Biological foundations of sequence analysis . . . 11
   1.1. Biological context . . . 11
      1.1.1. DNA, ribonucleic acid (RNA), and protein alphabets . . . 12
   1.2. Alignment operation . . . 12
   1.3. Technical considerations . . . 13
      1.3.1. FASTA . . . 13
      1.3.2. FASTQ . . . 14
      1.3.3. CIGAR . . . 15

2. Sequence alignment and search strategies . . . 17
   2.1. Alignment foundation and specification of a problem . . . 17
   2.2. Dynamic programming definition . . . 18
   2.3. Seed and extend . . . 20
   2.4. Limitations of existing tools . . . 22

3. The BioSeqDB alignment pipeline . . . 25
   3.1. Alignment as a database execution pipeline . . . 25

4. Minimizer extraction . . . 27
   4.1. From every base to a sparse, comparable sketch . . . 27
      4.1.1. Canonical *k*-mers and minimizer windows . . . 27
   4.2. Methods of minimizer generation . . . 30
      4.2.1. Scalar rescan . . . 30
   4.3. A1: Naive SIMD rescan . . . 31
   4.4. A2: Two-stack algorithm . . . 33
      4.4.1. One window minimum from two saved values . . . 33
   4.5. A3: Chronological SIMD . . . 34
   4.6. Bioseq prototype . . . 37
   4.7. Seeding prototype V1–V6: Naive SIMD rescan . . . 38
   4.8. Seeding prototype V8: Implementing A2 algorithm . . . 39
      4.8.1. Seeding prototype V8.5: Deferring hash calculation . . . 40
   4.9. Seeding prototype V9–V11: Stream interleaving and forced inline . . . 40
      4.9.1. Seeding prototype V12: Forced inline . . . 41
      4.9.2. Seeding prototype V13: Existmator fix . . . 42
      4.9.3. Seeding prototype V14: directly use nthash . . . 43
      4.9.4. Seeding prototype V15: Smaller minimizer space . . . 44
      4.9.5. Seeding prototypes summary . . . 44

4.10. GPU acceleration for seeding . . . 45
4.10.1. Sketch pseudocode and work . . . 46
4.11. BioseqDB GPU seeding . . . 47
4.12. Packed minimizer representation . . . 50
4.13. Configured CPU algorithms . . . 52

5. Chaining seed anchors . . . 55
5.1. From local seed hits to candidate mappings . . . 55
5.2. Anchor geometry and ordering . . . 55
5.3. Chaining components . . . 57
5.4. Chaining algorithms . . . 58
5.4.1. Shared pair score . . . 59
5.5. B1: minimap2 dynamic programming . . . 62
5.5.1. Skipping with counters and marked anchors . . . 63
5.6. BioseqDB SIMD DP: exact and bounded batch . . . 67
5.6.1. Candidates in SIMD vectors . . . 67
5.6.2. Reverse chaining (backtracking) . . . 70
5.6.3. Minimap2 backtracking . . . 71
5.6.4. BioseqDB SIMD backtracking . . . 75
5.6.5. Effect of repeated anchors on the first-path vector run . . . 78
5.6.6. BioseqDB SIMD performance comparison . . . 87
5.7. How minimap2 and G3SA split the dynamic program . . . 88
5.8. G3SA first pass for one seed segment . . . 90
5.9. Choosing one predecessor before computing its full score . . . 91
5.9.1. Minimap2 RMQ . . . 92
5.9.2. G3SA second pass: comparing $h(j)$ on the GPU . . . 94
5.10. BioseqDB implementation . . . 94
5.10.1. How BioseqDB RMQ differs from minimap2 . . . 96
5.10.2. BioseqDB Vulkan work split . . . 96

6. Final alignment and extension . . . 97
6.1. From a seed chain to a base-level alignment . . . 97
6.2. Affine-gap dynamic programming . . . 97
6.2.1. Banding and stopping criteria . . . 98
6.3. Alignment around a chain . . . 99
6.4. KSW2 with portable SIMD . . . 100
6.4.1. Difference representation . . . 100
6.4.2. Endpoint and traceback rules . . . 100
6.5. Adaptive block alignment . . . 101
6.5.1. Block movement and growth . . . 101
6.5.2. SIMD recurrence within a block . . . 101
6.5.3. Exact diagonal certificate . . . 102
6.6. Traceback, CIGAR, and edit counting . . . 102
6.7. Accelerator boundary and gridded traceback . . . 103
6.8. Runtime selection and design limits . . . 104

7. Alignment prefiltering . . . 105
7.1. Why filter after chaining but before alignment? . . . 105
7.2. SneakySnake chip-maze filtering . . . 106
7.3. GateKeeper shifted-mask filtering . . . 108
7.3.1. The exact rejection guard . . . 110
7.4. Dispatch and query-pipeline integration . . . 111
7.5. Limitations . . . 112
7.6. Conclusion . . . 112

8. Sequence representations and lazy evaluation . . . 113
8.1. One logical sequence, several physical forms . . . 113
8.2. Alphabet-generic compact ranges . . . 114
8.2.1. Logical and physical traits . . . 114
8.2.2. Ranks, symbols, and orientation . . . 115
8.3. Packing and resident PostgreSQL values . . . 116
8.3.1. Packing geometry . . . 116
8.3.2. A flat trailing-payload representation . . . 116
8.4. A lazy view over flat and fragmented storage . . . 117
8.4.1. The reader-facing facade . . . 117
8.4.2. Ownership and failure handling . . . 118
8.5. Persistent slices and lazy composition . . . 118
8.5.1. Immutable piece trees . . . 118
8.5.2. Bound and unbound values . . . 119
8.5.3. SQL views and transforms . . . 119
8.6. Access planning and materialization . . . 120
8.6.1. Consumer requirements . . . 120
8.6.2. Why eager conversion is avoided . . . 121
8.7. Contract for sequence consumers . . . 121

9. PostgreSQL API and custom scans . . . 123
9.1. Alignment as a relational operation . . . 123
9.2. SQL surface and ownership contract . . . 124
9.3. Planner support, custom paths, and custom plans . . . 124
9.4. Typed placeholder injection . . . 125
9.5. Bounded batch execution . . . 126
9.6. Join execution and rescans . . . 127
9.7. User-visible plan information . . . 127
9.8. Conclusion . . . 128

10. Physical sequence storage in PostgreSQL . . . 129
10.1. Storage as part of query execution . . . 129
10.2. Columnar relation layout . . . 129
10.2.1. Logical byte space and reservations . . . 129
10.2.2. Stripes, chunk groups, and streams . . . 130
10.3. Fragmented and packed sequence payloads . . . 132
10.3.1. Locator and fragment geometry . . . 132
10.3.2. Buckets, shards, and the canonical registry . . . 132
10.3.3. Insertion, concurrency, and cleanup . . . 133
10.4. Projection, pruning, and lazy reconstruction . . . 133

10.5. Coordination of repeated query requests . . . 134
10.6. Resulting storage contract . . . 135

11. Persistent sequence indexes . . . 137
11.1. Persistent indexes as derived database state . . . 137
11.2. Logical organization of minimizer indexes . . . 137
11.2.1. Minimizers per fragment . . . 138
11.2.2. Minimizers per sequence . . . 138
11.3. Bounded construction and publication . . . 139
11.4. Compatibility and integrity . . . 141
11.5. Selective cold loading and memory accounting . . . 141
11.6. Runtime index selection . . . 142
11.7. Differences from minimap2 `.mmi` . . . 143
11.8. Operational limits and conclusions . . . 144

12. Bounded sequence ingestion and integrity limits . . . 145
12.1. Bounded sequence ingestion and integrity limits . . . 145
12.2. Layered input pipeline . . . 145
12.3. Incremental parsing and direct compact encoding . . . 146
12.4. Bounded delivery and backpressure . . . 147
12.5. Header-to-column mapping . . . 147
12.6. SQL publication and error behavior . . . 148
12.7. Integrity limits . . . 149
12.8. Conclusion . . . 150

13. External sequence providers . . . 151
13.1. Remote databases as relations . . . 151
13.2. Provider and execution boundaries . . . 152

14. Source-driven SQL code generation . . . 153
14.1. One source for the PostgreSQL boundary . . . 153
14.2. Generated artifacts and safety . . . 154

15. Conclusions . . . 155

References . . . 157

Introduction

Sequencing instruments turn biological material into collections of strings. The useful interpretation of those strings rarely exists in isolation. A read has an instrument and sample provenance. A reference interval belongs to an assembly and annotation release. An alignment has a score, a strand, a position, an edit path, and a relationship to experimental metadata. Conventional workflows keep the strings in FASTA and FASTQ sequence files, Sequence Alignment/Map (SAM) and Binary Alignment/Map (BAM) alignment files, or mapper-specific index files while keeping much of the surrounding information in tables or ad-hoc scripts. Each tool sees only the portion of the workflow represented by its command-line arguments.

Relational database systems solve a different part of the problem. PostgreSQL provides durable storage, transactions, typed values, indexing, joins, extensible operators, and a cost-based planner. It does not, by itself, provide the representations or execution kernels required for billions of packed symbols and computationally expensive sequence searches. A naive user-defined function is not sufficient. Evaluating one alignment function independently for every row prevents batching, hides accelerator opportunities, and gives the planner little information about a predicate that can dominate the query.

BioSeqDB investigates the boundary between these two systems. It is implemented as a PostgreSQL extension whose non-database core is written in C++26. Biological symbols are represented by generic compact ranges rather than by a deoxyribonucleic acid (DNA)-only byte string. The alignment path is derived from minimap2: representative k-mers are extracted as minimizers, matching minimizers become anchors, compatible anchors are chained, and CPU or GPU alignment routes extend the selected regions. The port is not a line-for-line translation. The implementation introduces template-selected alphabets and layouts, Highway SIMD kernels, task-based parallel execution, Vulkan compute paths, persistent PostgreSQL-backed indexes, and prefilters for candidate regions. These changes must preserve the semantics that make minimap2 useful while operating on data that may be lazy, fragmented, cached, or obtained from an external provider.

The database boundary is equally important. An SQL expression such as `mmsearch(a, b)` has a conventional function definition so that PostgreSQL can type and plan it. The ordinary function body is not an alignment fallback: it fails closed if no BioSeqDB custom executor owns the expression. Its planner-support function assigns the unowned expression a high startup cost, while structural planner hooks can construct an owning custom path with explicit nonnegative costs. The resulting custom scan batches input, invokes the alignment pipeline, and makes its results available to planned expressions through injected PostgreSQL nodes. This is what permits alignment to participate in filters, projections, and joins while retaining the execution structure required by central processing unit (CPU) vector units and graphics processing units (GPUs).

The central research question is therefore not whether an existing aligner can be called from SQL. It is whether the aligner's data and control flow can become a database execution

primitive without losing algorithmic fidelity, resource bounds, or the ability to exploit modern processors. The investigation is organized around four supporting questions:

RQ1. Which parts of seeding, chaining, and extension can be expressed over a common compact sequence abstraction without eager conversion?

RQ2. Which stages admit SIMD or GPU implementations, and which input and resource constraints govern their selection?

RQ3. How can PostgreSQL plan and execute a set-valued, batch-oriented search while presenting an ordinary typed SQL interface?

RQ4. How should sequences, indexes, ingestion, external data, caches, and generated boundary code fit around the kernels so that the resulting system remains inspectable and reliable?

This thesis answers these questions through implementation and analysis of algorithmic and execution contracts. Historical designs appear only when they explain a constraint or a rejected optimization. They are not presented as simultaneous production architectures. Comparisons with minimap2, SIMD-Minimizers, the G3SA GPU seed-chain-extend system, Block Aligner, SneakySnake, GateKeeper, and Citus Columnar distinguish inherited mechanisms from BioSeqDB-specific changes.

The main contributions developed in the following chapters are:

C1. An alphabet-generic sequence representation that supports packed, fragmented, lazily composed, and PostgreSQL-backed data through a common range contract.

C2. A staged minimap2-derived mapper with scalar, SIMD, and Vulkan components and explicit runtime dispatch.

C3. Optional GPU-capable SneakySnake and GateKeeper candidate filters integrated without changing the meaning of an accepted alignment.

C4. Persistent sequence and minimizer-index storage derived from a columnar PostgreSQL access method, with bounded loading, caching, and selective cold-index reads.

C5. A planner and custom-scan interface that connects SQL expressions and joins to batch-oriented alignment.

C6. Structural generation and validation of SQL installation definitions and PostgreSQL function-manager boundary declarations from annotated C++ sources.

The document first establishes the biological and algorithmic context, then states the shortcomings that motivate an integrated system. The algorithm chapters follow data through minimizer extraction and chaining. They then describe the expensive downstream alignment stage before stepping back to the prefilter that guards that stage. This expository inversion makes the filter's safety and cost contract concrete without changing the runtime order, which remains chaining, prefiltering, and alignment. It then moves outward through sequence abstractions, PostgreSQL planning, physical storage, persistent indexes, ingestion, external providers, and generated SQL boundary code.

- Chapter 1 introduces biological sequences and sequencing data. It explains variation and the computational questions that motivate database-resident sequence analysis.

- Chapter 2 develops exact dynamic programming and seed-and-extend mapping. It also presents graph-based approaches and the limits of existing tools.
- Chapter 3 presents the complete processing path with its stage contracts and CPU and accelerator routes.
- Chapter 4 explains minimizer sketching and rolling hashing. It compares the SIMD and Vulkan implementations with minimap2 and SIMD-Minimizers.
- Chapter 5 derives the chaining objective and compares its scalar RMQ SIMD and GPU forms. It then states the selection policy.
- Chapter 6 describes affine-gap alignment with KSW2 and block-based SIMD extension. It also covers traceback and accelerator integration.
- Chapter 7 describes SneakySnake and GateKeeper. It explains their filtering guarantees and GPU mappings before showing how BioSeqDB applies them.
- Chapter 8 develops compact sequence ranges and bit-level views. It then presents fragmented and lazy sequences with PostgreSQL-backed adapters and alphabet dispatch.
- Chapter 9 follows an mmsearch expression through planning and batched execution. It explains joins and per-row result substitution.
- Chapter 10 explains the columnar-derived storage layout and fragment metadata. It also covers caching and request coordination.
- Chapter 11 describes index construction and storage with compatibility checks and selective cold loading. It also explains updates and differences from minimap2 indexes.
- Chapter 12 presents the bounded input/output pipeline for FASTA and FASTQ data. It covers compression and header parsing with typed PostgreSQL writes and failure handling.
- Chapter 13 explains generated API clients and the provider abstraction that exposes remote sequence databases as PostgreSQL relations.
- Chapter 14 explains structural source analysis and checked annotations. It then presents dependency ordering and generation of the PostgreSQL boundary.
- Chapter 15 answers the research questions and summarizes the final system. It identifies the principal limitations and possible extensions.

Chapter 1

Biological foundations of sequence analysis

1.1. Biological context

"Significant scrutiny and attention have been given to genome-sequencing costs and how they are calculated since the beginning of the field of genomics in the late 1980s". Institute, The Cost of Sequencing a Human Genome NHGRI (National Human Genome Research Institute) has carefully tracked costs per genome and expose this data publically.

In the last 15 years, there has been seen a massive development of sequencing technologies. Companies like Applied Biosystems, Illumina, Roche, and Pacific Biosciences have developed a plethora of sequencing platforms, e.g. SOLiD, GS FLX, and PacBio. These tools are able to generate an enormous amount of data in a matter of days. As an example, Illumina NextSeq generates up to 360 Gbp, split into reads of length up to 150 bp in merely 2 days (data taken from Illumina website).

Although those platforms vary in detail, the general idea behind them is as follows:

1. DNA is fragmented through physical and chemical methods. Adapters are appended to the ends of the fragments to form what is called a sequencing library.
2. Fragments of unwanted size or self-ligated adapters are removed from the library.
3. The library is immobilized on a chemical bead so that it can be steadily observed.
4. Amplification is performed, e.g. through PCR emulsion
5. A combination of optical devices and fluorophores is used to make reads.

With new advancements in sequencing technologies, the cost of sequencing a genome has been decreasing significantly to the point where most popular open databases store more than 10s of petabytes of data. To make this data usesable researchers often need to perform search operations:

- For bioinvestigative resarch: does this genomic sequence already contains known genes? Does it contain a known mutation? Is it a known pathogen?
- For evolutionary biology: is this genomic sequence a known evolutionary lineage? What is the genetic distance between this sequence and a known sequence?
- For legal research: is this DNA sample from known individual? Does it represent known kinship to someone else?

This search operation is using an operation called ”sequence alignment” which is the core concept of this thesis.

1.1.1. DNA, ribonucleic acid (RNA), and protein alphabets

DNA is a polymer containing one of four bases: adenine, cytosine, guanine and thymine. Their conventional one-letter symbols form the alphabet

$$\Sigma_{\text{DNA4}} = \{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}.$$

Four symbols are sufficient for an unambiguous DNA string, but experimental data introduces ambiguity. The symbol `N` conventionally means that the nucleotide at a position is unknown or was not resolved. Other International Union of Pure and Applied Chemistry (IUPAC) codes describe sets of possible bases, such as a purine at a position.

RNA uses adenine, cytosine, guanine and uracil, so a basic RNA alphabet replaces `T` with `U`. Many RNA molecules are transcribed from DNA. Crick, “Central Dogma of Molecular Biology”

A protein sequence is written over the twenty common amino-acid symbols. It’s alphabet is notably larger, but in most cases we can translate it to a DNA sequence and use the DNA alphabet directly.

Sequence class	Core symbols
DNA4	`A,C,G,T`
RNA4	`A,C,G,U`
DNA5	`A,C,G,T,N`
RNA5	`A,C,G,U,N`
PROT20	Twenty common amino-acid symbols

Table 1.1: Biological alphabets and the distinctions that a sequence-processing system must not erase. The table describes biological data in general. It does not assert that every alphabet or ambiguity convention is exposed by every BioSeqDB SQL path.

1.2. Alignment operation

Due to nature of biological sequences, search must be able to match fragments that are non-exact, thus alignment operation is similar to popular fuzzy-search operations on normal text. The alignment can be viewed as a processes that optimises gaps (insertions and deletions) in a way to optimise similarity metric. High similarity score between parts of sequences can indicate that they share same evolutionary origin or have a common purpose. By comparing newly found sequences to already researched ones, we can deduce lots of relevant biological data. Juliusz K and W, “Master thesis: An integrated and practical database solution for storage and efficient processing of large genomic datasets”

We will consider finite nonemptystrings over the DNA4 alphabet. A gap is a symbol indicated by `-`. An alignment is a process of adding gaps to the sequences so that they are of the same length and at the same time, there is no index where a gap occurs in both sequences.

As an example, let us present an alignment of $GATCAGATA$ and $CATTCAATCA$. One of the possible alignments is:

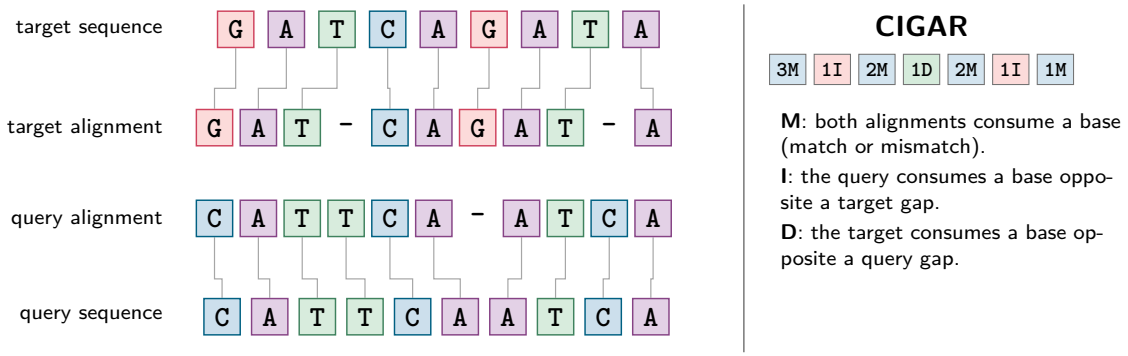


Figure 1.1: Alignment of the target sequence `GATCAGATA` and query sequence `CATTCAATCA`. Orthogonal connectors map each ungapped sequence position to its aligned column; gaps are intentionally unboxed.

There are 4 cases that can occur during alignments:

1. Insertion - a gap has been added to the first sequence
2. Deletion - a gap has been added to the second sequence
3. Mismatch - letters at a given index are different
4. Match - letters at a given index are the same

1.3. Technical considerations

A typical workflow for sequence analysis includes 3 data formats - FASTA, FASTQ, and SAM/BAM. Reference genome, stored in FASTA, is aligned with millions of reads in the FASTQ file and the result of alignment is saved as SAM/BAM file. The reason for the reference genome being in FASTA and reads in FASTQ is that we assume that the reference genome is correct, but reads produced by sequencers can contain various errors. Moreover, we're more likely to score alignments with high-quality bases higher than the ones with low-quality.

1.3.1. FASTA

FASTA is a text-based file format for storing nucleotide sequences. It is one of the most widely used formats in bioinformatics. The current standard specification is given by NCBI. Below is a description of the usually encountered version.

It consists of a series of entries separated by a new line. Each entry starts with a single-line header. The header starts with `>` and contains various metadata about the following sequence, like the description of the sequence, library accession number, and submission date. After the header comes the sequence itself represented with standard IUB/IUPAC acid codes, split into lines of length usually not exceeding 80 characters. Aside from A, C, T, G commonly encountered is code N, which means any of A/C/T/G. Lowercase letters are treated the same as uppercase. The usual extension is .fasta and .fa. An example FASTA file:

```
>PartOfHumanGenome|HG_9357/2022|2022-03-01
ACTTTGAGCTAGCATCGATTGGATCAGATGCAT
ACTCATCGGCAGTCGCATCAGCTTGGACTACGT
>PartOfMouseGenome|MG_5417/2019|2019-08-14
ACTATAGGAGCAGANNNNNNNNCTAGAGACATC
TCTGCGTNTNTTCCGGACGTACGTAGCATAGTC
```

The advantages of this format are simplicity and human readability. The downside is that it is not optimal in terms of size. To adhere to this issue, compression software is used, like gzip.

1.3.2. FASTQ

FASTQ is an extension of the FASTA format. It enhances FASTA by adding quality scores of sequences. It is a standard output format of sequencers.

Each entry consists of 4 lines. The first line starts with `@` and contains metadata about the sequence. The second line contains the sequence itself. The third line starts with `+` and may contain some additional metadata. The fourth line contains quality scores, represented by ASCII characters with codes from 33 to 126, one character per nucleotide. The file extension used is .fq and .fastq. Again, gzip can be used to reduce file size. An example FASTQ file:

```
@HumanGenomeSequencerRead
ACTACTAGACGG
+Genome20200301
*!]^##[$*@*!(
```

The quality score is usually a phread score, meaning Q satisfies $Q = -10 \log_{10} p$, where p is the probability of a corresponding nucleotide being incorrect. ASCII codes from 33 to 126 are mapped from 0 to 93.

1.3.3. CIGAR

CIGAR is an acronym for Concise Idiosyncratic Gapped Alignment Report and has been originally defined by the Exonerate alignment program. The CIGAR format is designed to contain the minimal information necessary for the reconstruction of an alignment. This format does not include nucleotides. Sweden, The Cost of Sequencing a Human Genome

- M - match or mismatch
- D - deletion
- I - insertion
- S - soft clipping. Can occur only at the beginning or at the end of the alignment. Indicates that nucleotides are clipped from the alignment.
- H - hard clipping. The same as soft clipping, except that the clipped part is not present in the SEQ field of the SAM file.
- N - skipped region. Used for representing introns (part of the gene that is non-coded) in the alignment.
- = - match
- X - mismatch

Chapter 2

Sequence alignment and search strategies

2.1. Alignment foundation and specification of a problem

Alignment can be done in different ways, where the core algorithms stays the same, but some of the conbditions change. It's important to understand what are the capabilities of standard dynamic programming algorithms and how they can be used to solve different problems.

Formally: Let $X = x_1 \dots x_n$ and $Y = y_1 \dots y_m$ be the sequences we align. An alignment is a sequence of columns

$$A = ((a_1, b_1), \dots, (a_L, b_L)), \qquad (a_r, b_r) \in (\Sigma \cup \{-\})^2 \setminus \{(-,-)\},$$

where first and second elements reciver contents of X and Y after gaps are removed:

1. An element of shape $a_r, -$ signifies an insertion in X
2. An element of shape $-, b_r$ signifies a deletion in Y
3. An element of shape m_r, m_r signifies a match
4. An element of shape a_r, b_r where $a_r \neq b_r$ signifies a substitution

This alignment is an optimized sequence correspondence under a scoring model. For substitution score $s(a, b)$ and gap cost $g(\ell)$, the score is

$$S(A) = \sum_{\text{symbol pairs } r} s(a_r, b_r) - \sum_{\text{gap runs } u} g(|u|).$$

The scoring model may be formulated in different ways, but one of the simplest and most-common conventions is to use an affine gap cost model:

$$g(\ell) = q + \ell e, \qquad \ell \geq 1, \tag{2.1}$$

where q is the opening cost and e is the extension cost.

A gap of size one costs then $q + e$. Some scientific papers mentioning $q + (\ell - 1)e$ use a different parameter convention. Affine scoring charges one opening cost per gap run and can be evaluated with a fixed number of states per matrix cell Gotoh, "An improved algorithm for matching biological sequences."

This scoring model isn't sufficient to define entire problem:

1. Global alignment consumes both complete sequences Needleman and Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins"

2. Local alignment may restart at zero and selects the best pair of substrings Smith and Waterman, "Identification of common molecular subsequences"
3. Semi-global and overlap goals consume selected ends while allowing specified prefix/-postfix to remain unaligned
4. Extension begins from a fixed anchor and chooses an endpoint under a stopping rule
5. Clipping will just finish on the rightmost gap and keep postfix unaligned
6. Local-extension fixes the start but permits an interior endpoint

Figure 2.1 shows schematic examples of these modes.

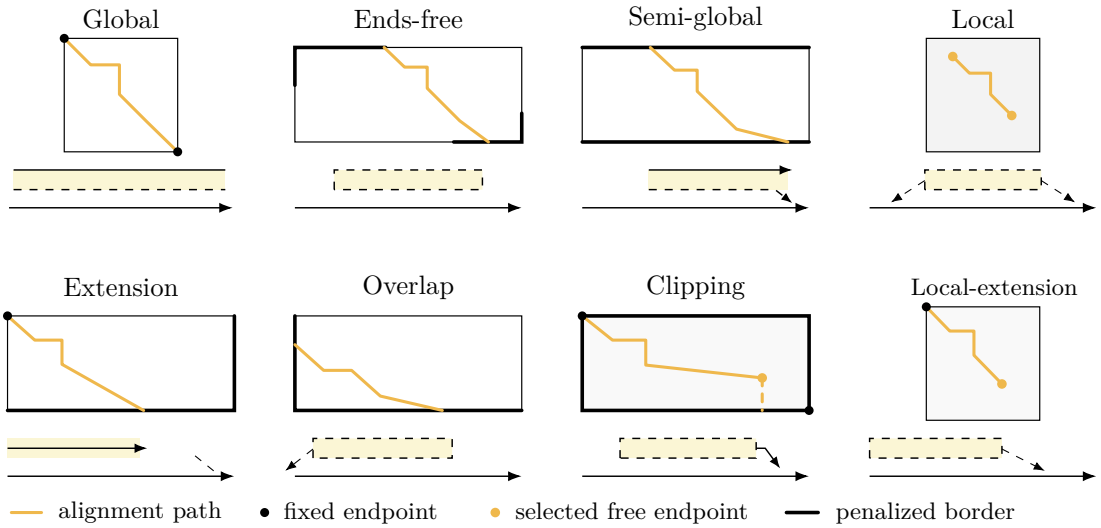


Figure 2.1: Eight alignment boundary modes. Yellow lines depcict possible tracebacks. Black dots mark fixed endpoints (for example for global alignment we always align entire sequence so endpoints are 0 and len(Y)-1), yellow dots mark selected free endpoints, and bold border segments mark penalized or anchored boundaries (usually the border is the length of sequence, but under certain conditions we might have limits on the alignment length and coordinates). In each lower schematic, the solid arrow is the reference and the yellow dashed track is the query. These panels showcase how different libraries/algorithms handle different goals across number of scientific resarch

Various practical implementations also use special stopping criteria, which usualy can be upper bound on the $S(A)$ score.

2.2. Dynamic programming definition

Let $H_{i,j}$ be the best score after consuming i symbols of X and j symbols of Y. For a simpler linear gap cost model d, we can define the algorithm's state matrix as:

$$H_{i,j} = \max\{H_{i-1,j-1} + s(x_i, y_j), H_{i-1,j} - d, H_{i,j-1} - d\}. \tag{2.2}$$

This is the intuitive definition based on 3 possible options (replacement or two variants of gap). Free-end alignment (like local) change the score values for the cells from which traceback may begin.

For affine scoring model we need to hold main state and one state for each gap orientation. Let $I_{i,j}$ and $D_{i,j}$ end in gaps in X and Y, respectively, and let $M_{i,j}$ end in a symbol pair. Under Equation 2.1, one formulation is

$$\begin{aligned} M_{i,j} &= H_{i-1,j-1} + s(x_i, y_j), \\ I_{i,j} &= \max\{H_{i,j-1} - q - e,\ I_{i,j-1} - e\}, \\ D_{i,j} &= \max\{H_{i-1,j} - q - e,\ D_{i-1,j} - e\}, \\ H_{i,j} &= \max\{M_{i,j}, I_{i,j}, D_{i,j}\}. \end{aligned} \tag{2.3}$$

Using H to open a gap permits a transition from any previous state. Local alignment also includes zero in the final maximum.

Algorithm 1 Affine-gap alignment with an explicit boundary contract

Require: sequences $X[1 \dots n]$, $Y[1 \dots m]$, scores s, q, e, and a boundary contract B
Ensure: score, endpoints, and an optional CIGAR string

```
initialize H, I, and D according to B
for i ← 1 to n do
    for j ← 1 to m do
        update M_{i,j}, I_{i,j}, D_{i,j}, and H_{i,j} with Equation 2.3
        store the predecessor of each winning state when traceback is requested
select the endpoint permitted by B
follow predecessors to an allowed start and run-length encode the operations
```

The algorithm state matrix can be used to retrieve the edit path as CIGAR string. This is the standard compact representation of gapped alignments described in chapter 1SAM/BAM Format Specification Working Group, Sequence Alignment/Map Format Specification

Affine model is some scenarios might not be enough as it doesn't distinguish between short and long gaps. The two-piece affine model can support such feature without complicating the scoring function too much:

$$g_2(\ell) = \min(q_1 + \ell e_1,\ q_2 + \ell e_2)$$

This model permits different cost slopes for short and long gaps. In our software we use single affine model for the fastest alignment modes (adaptive block), while KSW2 variants support dual affine model.

For large datasets evaluating entire large $O(nm)$ matrix can make significant practical difference:

- We can evaluate alignments only close to diagonal. Being close to diagonal means that there aren't many large gaps between matching fragments. Designing algorithms around diagonals can make us skip evaluation of many far cells that do not contribute to the result.
- In case of features that do not require traceback (CIGAR construction requires traceback, but score or coordinates of best match do not) we can just store relevant slice in memory and avoid allocating large memory region.
- We can introduce X-drop stop condition: stop when score falls below best score seen so far by more than X

- We can introduce Z-drop stop condition: the same as X-drop, but if the distance from diagonal is too high then stop as well (this means that we stop when there are too many gaps cummulatively)

Smart evaluation startegy can combine all these approaches. KSW2 and Block aligner for example use all of them. Block aligner Liu and Steinegger, "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices" evaluates only the most promising rectangular region in the matrix while KSW2 Suzuki and Kasahara, "Introducing difference recurrence relations for faster semi-global alignment of long sequences" focuses on diagonal-centered evaluation.

2.3. Seed and extend

Applying local dynamic programming between a query of length m and a reference of length N costs $O(Nm)$. Most reference positions don't contribute to the result. Long-read mappers therefore organize search into three stages Li, "Minimap2: pairwise alignment for nucleotide sequences."

First step is to use divide and conquer approach and split sequence into representatives known as seeds:

$$\begin{aligned}\mathsf{Seed}(x_1,\ldots,x_n) = \{(s_1,i_1,j_1),\ldots,(s_k,i_k,j_k)\},\\ \text{where } 1 \le i_a, j_a \le n \quad \text{for all } a \in \{1,\ldots,k\}.\end{aligned} \tag{2.4}$$

Seed (s_a, i_a, j_a) is some represenative of the sequence substring $x_a = x_{i_a} x_{i_a+1} \ldots x_{j_a}$. This can be implemented in different ways, this definition is the most abtract one. If a seed occurs in both query and reference, then we can consulde that two represented substrings are very similar. Using some concrete seeding function we can obtain intersection for query and reference:

$$S_{X\cap Y} = \left\{ (s, i_x, j_x, i_y, j_y) \;\middle|\; \begin{matrix}(s,i_x,j_x) \in \mathsf{Seed}(X),\\ (s,i_y,j_y) \in \mathsf{Seed}(Y)\end{matrix} \right\}. \tag{2.5}$$

1. Seeding Generate $Seed(X)$ and $Seed(Y)$ and obtain $S_{X\cap Y}$
2. Chaining As matching seeds will cover only parts of both sequences that match without errors, we still need to fill-in the gaps between them. We connect seeds from $S_{X\cap Y}$ into chains using similar dynamic programming approach as in usual aligner. This gives us some approximate alignments.
3. Extension Perform usual alignment algorithm on the approximate fragments to obtain final results.

Minimizers are one of many possible seeding model definitions. Given parameters w and k, a window W of length $l = w + k - 1$ contains w consecutive k-mers. The minimizer of the window is the smallest k-mer (subsequence of length k) in the window. For random minimizers, k-mers are ordered by a pseudo-random order, usually given by comparing hashes of the k-mers. In case of ties, the leftmost smallest k-mer is chosen. Roberts et al., "Reducing storage requirements for biological sequence comparison" Overlapping windows often retain the same representative, which is emitted once.

Canonical minimizers: DNA is double-stranded and majority of sequencing technologies do not distinguish these two strands. Genomic sequences thus need to satisy additional constraint: sequence and its reverse complement ($A \leftrightarrow T, C \leftrightarrow G$) should be handled in the same

way by the software (they should be indistinguisable) In our case this means that for minimizer of a window W at position p we need to have minimizer $\overline{W^c}$ at position $w-1-p = |W|-k-p$. Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast" In most of the software due to simplicity truly cannonical minimizers aren't used. Instead we take minimum of hashes of k-mer and its reverse complement: $m^{\text{cannonical}} = \min(m, \overline{m^c})$

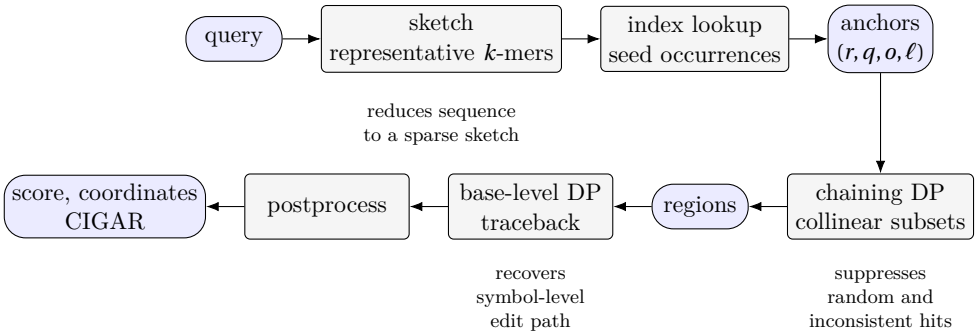


Figure 2.2: The seed–chain–extend decomposition

In case of Minimap2 and BioseqDB $S_{X\cap Y}$ is represented as tuple (r, q, o, ℓ) which holds reference coordinate r, query coordinate q, orientation o and span ℓ (length).

After sorting anchors by ending reference coordinate, with implementation-specific tie keys, a chaining score can be written

$$f(i) = \max\left(w_i,\ \max_{j<i}\{f(j) + \alpha(j,i) - \beta(j,i)\}\right), \tag{2.6}$$

where α rewards newly covered matching bases and β penalizes large gaps and diagonal changes. A representative minimap2 reward is

$$\alpha(j,i) = \min\big(\min(y_i - y_j,\ x_i - x_j),\ w_i\big).$$

Testing every $j < i$ is $O(A^2)$ for A anchors. Minimap2 scans a bounded recent window and stops after a configured number of unproductive indices, making the practical cost much smaller Li, "Minimap2: pairwise alignment for nucleotide sequences."

Algorithm 2 Conceptual bounded predecessor chaining

1. Input. Anchors $a_1, \ldots, a_A$ in canonical coordinate order.
2. Endpoint loop. For each $i = 1, \ldots, A$, perform the following steps:
   (a) Initialize $f[i] \leftarrow \mathsf{span}(a_i)$ and $p[i] \leftarrow \perp$.
   (b) For each candidate $j < i$ in the active distance and iteration window, perform the following steps:
      i. If a_j and a_i have valid order, strand, gap, and bandwidth, compute
      $$v \leftarrow f[j] + \mathsf{reward}(a_j, a_i) - \mathsf{gapPenalty}(a_j, a_i).$$
      ii. If the candidate is valid and $v > f[i]$, set $f[i] \leftarrow v$ and $p[i] \leftarrow j$.
      iii. Optionally stop the predecessor loop after the configured skip heuristic fires.
3. Backtracking. Backtrack from high-scoring endpoints through p.
4. Result. Return the chains that remain after filtering by score, seed count, and overlap.

2.4. Limitations of existing tools

Most of the current most popular tools implement variations of the seed and extend approach described in the previous section. However the algorithm outlined here is heavily underspecified to be directly implemented as a computer program:

- $S_{X \cap Y}$ can be implemented as interection of two hash maps where $Seed(X)$ is realized as map from seed to its start/end: $S \to \natural^2$ where seed itself is represented by it's hash $S = \mathbb{Z}$. This opens few important questions: what hashing strategy to choose? Which map implementation to choose?
- As $Seed(X)$ can be computed before alignment starts we can store it in memory and persist it to disk to speed up processing in the future. Which binary format we want to choose?
- Anchors as tuples will be implemented as some concrete binary structure and authors need to specify their representation (do we store them as AOS or SOA? What are widths of elements and their types?). Each decision will imapct performance as there can be millions of anchors per long sequence and additional bytes sum up to MBs of additional memory to be processed (and slowdowns).
- Finally, what is the most effective way to parallelize this computation (the most straightforward approach is to split input sequences X and Y into chunks)

Most of the current state of the art tools choose to:

1. File-based operation
2. Suboptimal or non-extensible internal data structures
3. Optimise only selected parts of the algorithm like hash function properties or one specific stage.

This leads to different consequences for the user.

File-based operation: Operating directly on the disk with simplistic wire format means that seed index is usually one giant binary blob not prepared for highly-concurrent access and interface doesn't offer modern tools like buffer pooling or lazy loading. The format of indexes are largely undocumented by authors. Missing of lazy loading feature is the most hurting to the user. If index is large it will be loaded entirely into memory and can overwhelm SWAP space. This degrades performance by several orders of magnitude and can be avoided by splitting sequences into smaller chunks. User doesn't know what is the exact relationship between sequence length and required memory space, so this process takes time and involves guessing. Additionally index implementation sometimes offer complex parameters to adjust its partitioning (like in case of MMseqs2), but entire responsibility is still shifted to the user and that leads to user frustration andrecurring OOM issues or severly degraded performance. Youngblut, MMseqs2 Github issue 338 Zhan, MMseqs2 Github issue 517 Software not only treats indexes as files, but it operates on FASTA files directly. This means the user cannot easily sort/split/extract or filter records without using external tools.

Suboptimal or non-extensible internal data structures: Minimap2 uses 31 bits to encode anchor's position meaning that practically entire human genome (>3Gbps) cannot be aligned at once and require splitting via other tools before any operation is performed. LexiMap (not using seed and extend approach) limits maximum genome size to 268Mbp Shen, LexicMap FAQs. Additionally software is implemented without proper abstractions - minimap2 code is non-easily extensible C code, while G3SA doesn't offer extension points for new types of indexes or sequences interfaces.

Selective optimisations: Optimisations are usually fixed for one single platform (AVX512 or SSE) and either get severe performance penalties (scalar fallbacks) or user cannot use them at all. SIMD support is quite popular even though it supports one or two selected platforms, but GPU support exists usually only for NVIDIA graphics cards. This leads to a difficult situation for the user as they need to choose between several versions of the same tool (Saurabh Kalikar, mm2-fast: Accelerating minimap2 for long-read sequencing applications on modern CPUs, Sadasivan H, mm2-gpu: Accelerating Minimap2 for Accurate Long Read Alignment on GPUs), while better, more modern software like G3SA exists purely on NVIDIA hardware Han et al., "G^{3}SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment" Modern hardware like MacBook Pro M3 with arm64 architecture won't be able to utilise any integrated GPU acceleration. While minimap2 offers compatibility layer to port SSE code to NEON, it does not vectorize all operations and code ported from SSE doesn't account for lane widths and instruction level parallelism available in M2-M4 chips. Feintzeig, SIMD + Parallelism: Ray Tracer Optimization Part 3

I wanted to address these limitations by implementing BioseqDB, Postgres extension that will provide the following features:

- Tight integration with Postgres meaning that user can operate on sequences (loaded FASTA files) directly using SQL rather than resorting to external toolkits.
- Support for indexes and storage structures designed with parallelism and lazy-loading from the start, meaning that software can be run on very RAM-constrained device and still perform well. At the same time design will incorporate assumptiuon about arbitrary large sequences so the user will be able to operate on >3Gbps records without any additional steps.

- Provide layer of abstractions written using latest C++26 standard, so the algorithms can be reused in separate software or extended to different types of new sequences and indexes.
- Provide optimised kernels for SIMD and GPU architectures written using architecture-agnostic, but performant code.

Tight integration with Postgres also means that as a side effect, we can utilise some benefits of persistent state: deferring expesive operations, keeping large indexing structures in memory in case use reruns query and more effective memory-disk caching schemes.

Chapter 3

The BioSeqDB alignment pipeline

3.1. Alignment as a database execution pipeline

This is empty for now.

Chapter 4

Minimizer extraction

4.1. From every base to a sparse, comparable sketch

The first computational stage of the BioSeqDB aligner needs to compute $\mathsf{Seed}(X)$. A target seed becomes a key in an index. A query seed is looked up in that index, and each compatible occurrence becomes an anchor for the chaining stage described in Section 5.1. The seed set must be sparse enough to reduce index and lookup costs, but dense enough that related sequences retain common seeds.

BioSeqDB uses minimizers. A minimizer is the smallest by some score function k-mer in a window of consecutive k-mers Roberts et al., "Reducing storage requirements for biological sequence comparison."

4.1.1. Canonical k-mers and minimizer windows

Let $S = s_0 s_1 \dots s_{n-1}$ be a nucleotide sequence. The k-mer beginning at zero-based position i is

$$K_i = S[i, i+k).$$

For DNA, the forward and reverse-complement encodings are

$$F_i = \mathrm{enc}(K_i), \qquad R_i = \mathrm{enc}(\overline{K_i}).$$

A canonical k-mer uses the same key on both strands. A common choice is

$$C_i = \min(F_i, R_i), \qquad z_i = \mathbf{1}[R_i < F_i],$$

where z_i records the selected strand. Applying a hash function H gives a candidate

$$x_i = (h_i, p_i, z_i), \qquad h_i = H(C_i).$$

The position p_i may denote the first or final base of K_i. BioSeqDB's minimap-compatible schemes use the final base because that is the convention used by minimap2 Li, "Minimap2: pairwise alignment for nucleotide sequences."

For a window of w consecutive candidates,

$$W_j = (x_j, x_{j+1}, \dots, x_{j+w-1}), \qquad m_j = \min_{x \in W_j} h(x).$$

Every identity attaining m_j is a possible minimizer. We must define which equal minimum wins and whether different positions with the same minimum hash are emitted. We should

lso avoid emitting the same hash again when that candidate remains minimal as the window moves. For independent random hashes, the expected density is approximately $2/(w+1)$ minimizers per candidate position.

Figure 4.1 follows one k-mer through hashing, window selection, and adjacent deduplication. The per-window minimum positions are calculated from the displayed hashes. In particular, the window beginning at hash position five has minimum position eight, not position five.

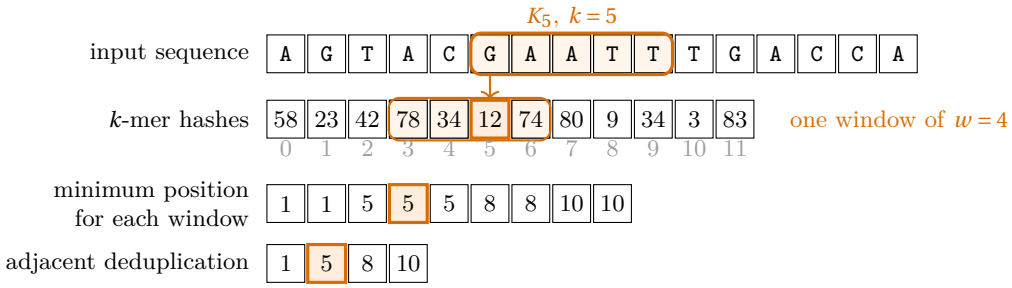


Figure 4.1: Minimizer extraction for $k=5$ and $w=4$ with illustrative hashes. Positions in this walkthrough are zero-based k-mer starts. BioSeqDB's packed minimap-compatible record converts them to final-base positions. The highlighted candidate has hash 12 and remains minimal for three consecutive windows, so adjacent deduplication emits its position once.

Minimizer stochastic properties

Minimizer density counts selected positions in one sequence. Let $N=L-k+1$ be the number of candidate k-mer starts in a fragment and let $M_L^{(1)}$ be the number of distinct positions selected when every window chooses its leftmost minimum. Every full window of w candidates selects one position. Therefore, for $N \geq w$,

$$\left\lfloor \frac{N}{w} \right\rfloor \leq M_L^{(1)} \leq N-w+1. \tag{4.1}$$

The lower bound follows by partitioning the candidates into disjoint windows. The upper bound equals the number of full windows. A sequence of strictly decreasing ranks attains it. An implementation that emits every tied minimum uses a different count $M_L^{(\mathrm{all})}$, whose upper bound is N.

Fix an ordering rule and break ties by choosing the leftmost minimum. For an infinite sequence whose bases are sampled independently and uniformly from alphabet Σ, the stochastic density is

$$\rho = \lim_{L\to\infty} \frac{\mathbb{E}[M_L^{(1)}]}{L-k+1}.$$

The general bounds are

$$\max\left\{\frac{1}{w}, \frac{1}{|\Sigma|^k}\right\} \leq \rho \leq 1. \tag{4.2}$$

Under independent continuous scores at candidate positions, the density is $2/(w+1)$. For a random fixed order on k-mer values, the density also depends on k, w, and overlap between adjacent k-mers Shur, On Minimizers of Minimum Density. The value $2/(w+1)$ is not a universal bound for a fixed order or sequence.

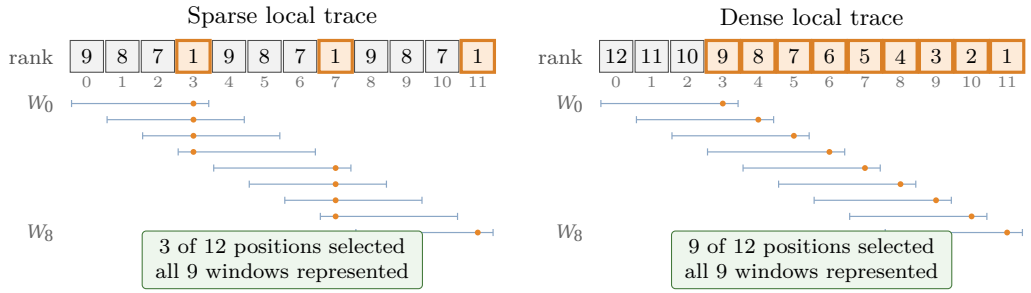


Figure 4.2: Two minimizer traces for $w = 4$. Thin brackets mark candidate windows, and orange dots mark their selected minima. The left trace reuses positions 3 and 7. The right trace selects a new position after each slide.

Density alone does not determine the number of anchors. Let $C_X(v)$ count the selected occurrences of canonical k-mer value v in fragment X. The number of raw anchor pairs between fragments U and V is

$$A(U,V) = \sum_{v} C_U(v) C_V(v). \tag{4.3}$$

Two fragments can each contain $\Theta(L)$ selected positions while their sets of selected values are disjoint. In that case they produce no anchors. Occurrence filters can only remove pairs from Equation 4.3.

Minimap2 minimizer sketch
mm_sketch 77

Anchor lookup and frequency filtering

Figure 4.3 uses fixed four-base seeds on both sides. The symbols h_1, h_2, and h_3 are some computed hashes.

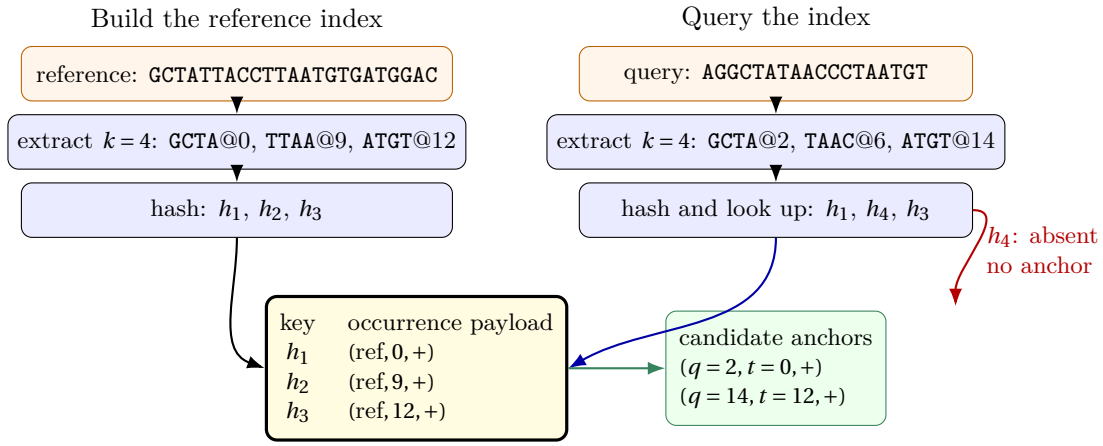


Figure 4.3: Reference-index construction and query lookup. Positions are seed starts in this example. A usable seed index stores some seed metadata like position and strand

Frequent seeds can produce very large occurrence lists. BioSeqDB can exclude seeds whose target frequency exceeds a configured threshold before anchors are constructed. This is common practice in other software as well: minimap2 can remove most common minimizers for specific sequence by using percentage threshold (`--q-occ-frac` $\in [0, 1]$)

4.2. Methods of minimizer generation

4.2.1. Scalar rescan

The most straightforward algorithm to select smallest hash in a given sliding window is to implement scalar rescan. We keep track of current minimum until it leaves the window. Then we need to do ”rescan” part which calculates new minimum from scratch. Additionally we perform deduplication of adjacent hashes, so if the minimum is unchanged, we don’t emit new minimizer. However if there’s new equal hash on different position, then we treat that as a new minimizer.

This approach is implemented by minimap2.

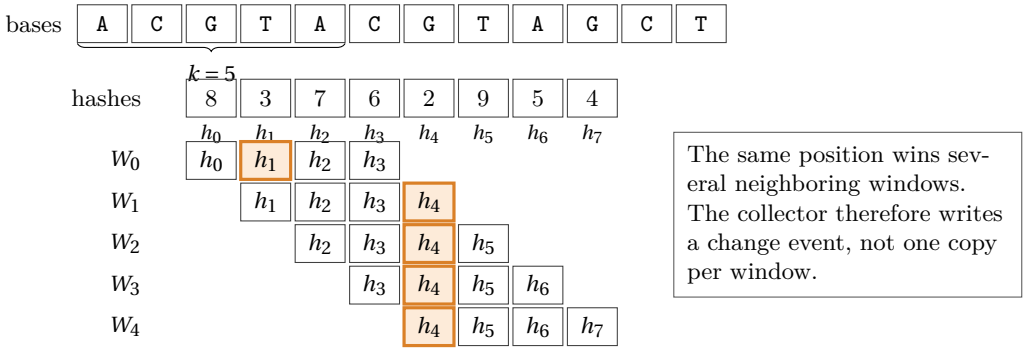


Figure 4.4: Minimizer extraction with illustrative $k = 5$, $w = 4$ hashes. Each row is one window of four adjacent k-mers. Orange marks the selected position. The hash value 2 remains selected while the window moves, so those windows do not require repeated output.

Equal hashes at different positions are not duplicate records. We wanmt to keep both to guarantee higher minimizer density.

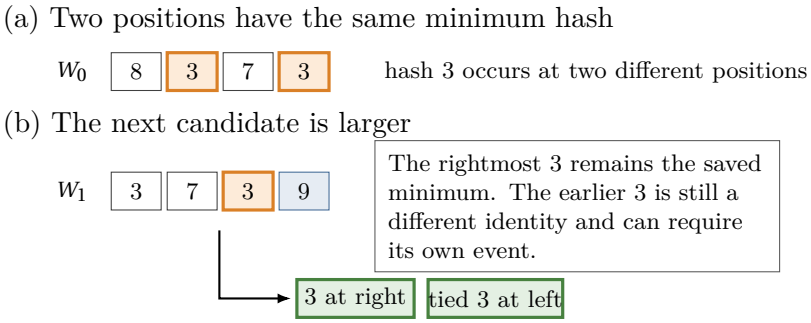


Figure 4.5: Why a hash value alone does not define the output. Both orange cells have hash 3, but they refer to different k-mer positions. Minimap2 keeps the rightmost equal position as its saved minimum and has explicit rules for writing other tied identities.

Minimap2 stores the most recent w candidates in a circular buffer It also stores the current minimum and the array slot containing it.

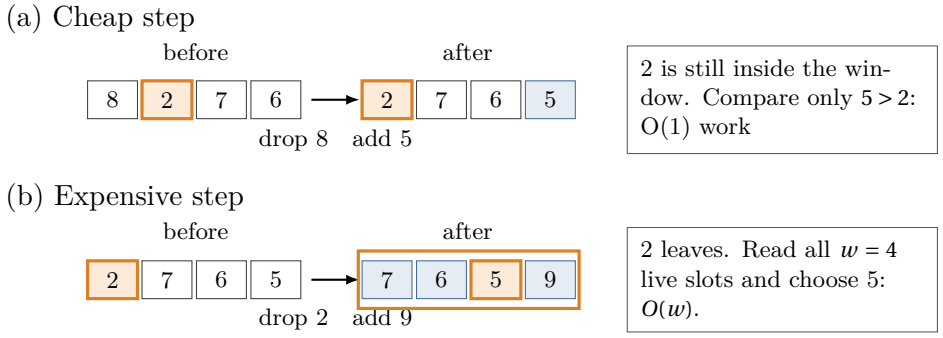


Figure 4.6: The two costs of one minimap2 window advance. Gray cells are already in the ring, blue is the entering candidate, and orange is the saved or newly found minimum. A normal step performs one comparison. When the saved minimum leaves, the algorithm reads the complete window again (rescan)

Minimap2 stores kmers and apply hash function each time from scratch.

Algorithm 3 Scalar minimap2-style rescan

1. Store k-mer binary representation and its reverse complement and update them each step. Calculate hash for both of them and take the minimum.
2. Write this hash into the next slot of the w-slot circular buffer.
3. If new hash is less than or equal to the saved minimum, emit new minimizer.
4. Else, if the overwritten slot held the current minimum, emit new minimizer and scan all w slots from oldest to newest, and save the rightmost record having the smallest hash as new minimum.
5. At the end of the sequence, write the remaining saved minimum from buffer as final minimizers.

4.3. A1: Naive SIMD rescan

Let's assume we have scalar linear iteration over data. The first though is to unroll the loop so each element of a SIMD vector corresponds to one iteration. This approach however works only if each iteration step is independent. If we have any condition that requires knowledege of results of previous step the SIMD vector's elements will be dependent on each other and won't give us any advantage. The other approach would be to threat each element not as consecutive input items, but as separate entities handling different parts of the input. Each lane will handle separate parts of iteration, so we will avoid data dependency of vector's elements on each other and replace it with the dependency between current state vector and previous state vector. In this document, I will define a SIMD lane as one numbered element of a vector register.

Each SIMD lane will load fragments of input data:

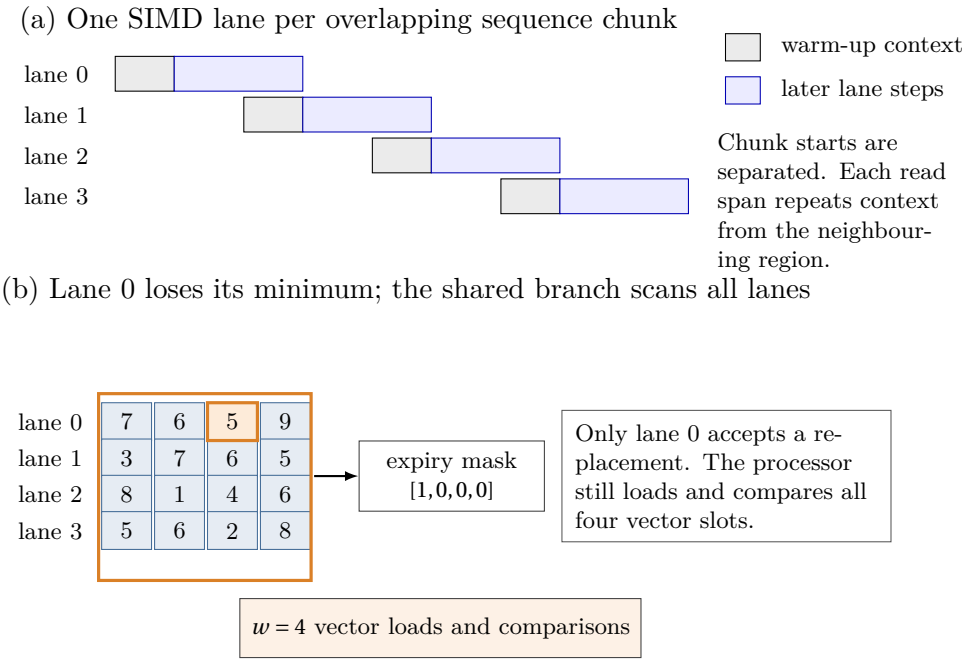


Figure 4.7: Why direct SIMD rescan is costly. Each lane follows a different sequence piece. A nonzero expiry mask enters one shared loop over all w vector slots. Masking protects lanes 1–3 from changing their answers, but it does not remove their loads and comparisons.

Note that algorithm fundamentally stays the same, we only now divided input sequence into chunks and translated each operation (hashing, comparisons) into SIMD instructions:

Algorithm 4 SIMD minimap2-style rescan, with the output cases stated plainly

1. Split input sequence into chunks each corresponding to the lane of SIMD vector. Each lane handles different chunk (chunks overlap slightly to make sure we don't omit any window)
2. Store k-mer binary representations and its reverse complements and update them each step for each lane. Calculate hashes for both of them and take the minimum for each lane.
3. Write all hashes for each laneinto the next slot of the w-slot circular buffer (this buffer now stores SIMD vectors instead of single hashes).
4. If the new hash vector is less than equal to the minimum vector element-wise, extract corresponding elements and emit new minimizers.
5. Else (new hash vector is greater then minimum vector on each position), if the overwritten slot held the current minimum for any lane, emit new minimizers and scan all w slots from oldest to newest, and save the new minimums vector.
6. At the end of the sequence, emit all remaining minimums as final minimizers.

As you can see this algorithm works well only if new hash values are greater than current minimum on each position. If not, we still have branching. This is problematic as vectorized algorithms works well if the branching is minimal and CPU pipeline isn't penalized for wrong branch prediction. Even if we assume the predictor is perfect, we still do expensive rescan for each lane if at least one needs to be updated. Assume one lane needs a rescan with probability

$1/w$, then a group of L independent lanes enters the shared slow path with probability

$$1-\left(1-\frac{1}{w}\right)^{L}.$$

For $w=10$ and $L=4$, this is about 34.4%. This means that we will need to duplicate work much more often than in scalar case.

4.4. A2: Two-stack algorithm

In different approach we can preserve prefix and suffix minima to calculate the window minima wrom both of them. SIMD-minimizers paper introduces this approach as well as usage of rolling hashes (nthash) instead of calculating hash each time from scratch. Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast"

4.4.1. One window minimum from two saved values

Split the hash stream into blocks of w values. Within each block, let P_i be the smallest value from the block start through position i, and let Q_i be the smallest value from position i through the block end. A window starting at j and ending at $e=j+w-1$ crosses one block boundary and has minimum

$$\min(Q_j, P_e).$$

The suffix values are recalculated in a fixed w-step loop each w steps. This means that we still have "rescan" part, but it becames unconditional.

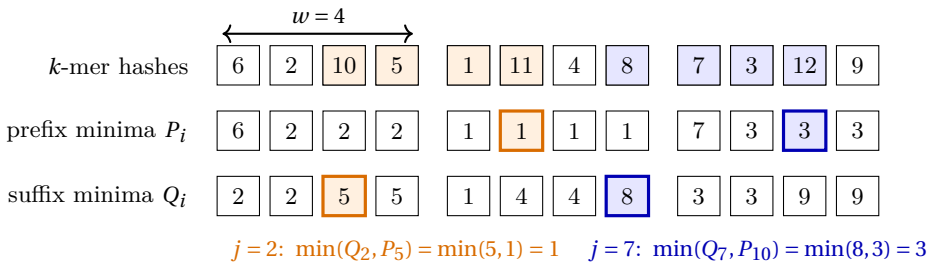


Figure 4.8: Sliding minima from blockwise prefixes and suffixes. Blocks have the same width as the minimizer window. The two shaded crossing windows each use one suffix value from the starting block and one prefix value from the ending block.

Fixed-width SIMD vectors

The SIMD-Minimizers use fixed-vector size architecture, where the SIMD vector is set to be uint32 times 8. The split per lane is the same as discussed previously. On Neon architecture only 128-bit-wide register is available, so the 256 fixed vector generates separate store intructions.

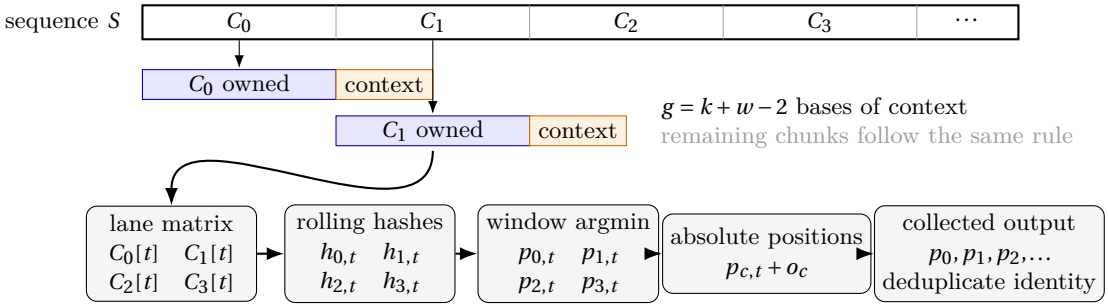


Figure 4.9: Vectorized minimizer extraction over logical chunks. Each chunk owns a disjoint range and reads enough context to complete its boundary windows. Lane-wise rolling hashes and minima produce local positions. Adding each chunk origin and deduplicating full identities yields a global sketch. The symbolic chunk size is an execution parameter rather than part of minimizer semantics.

Note that in this setup we can store full minimum vectors and later use matrix transposition to deduplicate them (in case that minimum stays the same over window): At first, one SIMD vector contains the selected positions for several far-apart streams at the same step. After several steps, the SIMD-Minimizers paper algorithm rearranges that output matrix so one row contains consecutive windows from one stream. Consecutive windows often select the same position. Once those positions are adjacent in a row, the paper compares neighbors and keeps only positions that changed. Finally, when the ordered stream results are joined, it removes the repeated boundary position caused by the overlapping input pieces.

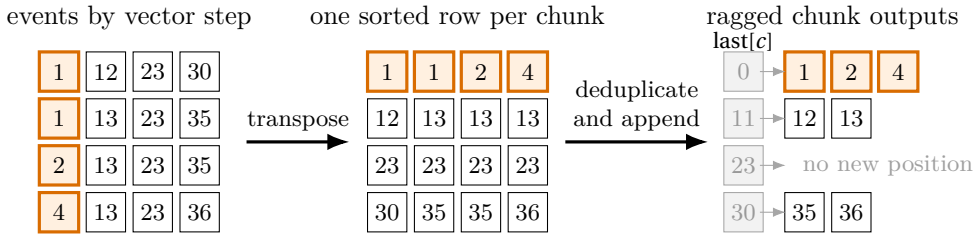


Figure 4.10: Transposition and boundary deduplication for four logical chunks. The input rows are vector steps and the columns are chunks. Transposition makes each chunk a sorted row. Each row has its own previous-position carry, shown in gray. Only new positions are appended, so padding cells are not part of the output.

4.5. A3: Chronological SIMD

Looking at A1 we observed that the main problem was that far-apart sequence chunks can have different characterisistics which cause us to do lots of additional work. Instead of splitting sequence that way, we can work on just one part of neighbouring positions.

We will try to calculate minimums for all local windows i.e calculate result for $X[p, p+1, ..., p+k+w-1]$ (this is window of size w for kmers of size k). Physically we only have L SIMD lanes so we will call that a group and advance only by L positions at the time (i.e from $X[p, p+1, ..., p+k+w-1]$ to $X[p+L, p+L+1, ..., p+k+w-1+L]$).

Potentially we can have more than one group per call. It offers some advantages discussed later. If the group number is higher, then we just execute their operations serially and perform result emission only after all groups execute.

The idea for one group is as follows:

1. We load L new sequence positions into L SIMD lanes each one is shifted by 1 using load instructions

2. For each lane we compute minimum along with strand direction and information which position it is. We call that new candidates: $x_0, \ldots, x_{L-1}$

3. We iterate through all candidates i.e $D \leftarrow T \parallel (x_0, \ldots, x_{L-1})$ where T are previous $w-1$ candidates and calculate new minimums. This gives us M which are L minimums for entire parts of sequence:

$$\begin{aligned}\{&\mathsf{Seed}(X[p, p+1, \ldots, p+k+w-1]),\\ &\mathsf{Seed}(X[p+1, p+2, \ldots, p+k+w]), \ldots,\\ &\mathsf{Seed}(X[p+L, p+L+1, \ldots, p+k+w+L])\}.\end{aligned}$$

   Now this along last previous result c^* forms P: $\mathbf{P} \leftarrow (c^\star, M_0, \ldots, M_{L-2})$

4. For each lane ℓ we compare P_ℓ with x_ℓ. If $h(x_\ell) \le P_\ell$ then P_ℓ is a new minimizer and we emit it. Otherwise if $\mathrm{pos}(P_\ell) = \mathrm{pos}(x_\ell)$ then we see P_ℓ expired so we emit it.

5. If the expiry happens then we go through all w windows and check if their minimum has the same value but different position. This only requires checking $m_0, m_1, ..., m_{w-1}$. If yes then we do slow path: extract the ties in order (hence the name of the algorithm: chronological)

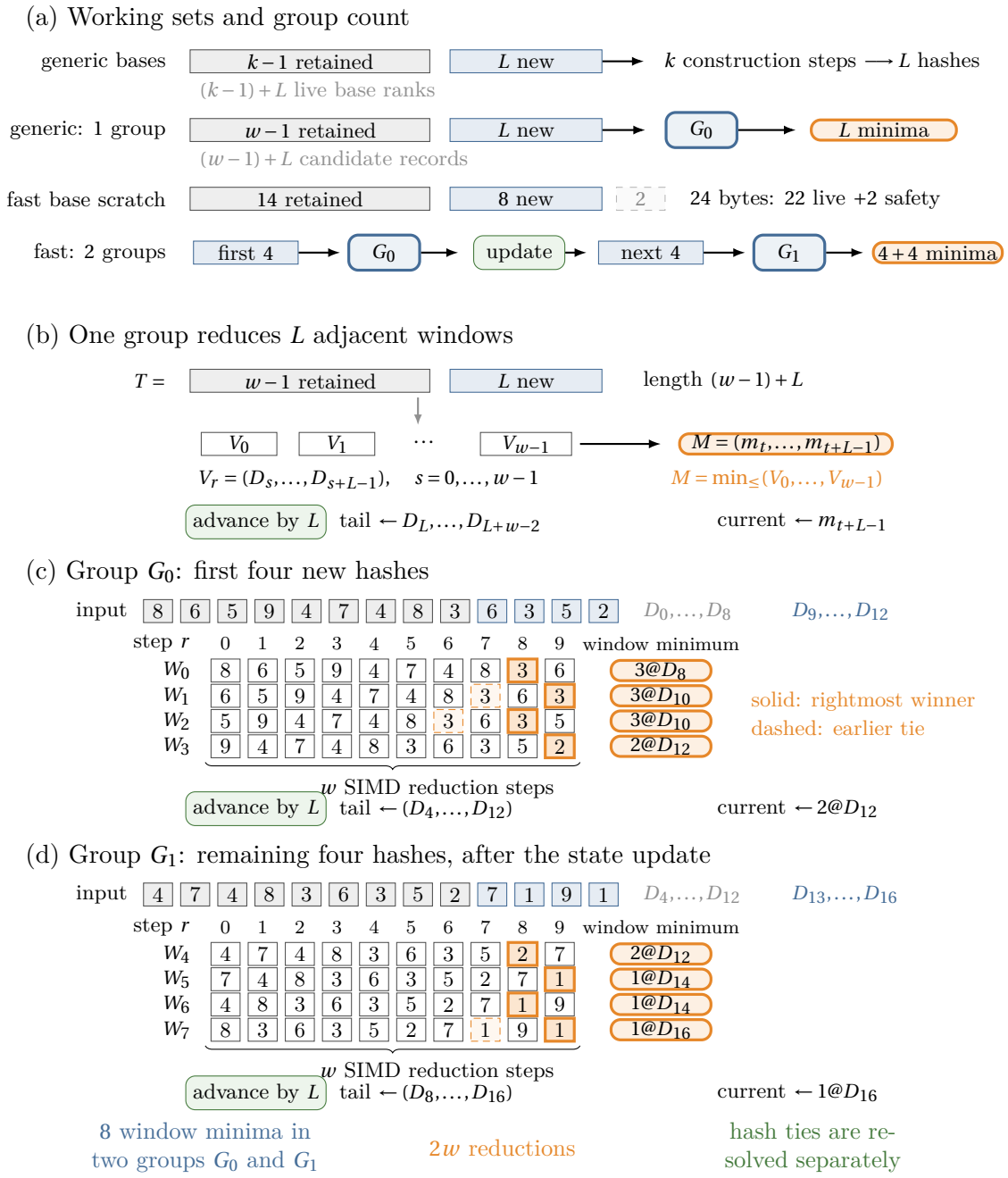


Figure 4.11: Working sets and execution of the chronological adjacent-window kernel. One group combines previous $w-1$ candidates with L new candidates, then loads w shifted vectors, and reduces them componentwise to the rightmost minima of L adjacent windows. Orange cells represent selected minimums (candidates for potential minimizer emit). A solid orange cell is the rightmost equal minimum selected by the source's $\leq$ comparison and a dashed cell is an earlier equal minimum.

The figure 4.11 illustrates calculation of window minima for $L = 4$ lanes: $x_0, x_1, ...x_{L-1}$. The entire algorithm looks as follows:

Algorithm 5 One iteration: build L k-mers and rescan L adjacent windows

Carry: C = last $k-1$ bases, T = last $w-1$ candidates, $c^\star$ = preceding window minimum
while the next L candidate bases are available do
  $U \leftarrow$ next L bases
  $Y \leftarrow C \parallel U$
  $\mathbf{F}, \mathbf{R} \leftarrow \mathbf{0}$
    Build forward/reverse kmers symbol by symbol.
  for $t = 0, \ldots, k-1$ do
    $\mathbf{z} \leftarrow \mathrm{load}_L(Y[t \,..\, t+L))$
    $\mathbf{F} \leftarrow F \parallel z$
    $\mathbf{R} \leftarrow \overline{z} \parallel R$
  Calculate hashes from kmers.
  for all lanes ℓ do
    $a_\ell \leftarrow H(\min(F_\ell, R_\ell))$
    if $F_\ell = R_\ell$ or $a_\ell = h_{\max}$ then
      $x_\ell \leftarrow \infty$
    else
      $x_\ell \leftarrow \mathrm{pack}(a_\ell, \mathrm{position}_\ell, [R_\ell < F_\ell])$
  $D \leftarrow T \parallel (x_0, \ldots, x_{L-1})$
  $\mathbf{M} \leftarrow \infty$
    Calculate M, which holds the rightmost minima of all w windows.
  for $s = 0, \ldots, w-1$ do
    $\mathbf{V} \leftarrow D[s \,..\, s+L)$
    $\mathbf{M} \leftarrow \mathrm{select}(h(\mathbf{V}) \le h(\mathbf{M}), \mathbf{V}, \mathbf{M})$
  One step = three field loads + one compare + three selects.
  Hence M_ℓ is the rightmost minimum of the complete window ending at x_ℓ
  $\mathbf{P} \leftarrow (c^\star, M_0, \ldots, M_{L-2})$
  for all lanes ℓ do
    $e \leftarrow \emptyset$
    if $x_\ell = P_\ell$ then
      $e \leftarrow \mathrm{event}^{\mathrm{NEW}}(P_\ell)$
    if $\mathrm{pos}(P_\ell) = \mathrm{pos}(x_\ell)$ then
      $e \leftarrow \mathrm{event}^{\mathrm{EXPIRED}}(P_\ell)$ expiration
    if e is Expired then
      Check all w windows for an equal hash on a different position
      $e \leftarrow \mathrm{event}^{\mathrm{TIE}}(P_\ell)$
    if e is Tie then
      Reread the w records only to construct ordered output slow path
  $C \leftarrow Y[L \,..\, L+k-1)$
  $T \leftarrow D[L \,..\, L+w-1)$
  $c^\star \leftarrow M_{L-1}$
if $0 < q < L$ bases remain then
  Run the same body once with q active lanes and sentinels elsewhere
  Both loops still execute k and w steps
  $C \leftarrow Y[q \,..\, q+k-1)$, $T \leftarrow D[q \,..\, q+w-1)$, $c^\star \leftarrow M_{q-1}$

4.6. Bioseq prototype

We established different approaches to calculate minimizers in previous sections.Now we want to discuss practical considerations when it comes to implementing those algorithms. The work started by exploring minimap2 source code and understanding its behaviour. The first step was to port minimap2 code into modern C++. This was required to have a stable code where

we could test and benchmark different algorithms. I encountered several key problems that lagged implementation by considerable number of months:

1. Minimap2 code uses direct C arrays access for lots of different data structures. Minimizers are encoded as uint128 type represented by a struct with two uint64 fields with abstract filed names. Circular buffers are represented by single raw pointer and require macros for any operations. The encoding and semantics of fields can sometimes change even though underlying uint128 representation stays the same (it's the same field name). This complicates any manual static analysis of the code I conducted.

2. Minimap2 uses raw instrinsics for SIMD operations which makes translation to generic SIMD very tedious.

3. Some parts of minimap2 hash logic are undocumented either in official paper or in-code.

The porting involved several iterations over the minimap2 source to understand how index is split, how parallelization works and how exactly minimizers are represented. After achieving runnable pipeline (with seeding, chaining, alignment and estimator) I started to work specifically on the minimizer extraction part as it's first in the pipeline.

Label	Main change What it means for the minimizer algorithm
V1–V6	naive SIMD rescan approach equivalent to A1
V7–V8	hybrid algorithm A1 where we use two-stack for minimum calculation
V8.5	defer minimap2 hashing to later stages
V9-V10	hand-written input and matrix loop
V11	two independent Highway lane groups
V12	force the per-base body inline
V13	fixes of estimator

Table 4.1: History of the first prototypes

4.7. Seeding prototype V1–V6: Naive SIMD rescan

The first approach (apart from direct minimap2 scalar rescan already existing in the ported code) was implementation of A1 SIMD rescan.

As correctly predicted our benchmark (Table 4.2) showed that the scalar `mm_sketch` took 12.5 ms on a 903 kbp pair, while the "vectorised" `mm_sketch_simd` took 24 ms – a 2× slowdown. This was caused by the fact that any minimum going outside the window causes rescan of all lanes.

Phase	scalar (ms)	V6 SIMD (ms)	V6 over scalar
`mm_sketch`	12.50	14.94	+19 %
`mm_sketch_simd`	0	23.88	—
`target_index_build`	16.0	14.3	−11 %
`collect_seed_hits`	6.5	11.0	+69 %
`mg_lchain_dp`	16.0	0.0	— (replaced)
`mg_lchain_simd`	0.0	8.9	—
`mm_est_err`	0.7	7.2	+929 %
`mm_align_skel.`	87.5	86.4	−1 %
Pipeline total per pair	158.4	170.7	+8 %

Table 4.2: Phase breakdown at 903 kbp, V3 baseline – pre-SIMD-Minimizers work. The naive SIMD path (V6) is +8 % slower than scalar despite four lanes of parallelism. The slowdown is split between three regressions: (a) the SIMD minimizer's `mm_sketch_simd` is itself slower than scalar `mm_sketch`; (b) the SIMD chainer feeds anchors into `collect_seed_hits` that hit a worse code path; and (c) the SIMD chain anchors are not cleanly indexable into the sorted query-minimizer array, so `mm_est_err` explodes. The V13 work mostly fixes (a) and (c).

The downstream regressions on `collect_seed_hits` and `mm_est_err` were caused by SIMD chainer producing a slightly different anchor distribution than scalar. This problem is orthogonal to the minimizer extraction and we mention it later.

4.8. Seeding prototype V8: Implementing A2 algorithm

The single most important external input to this work was the SIMD-Minimizers paper of Groot Koerkamp & Martayan Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast." Its central insight is that the rescan method is fundamentally data-dependent, and that any data-dependent branch costs more in SIMD than in scalar because lanes' branch targets are tied to each other.

Algorithm 6 sketches the V8 inner loop.

Algorithm 6 V8 `collect_minimizers_two_stack_simd` (per-step body)

state: prefix_min[L], ring[w][L], step, ring_idx
hash ← `rolling_kmer_hash`.Next($d, c_{\text{in}}, c_{\text{out}}$)
strand, palindrome ← StrandMask(hash)
valid ← ¬palindrome
hs ← IfThenElse(strand, hash | 1, hash & ~1) strand bit packed in LSB
elem ← IfThenElse(valid, ((hash & 0xffff_0000) | pos_local), 0xffff_ffff)
ring[ring_idx] ← elem single ring buffer slot per lane
better ← elem < prefix_min
prefix_min ← IfThenElse(better, elem, prefix_min)
ring_idx += 1, step += 1, pos_local += 1
if ring_idx = w then rebuild suffix
    ring[i] ← min(ring[$i..w-1$]) for all i $O(w)$ uniform branch
    prefix_min ← elem
    ring_idx ← 0
window_min ← min(prefix_min, ring[ring_idx])
if step ≥ $k+w-1$ and window_min.pos ≠ last_emit_pos then
    emit window minimum to per-lane scalar collector
    last_emit_pos ← window_min.pos

The V8 numbers were already encouraging: `mm_sketch_simd` dropped from V6's 24 ms to 17 ms at 903 kbp – a 30 % reduction on the minimizer-only phase, which translated into the SIMD pipeline being only 1.4 % behind scalar overall (down from V6's 8 % behind).

Note that in this prototype we weren't using nthash rolling hash, but the original minimap2 hash function.

4.8.1. Seeding prototype V8.5: Deferring hash calculation

Till this time we were using minimap2 hash function for minimizers2, but the observation was that we can use cheaper hash function for ordering values inside the inner loop and then hash the emitted values. If the hash function used will be significantly cheaper we can have proper speedup. The cheap inner hash needs only one property: be a bijection on the $2k$-bit kmer space, so that $\text{lex}(\text{kmer}_1) = \text{lex}(\text{kmer}_2) \iff \text{kmer}_1 = \text{kmer}_2$. We use the "lexicographic-canonical hash" – the bit-packed min(forward, rev_compl) but multiplied by Knuth's golden-ratio constant $0x9E3779B9$ Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching.

The arithmetic of the saving:

$$\underbrace{2.25 \cdot 10^5}_{\text{iters / Mbp}} \times \underbrace{(7\,\text{stages} - 2\,\text{stages})}_{\text{ops saved per iter}} \approx 1.1 \cdot 10^6\,\text{SIMD ops saved}$$

The measured saving was ~ 5 ms per 1 Mbp pair on `mm_sketch_simd`, taking the cost from V8's 17 ms down to 12 ms.

4.9. Seeding prototype V9–V11: Stream interleaving and forced inline

A close reading of the assembly created by Apple Clang LLVM revealed inner loop has unfriendly data dependency chain:

`shift_fwd` → `or` → `and` → `Min(fwd, rc)` → `Mul(lex, golden)`.

The Apple M1 has four 128-bit NEON pipes that issue independently, so a single 4-lane stream wastes three out of four pipes for any given cycle of the dependency chain. This feature is called ILP (Instruction Level Parallelism).

This is limitation that wasn't observed in the original SIMD minimizers paper as Rust code used fixed-size SIMD vectors, where we use adaptive vector size equal to possible physical register size. The possible optimisation here is to keep two independent 4-lane streams in flight simultaneously and interleave their operations in source order:

```
shift_fwd_lo, shift_fwd_hi = ShiftLeft(state_fwd_lo / hi)
state_fwd_lo, state_fwd_hi = And(Or(shift_fwd_, bp_), kmer_mask)
state_rc_lo,  state_rc_hi  = Or(ShiftRight(state_rc_), new_high_)
lex_lo,       lex_hi       = Min(state_fwd_, state_rc_)
hash_lo,      hash_hi      = And(Mul(lex_, golden), hash_mask)
elem_lo,      elem_hi      = IfThenElse(valid_, hash_|pos_, max)
```

We chose 2 streams rather than 4 execution stream because the per-stream state plus the shared ring buffer for $w = 10$ already consumes ~ 26 NEON registers per stream. Doubling it could make the ring buffer spill to stack losing the ILP benefit.

The wall-clock effect at 903 kbp was a ~ 4 ms reduction on `mm_sketch_simd` (from 12 ms to 8 ms); Figure 4.12 plots the full evolution.

4.9.1. Seeding prototype V12: Forced inline

V11 was good but disappointing: the wall-clock saving was less than the theoretical 2× ILP win we expected. `xcrun llvm-objdump -d build/bioseq/bioseq_bench` on the V11 wrapper revealed the underlying problem.

```
ldp q0, q1, [sp, #0x150]       // load bp_row from stack
ushr.4s v2, v1, #0x2           // extract per-BP value
movi.4s v1, #0x3
stp q0, q2, [sp, #0x150]       // store back
and.16b v0, v0, v1
and.16b v1, v2, v1
add x0, sp, #0x5, lsl #12      // prepare lambda's captured-this
add x0, x0, #0x770
movi.2d v2, #0xffffffffffffffff
add x1, x22, #0x8
bl 0x1002acca4 <step_one_bp_pair lambda>   // the offending call!
```

Despite the per-step body being decorated with `[[gnu::always_inline]]`, Clang lowered the lambda as a function call at every BP step. Each call paid:

- Function-call prologue/epilogue (`stp x28-x30 [sp,...]`).
- Captured-state spill/reload via stack – every per-stream state register was loaded from the captured-this pointer at the start of each lambda invocation, and stored back at the end.
- No ILP across iterations – each lambda call is its own function frame, so dependent state spills to memory between consecutive iterations.

Figure 4.12 shows full evolution of the prototypes. The scalar part involves unavoidable SIMD overhead of collecting results into map, scalar scramble hashing (minimap2 hash) and encoding output into the bit structure.

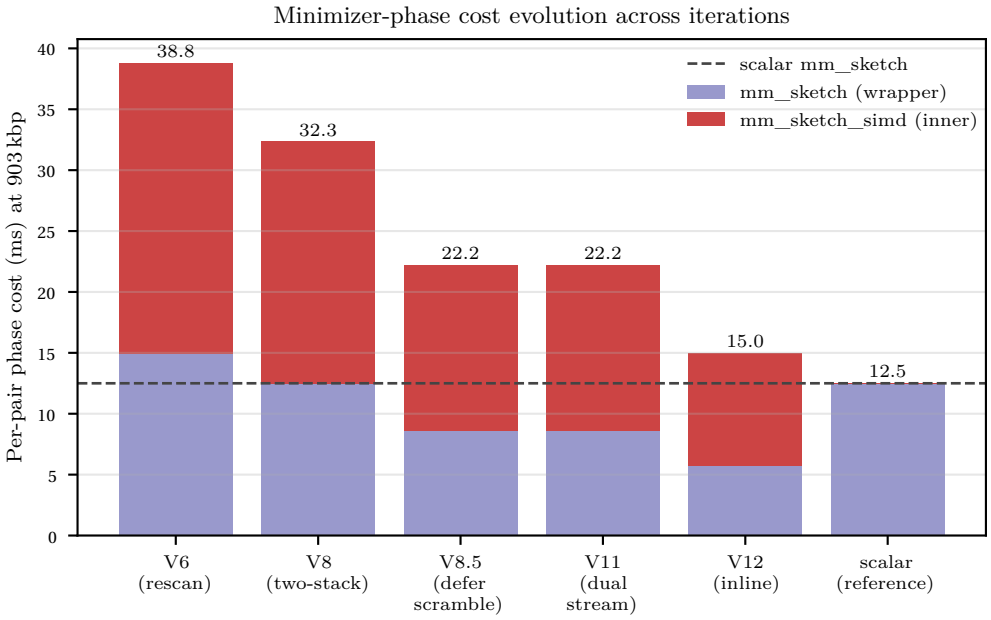


Figure 4.12: Evolution of the minimizer-phase cost (`mm_sketch` wrapper plus `mm_sketch_simd` inner) at 903 kbp across the iteration rounds discussed inuujj §4.7–§4.9. Numbers are means of three runs at `-O3 -DNDEBUG`. V8 is the two-stack port, V8.5 defers the minimap2 hash to emit time, V11 adds dual-stream interleaving, and V12 forces inlining of the per-step body via a free-function template. The dashed horizontal line is the scalar `mm_sketch` baseline. Even with some optimisations still SIMD routine is slower than scalar execution.

4.9.2. Seeding prototype V13: Existmator fix

For each chain, minimap2 estimates the per-base sequence divergence from a Poisson model on the minimizer-set intersection Li, "Minimap2: pairwise alignment for nucleotide sequences":

$$\hat{\epsilon} = \frac{1}{k} \log \frac{n}{m}$$

where n is the number of query minimizers in the chain's reference span and m is the number of those that are actually seeded (i.e. that appear as anchors in the chain). Computing n requires knowing which positions are query minimizers, which is a set membership test against the query's minimizer-position set.

The divergence estimator is used to measure the quality of the seeding. If the divergence is too high, the seeding is not good and the chain is not valid.

When porting minimap2 code we had hidden one important detail visible on table 4.2. The `mm_est_err` section had significant 9-times slowdown. This is because we are using separate chunks for each lane, but the output minimizers were stored in the buffer. Minimap2 normally receives its `mini_pos` array already sorted (because of how scalar loop works). This means that in our case we did the most naive approach:

1. Perform `std::sort` on the array to sort minimizers by positions
2. Perform `std::lower_bound` on the array to test membership

This caused severe slowdown, which we mistakenly attributed to seeding algorithm itself. More on divergence is described in the postprocessing sections.

Figure 4.13 stacks the per-phase cost at 903 kbp for V12-pre versus V13. The two large gains are `mm_est_err` (from 7.88 ms to 0.85 ms) and the V12 `mm_sketch` stack (from 14.99 ms to 9.23 ms, already covered in previous section.

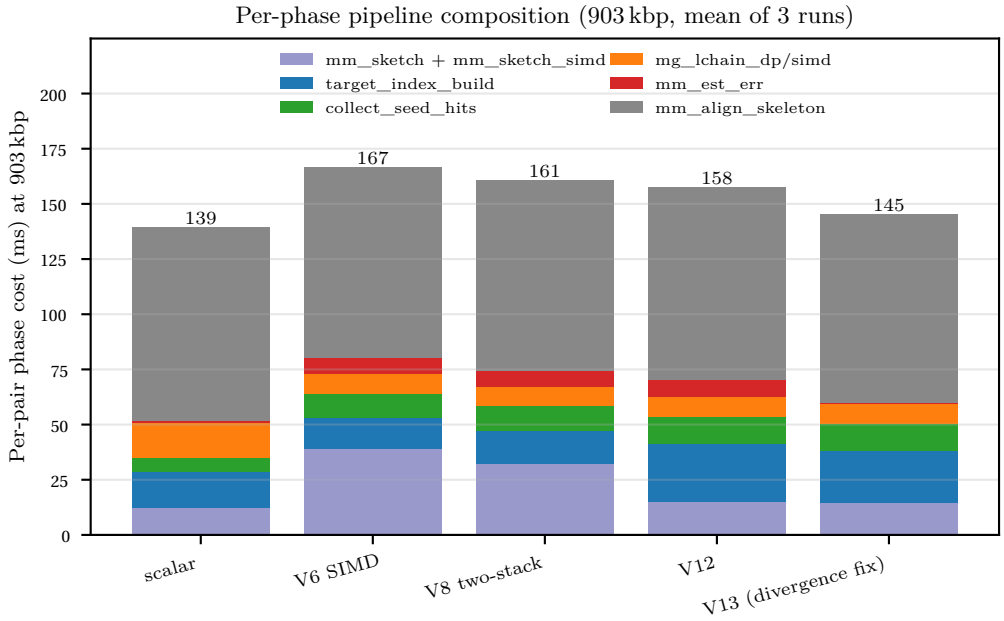


Figure 4.13: Per-phase composition of the per-pair pipeline at 903 kbp we can see that V13 reduced exploded divergence phase

4.9.3. Seeding prototype V14: directly use nthash

So far we utilised minimap2 hash function directly. One other approach was replacing this hash function directly. This should have given us measurable speedup. For nthash we can construct LUT (lookup table) accessed via SIMD instruction.

- `mm_sketch_simd` cost increased by 2 ms (the table lookup used NEON `tbl.16b` instruction and hashing required two rotates, and some numbers XORs per stream per sequence symbol).
- `collect_seed_hits` dropped by 0.4 ms (this isn't significant enough).

My investigation pointed me to the fact that I was trying to optimise incorrect region of computations. The benchmark during all experiments evolved and improved with each iteration. This time the measurement didn't report that most of the time was spent on inserting minimizers into the index.

The gap in performance didn't come from the minimizer hash distribution (or hashing speed itself) but from the usage of `std::unordered_map`. The prototype's target index used STL's default hash table, while the C minimap2 path uses an open-addressing `khash_t<uint64_t>`. This shows another problem with directly porting minimap2 into C++. `std::unordered_map` is known problem with standard library that we overlooked during implementation similarily to how `std::sort` was used for divergence estimation. The cleanest solution here was implementing indexing structures as flat hash maps directly.

4.9.4. Seeding prototype V15: Smaller minimizer space

In all of the experiments we used adjustable minimizer represetations with hash width of 16 bits (minimizer representation is explained in separate section). However smaller represetations mean more efficient algorithms. Existing iterations of seeding algorithm all used adjustable SIMD vector widths provided by C++ Highway library. This means that narrower input type will automtically increase number of available lanes L thus increasing potential performance in all algorithms. However having smaller type means that we need to either get rid of strand information (1 bit) or use narrower position representation (this is not always possible) or use narrower hash representation. We conducted early experiment testing V12 prototype with narrower hash of 14 bits.

	V12 (pre)	V12 (post-Goal-2)	delta
mm_sketch_simd	9.0 ms	9.5 ms	+0.5 ms (insignificant change)
target_index_build	24.7 ms	4.9 ms	−19.8 ms (~85% fewer keys)
collect_seed_hits	12.3 ms	842.0 ms	+830 ms (70x regression)
mg_lchain_simd	8.9 ms	299.9 ms	+291 ms (34x regression)
mm_align_skeleton	87.2 ms	4811.0 ms	+4724 ms (56x regression)

Hash-space collision amplification

The kmer space stored in `target_index` hash buckets collapsed to ~16000 distinct keys for 168000 query minimizers. Each query had ~10 collisions per bucket, meaning the chain-anchor count exploded, and the alignment had to walk through all of them. This means that we sped up index lookups significantly (less memory accesses required), but at the same time the more costly parts of the pipeline get more false positives that need to be verified.

4.9.5. Seeding prototypes summary

TODO

Cite this stack overflow answer: https://stackoverflow.com/a/25027750

The combination of the V12 inlinening fixes, the V13 estimator fix and replacing `boost::unordered_map` (marked as $V13^*$) yielded observable improvements. We didn't want to implement new hashing scheme at this time, only see the impact of using different hash storage.

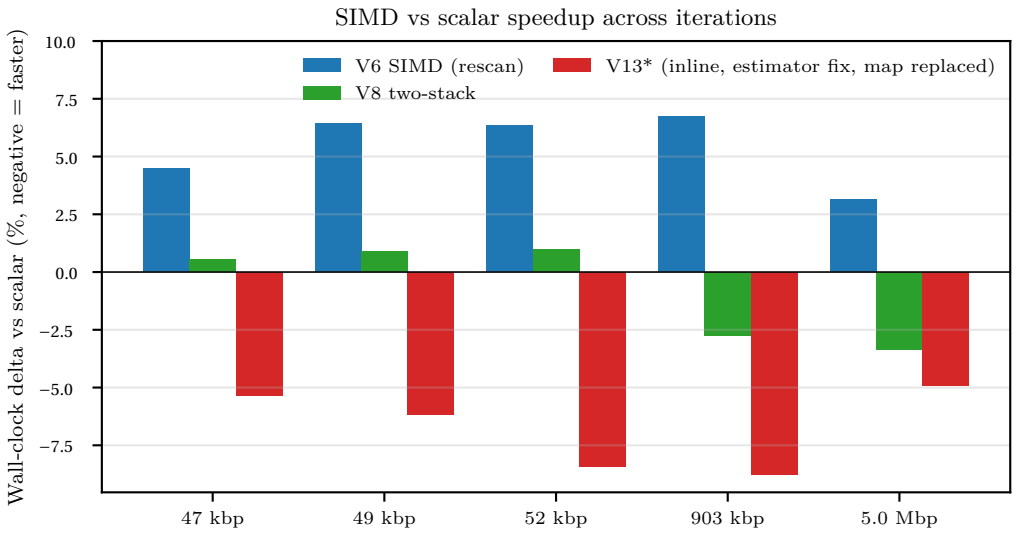


Figure 4.14: Wall-clock delta versus scalar for the three SIMD configurations, expressed as percent (negative = faster). $V13^*$ now performs better than scalar, however this is nowhere near results reported in SIMD Minimizers paper.

The summary table:

Length	scalar (ms)	V13 (ms)	V13 vs scalar	V13 vs V6 SIMD
47 kbp	14.56	13.78	−5.4 %	−9.4 %
49 kbp	14.94	14.02	−6.2 %	−11.8 %
52 kbp	15.09	13.82	−8.4 %	−13.9 %
903 kbp	188.93	172.38	−8.8 %	−14.5 %
5.0 Mbp	1171.0	1113.3	−4.9 %	−7.8 %

Table 4.3: Total measured time for alignment (mean of 3 runs from `-O3 -DNDEBUG`

4.10. GPU acceleration for seeding

G3SA Han et al., "G^{3}SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment" provides implementation of multiple useful kernels including one specific to minimizer generation phase.[1]

G3SA seeding kernel
mm_sketch_device 83:1

G3SA implements the A3 adjacent-window computation graph, in CUDA SIMT. Let $L = 32$ be the number of CUDA threads. For group $r \in \{0,1,2,3\}$ and thread $t \in [0, L)$, define

$$j^\star_{r,t} = \max \{ \arg\min \}_{0 \le j < w} h(C_{rL+t+j}), \qquad W_{rL+t} = C_{rL+t+j^\star_{r,t}}.$$

The outer maximum keeps the rightmost record among equal minimum hashes. The vector $(W_{rL}, \ldots, W_{rL+L-1})$ is the componentwise reduction of w shifted candidate vectors. This is the same exact computation as Algorithm 5, but CUDA threads replace SIMD lanes. One difference is that there's no some global state preserved so it's need to be recomputed at initialisation.

[1] Inspected code from official G3SA repository that supplies CUDA kernels along with CPU host-side code, https://github.com/sunwookim028/G3SA

The computation is divided according to the following geometry:

$$L = 32, \qquad G = 4, \qquad n_{\text{owned}} = LG = 128, \qquad n_{\text{shared}} = L(G+1) = 160.$$

We have 128 blocks of 32 threads. Block b handles reads $b, b+128, b+256, \ldots$, one read at a time. For each read it advances by $n_{\text{owned}} = 128$ window starts. In one advance, thread t attempts candidate slots $t, 32+t, \ldots, 128+t$ and then reduces the four windows beginning at $t, 32+t, 64+t$, and $96+t$. Figure 4.15 shows this transpose between candidate construction and window reduction.

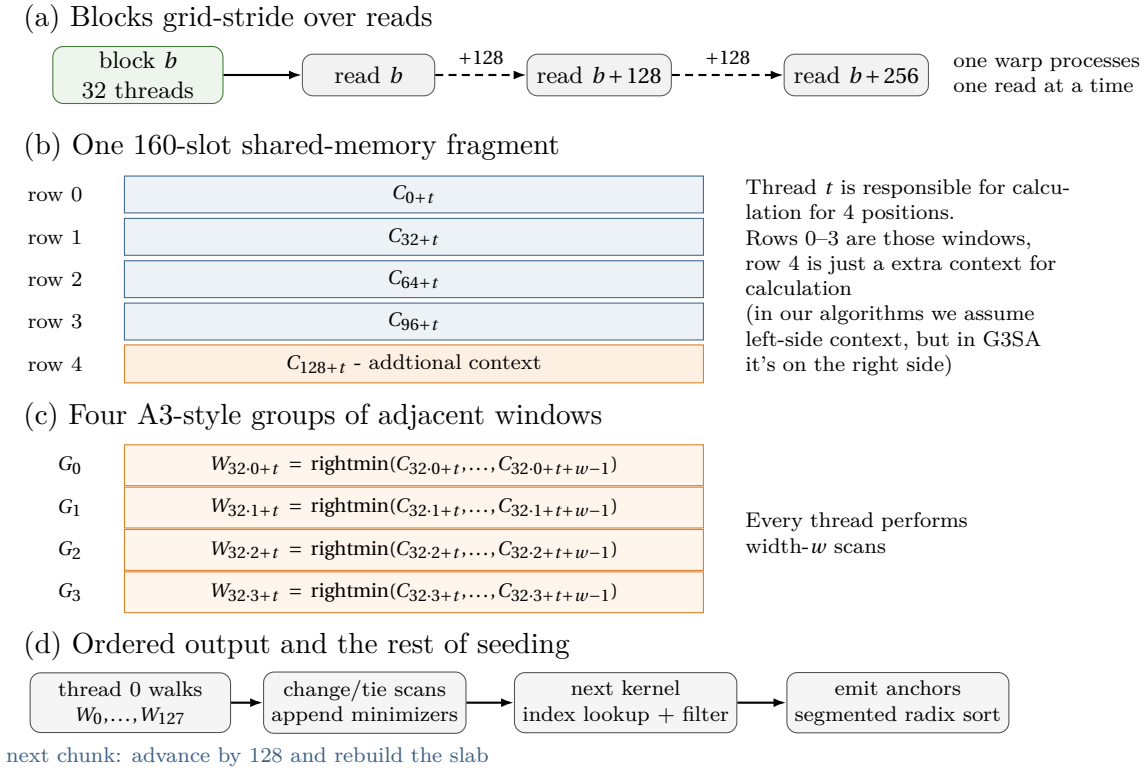


Figure 4.15: G3SA algorithm reconstructed from the CUDA code. This is similar to A3, but has specific work split per thread/warp (standrd CUDA SIMT model)

4.10.1. Sketch pseudocode and work

Here, PublishBlockWrites returns a block-scoped phase token. The token becomes ready only after every thread in the block has reached that phase and the named shared-memory writes are visible to every thread; awaiting it is therefore a collective rendezvous.

Algorithm 7 G3SA sketching CUDA algorithm

```
Require: concatenated reads S, offsets O, lengths N, index parameters k, w
shared Base[160], Cand[160], Winner[128]
procedure SketchBlock(b, t)   b is a block; t ∈ [0,32) is its thread
    for q ← b; q < |N|; q ← q + gridDim_x do
        p ← O[q], e ← O[q] + N[q], m ← 0
        while p < e do
            for r ← 0 to 4 in every thread do
                u ← 32r + t
                Base[u] ← (p + u < e) ? S[p + u] : NUL
                Cand[u] ← ∞
            baseReady ← PublishBlockWrites(Base, Cand)
            await(baseReady)
            for r ← 0 to 4 in every thread do
                u ← 32r + t
                Cand[u] ← GetHashedKmer(Base, u, k, p − O[q])
            hashReady ← PublishBlockWrites(Cand)
            await(hashReady)
            for r ← 0 to 3 in every thread do
                s ← 32r + t, best ← ∞
                for j ← 0 to w − 1 do
                    if Cand[s + j].hash ≤ best.hash and p + s + j + k − 1 ≤ e then
                        best ← Cand[s + j]
                if p + s + w + k − 1 ≤ e then
                    Winner[s] ← best
                else if N[q] < w + k − 1 and s = 0 and best < ∞ then
                    Winner[0] ← best   source short-read case
                else
                    Winner[s] ← 0
            winnerReady ← PublishBlockWrites(Winner)
            await(winnerReady)
            if t = 0 then
                m ← AppendChangesAndHashTies(Cand, Winner, w, O[q], m)
            p ← p + 128
        if t = 0 then
            minimizerCount[q] ← m
```

4.11. BioseqDB GPU seeding

The BioseqDB minimizer generation uses minimap2's scalar rescan approach ported to GPU. Specifically input is divided into tiles, where each invocation handles one tile using rescan algorithm.

When talking about GPU shaders, Vulkan's workgroup correspond to CUDa thread block, while subgroup is a direct equivalent of CUDA warp.

Let Q be the uploaded query batch and let $\mathcal{T} = (t_0, \ldots, t_{T-1})$ be its flat tile list. The ordered dispatch uses $G = \min(\lceil T/64 \rceil, G_{\max})$ workgroups, where $G_{\max}$ is the device's workgroup limit.

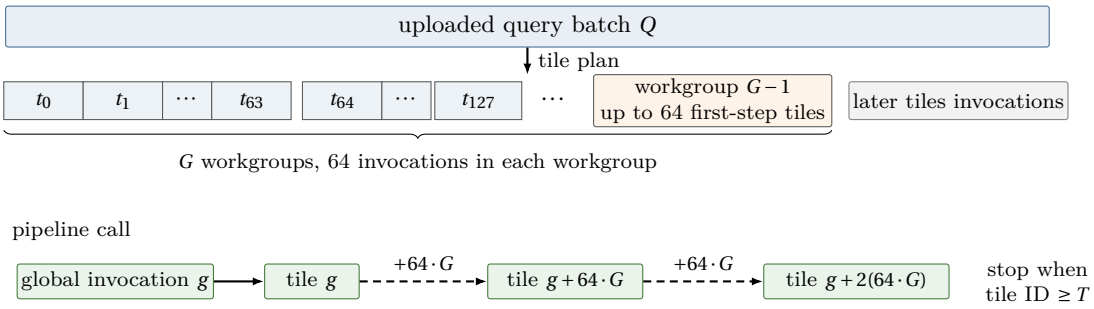


Figure 4.16: Tile assignment in an ordered-sketch dispatch. Each of the G workgroups has 64 invocations. Invocation g processes one tile at a time and advances by $64 \cdot G$ while another tile remains.

When we collect results we need to either: have fixed memory offsets per tile and perform another host pass to extract results or we can do lazy approach where we execute pipeline twice. First we use count mode to caculate where to place results and then do a second pass to actually write them. This makes pipeline do twice as much work, but avoids any additional host pass and we can directly use output results. The limtiations of our work is that we didn't measure the difference between these approaches or between G3SA approach.

The core part is OrderedSketchTile which runs the rescan algorithm in count mode and writes Count. The Count[j] is the number of output records for tile j. PrefixScanBlock reads Count. Each of its B single-invocation workgroups scans at most $S = 256$ entries. It writes local offsets into Offset and one total per block into Block. Two stages (calls) of PrefixScanAdd are used to calculate global offsets.

The positive value A is the pessimistic output capacity planned by the caller using heuristic. HostAndDevice creates an A-record device output before count. If the actual pipeline overflows, then we return error.

Algorithm 8 Vulkan scheduling in BioseqDB

Require: packed query batch Q, k, w, device group limit $G_{\max}$

Require: positive planned output capacity A

Require: policy $P \in \{\text{HostAndDevice}, \text{DeviceOnly}\}$

$\mathcal{T} \leftarrow \text{PlanTiles}(Q, k, w, G_{\max})$
$T \leftarrow |\mathcal{T}|$
return without a GPU dispatch if $T = 0$
$G \leftarrow \min(\lceil T/64 \rceil, G_{\max}), \quad S \leftarrow 256, \quad B \leftarrow \lceil T/S \rceil$
PrepareBoundDeviceInputs$(Q, \mathcal{T})$
allocate device buffers $\underline{\text{Count}}[T]$, $\underline{\text{Offset}}[T]$, and $\underline{\text{Block}}[B]$
if $P = \text{HostAndDevice}$ then $\mathit{Output} \leftarrow \text{HostCreateDeviceOutput}(A)$
BeginCommandBuffer
RecordHostToGPUBarrier
$\mathit{countTask} \leftarrow \text{RecordOrderedSketchTile}(\text{Count}, Q, \mathcal{T}, k, w, G, \mathit{Count})$
AwaitGPU($\mathit{countTask}$)
$\mathit{localTask} \leftarrow \text{RecordPrefixScanBlock}(\mathit{Count}, \mathit{Offset}, \mathit{Block}, T, S, B)$
AwaitGPU($\mathit{localTask}$)
$\mathit{blockTask} \leftarrow \text{RecordPrefixScanAdd}(0, \mathit{Block}, B)$
AwaitGPU($\mathit{blockTask}$)
$\mathit{offsetTask} \leftarrow \text{RecordPrefixScanAdd}(1, \mathit{Offset}, \mathit{Block}, T, S)$
if $P = \text{DeviceOnly}$ then
    SubmitAndHostAwait($\mathit{offsetTask}$)
    $N_{\text{out}} \leftarrow \mathit{Offset}[T-1] + \mathit{Count}[T-1]$
    return overflow if $N_{\text{out}} > A$
    $\mathit{Output} \leftarrow \text{HostCreateDeviceOutput}(N_{\text{out}})$
    BeginCommandBuffer
    RecordHostToGPUBarrier
AwaitGPU($\mathit{offsetTask}$)
$\mathit{emitTask} \leftarrow \text{RecordOrderedSketchTile}(\text{Emit}, Q, \mathcal{T}, k, w, G, \mathit{Offset}, \mathit{Output})$
SubmitAndHostAwait($\mathit{emitTask}$)
if $P = \text{HostAndDevice}$ then
    $N_{\text{out}} \leftarrow \mathit{Offset}[T-1] + \mathit{Count}[T-1]$
    return overflow if $N_{\text{out}} > A$
    $\mathit{HostOutput} \leftarrow \text{CopyMapped}(\mathit{Output}[0 \ldots N_{\text{out}}))$
else
    $\mathit{HostOutput} \leftarrow \varnothing$
return $(\mathit{Output}, \mathit{HostOutput}, N_{\text{out}})$

Algorithm 9 `RecordOrderedSketchTile` call

procedure RecordOrderedSketchTile($g, mode$)

Require: global invocation ID $0 \le g < 64 \cdot G$

Require: dispatch-wide $mode \in \{\text{Count}, \text{Emit}\}$

Require: bound $Q, \mathcal{T}, T, k, w, G, Count$
from Algorithm 8

Require: bound Offset and Output in Emit mode

```
j ← g
while j < T do
    localCount ← 0
    for all record e streamed by RunScalarRescanTile(Q, T[j], k, w) do
        if mode = Emit then
            Output[Offset[j] + localCount] ← e
        localCount ← localCount + 1
    if mode = Count then
        Count[j] ← localCount
    j ← j + 64 · G
```

4.12. Packed minimizer representation

When running BioseqDB minimizer generation we need to have a way to store the results in a format that can be resused in other stages of the pipeline.

Minimap2's `mm_sketch` emits each minimizer in an `mm128_t`, two `uint64_t` words named `x` and `y`. The C++26 declaration below shows only interpretation of the fields, while minimap2's itself uses binary shifts and masks to set and extract specific values (which made its manual code analysis very challenging).

```
struct MinimapSketchFields {
  std::uint64_t kmer_hash : 56;   logical k-mer hash
  std::uint64_t span      :  8;   input-base span
  std::uint64_t strand    :  1;   canonical orientation bit
  std::uint64_t last_pos  : 31;   zero-based final-base position
  std::uint64_t rid       : 32;   reference/query-segment ID
};
```

Minimap2's C code construction is done via:

$$x = (hash \ll 8) \mid span, \tag{4.4}$$

$$y = (rid \ll 32) \mid (last_pos \ll 1) \mid strand. \tag{4.5}$$

Normally span equals k (or is $< k$ for first $k-1$ bases). Minimap2 suppors homopolymer compression (HPC) togglable by command line option `-H` which collapses strings of one repeating character into single character occurence. If HPC is enabled, span can have values $0 \le span \le 255$. If span is increased beyond 255 (there are 255 same characters in the input sequence next to each other), then minimizer is completely abandoned until new symbol is found. In BioseqDB we use/or not HPC by default?

TODO

Find out if BioseqDB uses HPC.

The `rid` value identifies a reference during indexing and a query segment during query sketching. Note that the ID itself isn't useful during seeding, chaining or alignment, it's only useful at the end when reporting results.

When porting the core of minimap2 algorithms we observed the following facts:

1. The span contains no useful information as HPC is disabled by default and we don't have to support it at this time. For HPC disabled it can only carry information if the k-mer was at the start of the sequence (this can be checked via last_pos field)
2. rid is completely useless during most of the pipeline
3. last_pos which is the end position of the k-mer is only 31 bits which introduces requirement to split long sequences before using minimap2.
4. kmer_hash is wide, but using more bits for hash than binary representation of k-mer is not giving any advantage (for example for DNA4 alpohabet if we encode each symbol in 2 bits then k=15 requires 30 bits for k-mer representation. Hashing 30-bits into 56-bit wide hash doesn't give any advantage as huge part of hashing space won't be used)

BioSeqDB's minimizer producers and collectors use `minimizer_representation<L,K>`. Every form stores the hash, zero-based final position, and strand.

The longest form still uses span set to k, but it's not used for anything other than just padding of fields. Here L is the number of position bits and K is the k-mer length. This L is unrelated to the SIMD lane count used earlier in the chapter.

The type first reserves

$$H = \min(2K, 56)$$

hash bits. Let $R = H + L + 1$, where the last term is the strand bit. The type uses the first width $W \in \{16, 32, 64\}$ for which $R \le W$. Every bit above the position and strand fields then belongs to the hash, so the stored hash field has $W - L - 1$ bits. If $R > 64$, the record uses two 64-bit words. One word contains a 56-bit hash and an 8-bit span. The other word contains the position, the strand bit, and any unused high bits.

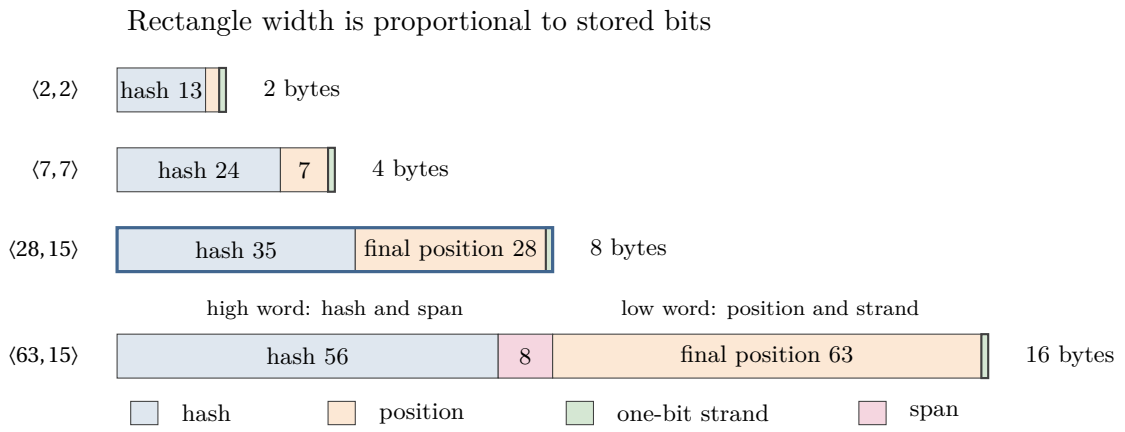


Figure 4.17: Bit fields in four `minimizer_representation<L,K>` instantiations, shown from high bits on the left to low bits on the right. Blue is the hash, orange is the zero-based final position, green is the one-bit strand, and purple is the span in the two-word form. The highlighted $\langle 28,15\rangle$ record is the default 64-bit sketch-data form.

The L-bit field represents final positions from zero through 2^L-1, so it can describe a local sequence span of at most 2^L bases. For $K=15$, values through $L=33$ still fit in 64 bits, while $L=34$ requires the two-word form. The $\langle 28,15\rangle$ form has a 35-bit hash field, but its minimap2 integer hash occupies only 30 bits.

Currently the main parameter that drives used representation width is the number of position bits L (this is compared against kmer_size_buckets array). Theoretically the template can produce a two-byte record, but this bucket isn't suppored at this moment (such small width make's it problematic to adjust as we need to have very short hash, which isn't ideal as shorter k-mers are more prone to collisions or limit the size of sequence to less than 256, which would be very specific optimisation) The $\langle 2,2\rangle$ on the figure is just the demonstrative example to show that template is supported even in very degenerated edge cases (we don't actually use special form for sequences with less than 4 bases as it would make very little sense).

4.13. Configured CPU algorithms

The PostgreSQL setting `bioseq.aligner.minimizer.algorithm` chooses the minimizer implementation. Its default value is `scalar`. Table 4.4 maps the four CPU values to the algorithm families described earlier. Additionally we allow dispatching Vulkan kernel.

Setting value	Current CPU route
`scalar`	Scalar rescan with the minimap2 integer hash.
`simd_minimap2_reference`	A1 with the minimap2 integer hash for DNA4. Other alphabets use scalar rescan.
`bounded_nthash_simd`	A2 with 32-bit ntHash. The exposed route is limited to DNA4 with $k=15$ and $w=10$.
`two_stack_nthash`	Normally A3 chronological SIMD with the minimap2 integer hash.

Table 4.4: CPU minimizer implementations exposed by the PostgreSQL setting.

Limitations

- Scalar rescan with the minimap2 integer hash is the default algorithm.
- Bounded ntHash SIMD is limited to DNA4 with $k = 15$ and $w = 10$.
- We don't provide G3SA-style GPU algorithm or didn't measure performance difference between it and current rescan approach.

Position locality

In case of generating minimizers we can operate on local coordinates meaning that our minimizer representation can just track position relative to the sequence fragment. Sequence fragmentation in storage is described in later chapters. However for next steps of processing (chaining) we use global coordinates. It would be possible to take advantage of that locality for example by building index that stores only minimizers for some shorter sequence fragments (this is also described in later chapters), but for simplicity even in that case we just map local coordinates afterwards to global ones at all times. This have the cost of anchors being wider in memory (as we need to use int64 for global coordinates).

Chapter 5

Chaining seed anchors

5.1. From local seed hits to candidate mappings

When minimizers are generated and we take the intersection (by hash value) of the minimizer sets between the query and the reference, we get a set of seed hits which is encoded as anchors.

5.2. Anchor geometry and ordering

A seed hit is represented by an anchor

$$a_i = (r_i, q_i, \sigma_i, \ell_i, g_i, \phi_i),$$

where r_i and q_i are inclusive seed-end coordinates on the reference and query, σ_i is the strand, ℓ_i is the query span, g_i identifies a query segment, and ϕ_i stores additional ordering flags.

Below you can see the physical representation of the anchor in memory in minimap2 and in our implementation. We could potentially use adaptive representation similar to minimizers, but anchors are fewer than minimizers, so the benefit would be limited for additional complexity of C++ templates.

```
typedef struct {
  uint32_t n;                    Size of cr array (number of matches in the reference sequence)
  uint32_t q_pos;                Query position
  uint32_t q_span       : 31;    Query minimizer span in symbols
  uint32_t flt          : 1;     If the minimizer is filtered out (very frequently occuring in the refe
  uint32_t seg_id       : 31;    Query segment identifier
  uint32_t is_tandem    : 1;     Tandem repeat flag
  const uint64_t *cr;            Multiple reference coordinates per anchor
} mm_seed_t;
```

```
struct anchor {
  std::uint64_t ref_position;    Absolute reference coordinate
  std::uint64_t query_position;  Absolute query coordinate
  std::uint32_t is_reverse;      Reverse-strand flag encoded as zero or one
  std::uint8_t query_span;       Query minimizer span in symbols
  std::uint8_t flags;            Seed flags propagated by collection
  std::uint16_t segment_id;      Query segment identifier
};
```

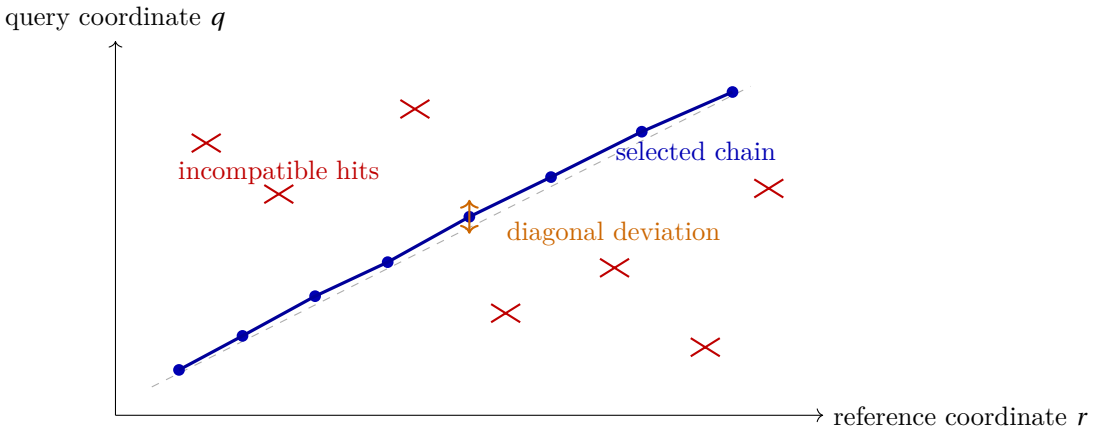


Figure 5.1: Chaining selects growing, approximately collinear path so we can get closer to the final result. Base-level alignment (later stage of the pipeline) will later resolve gaps between anchors.

TODO

Improve anchor API

Figure 5.1 shows viasual interpretation. Valid anchors that can be used for chaining lie below and to the left of the current anchor. A chain can move between nearby diagonals to represent an insertion or deletion, but large diagonal changes receive a penalty or are not considered at all.

Why we look for anchors that are colinear?

In the perfect scenario without gaps $r_i - q_i$ is constant. An increasing diagonal coordinate indicates that in reference we matched more symbols than in the query (meaning insertion/deletion has occured). Chaining needs to specify costs for abandoning the main diagonal too much (intuitevely we don't want the case in which we introduce very large gap in the reference sequence just to match one or two symbols in the query, because it's improbable that some very large fragment was replaced during evolutionary processes by just a few bases)

As we would like to have some well defined order of anchors, we sort them by strand and reference coordinate first, then by the additional metadata:

$$\begin{aligned} K_r(a_i) &= (\sigma_i \ll 63) \mid \big(r_i \,\&\, (2^{63}-1)\big), \\ K_q(a_i) &= (g_i \ll 48) \mid (\phi_i \ll 40) \mid (\ell_i \ll 32) \mid (q_i \bmod 2^{32}). \end{aligned} \tag{5.1}$$

Comparison is lexicographic in K_r, K_q, and the full q_i. The hashing doesn't need full 64 bits for query position i.e if query coordinates q_i exceed 2^{32} we still have correct order as the leftmostpart is still the fragment number g_i (fragments won't be larger than 2^{32}). Fragmentation of sequences is discussed later in the storage chapter.

The ordering is required, because the chaining wants to process anchors in sequential order (increasing query positions), while we build chains of anchors using reference positions.

5.3. Chaining components

We defined the anchors that are input to chaining pipeline stage, however chaining itself isn't a monolith. It can be viewed as composition of several steps.

> **Chaining pipeline**
>
> Although chaining can be decomposed into several steps that are dispatched to differrent functions, in BioseqDB this is an internal implementation detail and we don't allow users to alter for example which backtracking backend is used (although we could expose parts of chaining as composable components in the future)

Forward chaining (which is the core component) produces the score array f and predecessor array p. Second most important step is backtracking (as typically in case of DP algorithms which fill some state matrices) reads the sorted anchors, f, and p. The backtracking writes theflat output anchor array supplied by the caller, denoted by A (A can be viewed as type `anchor A`

).

The chaining can be fully decomposed into the following steps:

1. Forward chaining reads the sorted anchors and writes the temporary f and p arrays.
2. Backtracking reads these inputs and writes the accepted chain records plus full anchor copies in A.
3. Region generation converts each chain record into one alignment region while retaining A.
4. Region selection filters the regions while retaining A.
5. Base-level alignment reads the selected regions, their slices of A, the query and target sequences, and the alignment parameters. It fills the base-level fields in the regions supplying exact start/end coordinates, match qualities etc.

BioseqDB represents every chain record with the following structure:

```
struct chain {
  int32_t score        = 0;   Accepted-chain score
  int32_t anchor_count = 0;   Anchors in the chain
  int32_t anchor_start = 0;   Start of the chain slice in A
};
```

Where `anchor_start` is the index of the first anchor in the chain in the array A.

For a non-empty record c, its anchors occupy consecutive positions in forward order:

$$A[c.\texttt{anchor_start}], \ldots, A[c.\texttt{anchor_start} + c.\texttt{anchor_count} - 1].$$

The array A is constructed using f and p arrays. For every anchor i, the two stored values mean:

f_i the score of the best chain that the selected forward algorithm found with anchor i at its end.

p_i the index of the anchor immediately before i in that chain. The value is -1 when the best chain found for i starts at i

For example,

$$p_7 = 5, \qquad p_5 = 2, \qquad p_2 = -1 \quad \Longrightarrow \quad 7 \to 5 \to 2 \to -1.$$

5.4. Chaining algorithms

Similar to the minimizer chapter here we will discuss different approaches to chaining. The algorithm itself is based on dynamic programming scoring, but the details differ.

Table 5.1: Comparison of different chaining algorithms

ID	Algorithm	Defining decision	Implementations
B1	Ordered predecessor scan	Visit recent candidates first. Increase the skip counter when a marked candidate does not improve the score. Decrease a positive counter after an improvement. Stop when the counter exceeds its limit. Also score cached predecessor m when it exists and the main scan does not reach it	minimap2 non-RMQ chaining, BioSeqDB scalar and exact SIMD
B2	Source-to-target DP	Apply each completed source score in parallel to later targets admitted by the kernel's index and distance checks	G3SA first CUDA pass
B3	Ranking-value second pass	Rank predecessors by a value calculated from their saved scores and coordinates. Keep at most one predecessor with the largest accepted ranking value. If one was saved, calculate its full pair score	G3SA second CUDA pass
B4	Range-minimum-query tree DP	Use a balanced tree whose nodes cache the smallest stored priority in each subtree, then calculate the full score for the selected predecessor	minimap2 RMQ and BioSeqDB's experimental RMQ route, which includes both query-coordinate boundaries and omits the bandwidth check
B5	Per-element batch-limited DP	Keep one running score for each vector element. Stop after enough consecutive batches leave all of those scores unchanged	BioSeqDB bounded CPU SIMD
B6	Per-target batch-limited DP	Keep one best score for the target. Stop after enough consecutive batches fail to increase that score	BioSeqDB Vulkan

Scalar and exact SIMD both implement B1 because they make the same ordered decisions. G3SA selects a short kernel or one of two tiled launch configurations according to range size within B2 and B3. BioSeqDB CPU SIMD and Vulkan use different batch stopping rules, so they are B5 and B6.

All implementations pass their saved scores and predecessors to later code that builds and classifies mappings.

The BioSeqDB RMQ implementation includes both query-coordinate boundaries. It can select a predecessor with the same query coordinate as the target and accept it without a bandwidth test. We classify the tree design as B4 and describe these defects in Section 5.10.

5.4.1. Shared pair score

Let's say we have two anchors with coordinates q_j, q_i and r_j, r_i on the query and reference. Let's assume the minimizer span is always k. We now can say that after incorporating (r_j, r_i) anchor we will cover $q_i - q_j$ symbols on the query and $r_i - r_j$ symbols on the reference, however those two jumps can be interpreted as taking some symbols from query and assuming they form replacement and the rest forms a gap. For example let's say $q_i - q_j > r_i - r_j$. This means that we match more symbols in query than in reference, so indicates some deletion in the reference. So we can say $r_i - r_j$ symbols are replaced (or stay the same) while the leftover $(q_i - q_j) - (r_i - r_j)$ symbols form this deletion. This thinking is symmetrical so we just only need to use minimum and absolute value to make it more generic.

Now we introduced $g(j,i)$ (the advance shared by the reference and query) and $\delta(j,i)$ (the remaining advance in whichever sequence moves farther). As k-mers represent hashed fragment of length k we can assume that in our match we have exactly $\min\{q_i - q_j, r_i - r_j, k\}$ new symbols that matched. Hence we now define our first primitive score as $\min\{q_i - q_j, r_i - r_j, k\} = \min\{g(j,i), k\}$.

This scoring has some problems with the most important one being that it doesn't penalize the case when we have a large gap in the reference sequence just to match one or two symbols in the query. This is improbable that some very large fragment was replaced during evolutionary processes by just a few bases. For that reason we will use function $\lfloor \text{Penalty}(\delta, g) \rfloor$. The penalty function should satisfy the following properties:

- There's no penalty for doing nothing at all: $\text{Penalty}(0,0) = 0$
- For $g \leq k \wedge \delta = 0$ there's no gap so there's shouldn't be any penalty: $\text{Penalty}(0,g) = 0$
- Penalty should be monotonically increasing with respect to δ: $\text{Penalty}(\delta, g) \leq \text{Penalty}(\delta + 1, g)$
- Penalty should be increasing slower than linearly with respect to g: $\text{Penalty}(\delta, g+1) < \text{Penalty}(\delta, g) + 1$. This guarantees that increasing g without touching δ benefits overall score (we prefer replacement region over gap region)
- For small g and δ our penalty function should prefer increase in g rather than δ. This is equivalent of saying that we want to stay close to the diagonal and reduce the gap size introduced: $\text{Penalty}(\delta + 1, g) \geq \text{Penalty}(\delta, g + 1)$
- For large g and δ our penalty function should be asymptotically the same for both δ and g. This is equivalent of saying that if we encounter very large gaps, then it doesn't matter if we increase the matching region between two fragments or increase the gap as both fragments are very dissimilar: $O_\delta(\text{Penalty}(\delta, g = \text{const})) = O_g(\text{Penalty}(\delta = \text{const}, g))$

Symmetry

Using g and δ as only parameters for penalty function guarantees that scoring is symmetric. Symmetry makes sure that gap costs are the same for both query and reference.

Our final score is then something of the following structure:

$$\begin{aligned} \text{Score}(j,i) &= \min\{g(j,i), k\} - \lfloor \text{Penalty}(\delta(j,i), g(j,i)) \rfloor \\ &= \min\{g(j,i), k\} - \lfloor \alpha * g(j,i) + \beta * \delta(j,i) + \gamma * f(\delta(j,i)) \rfloor \\ & 0 \leq \alpha, \beta, \gamma \leq 1 \end{aligned} \tag{5.2}$$

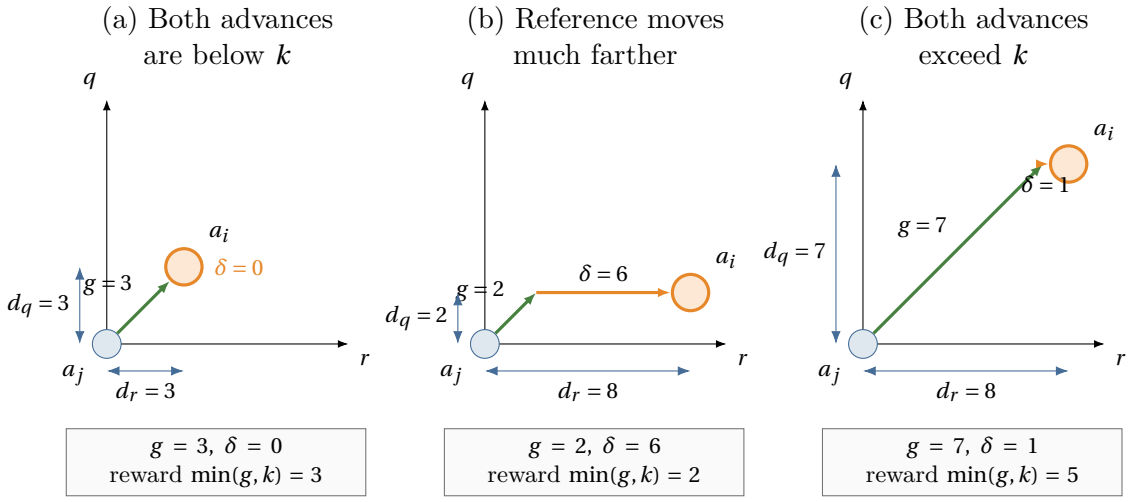


Figure 5.2: Three anchor displacements for $k = 5$. The blue brackets are the actual reference and query advances. The green diagonal step advances both coordinates by $g = \min(d_r, d_q)$. The orange horizontal step is the remaining $\delta = |d_r - d_q|$. It would be vertical when $d_q > d_r$. Panel (c) also shows that the reward stops at k, even though $g > k$.

The minimap2 scoring definition E looks exactly as what we have just specified:

$$\begin{aligned} d_q(j,i) &= q_i - q_j, & d_r(j,i) &= r_i - r_j, \\ \delta(j,i) &= |d_r(j,i) - d_q(j,i)|, & g(j,i) &= \min\{d_r(j,i), d_q(j,i)\}. \end{aligned} \tag{5.3}$$

$$E(j,i) = \min\{\ell_j, g(j,i)\} - P(j,i), \tag{5.4}$$

where

$$P(j,i) = \begin{cases} 0, & \delta(j,i) = 0 \ \wedge \ g(j,i) \le \ell_j, \\ \left\lfloor \lambda_g \delta(j,i) + \lambda_s g(j,i) + \frac{1}{2}\,\mathrm{mglog}_2\big(\delta(j,i)+1\big) \right\rfloor, & \text{otherwise.} \end{cases} \tag{5.5}$$

The function mglog_2 is minimap2's polynomial approximation of $\log_2$. So far in the abstract definition of scoring we didn't mention diagonal, but nonnegative coefficient λ_g can be viewed as diagonal-gap penalty (further away from diagonal we penalize more), where the nonnegative coefficient λ_s is the penalty for distance. The value ℓ_j is the query span of predecessor j. (In our example above we used k for all k-mers sizes, but as mentioned before minimap2 with specific settings can compress regions of same repeating symbols so we just replace k with kmer span ℓ_j. All the logic stays the same)

Minimap2 chaining link score
comput_sc 113

Score units

In this definition we view scoring function as expressed in number of matched symbols (not some arbitrary large numbers), so the coefficients must be set accordingly so the values are scaled correctly. In the dynamic programming formulation we then can take maximum of E value and ℓ_i to make sure that in the worst case scenario we assume that we at least matched the length of the minimizer. There are no formal limitations on this normalization, but it makes reasoning about the scores easier.

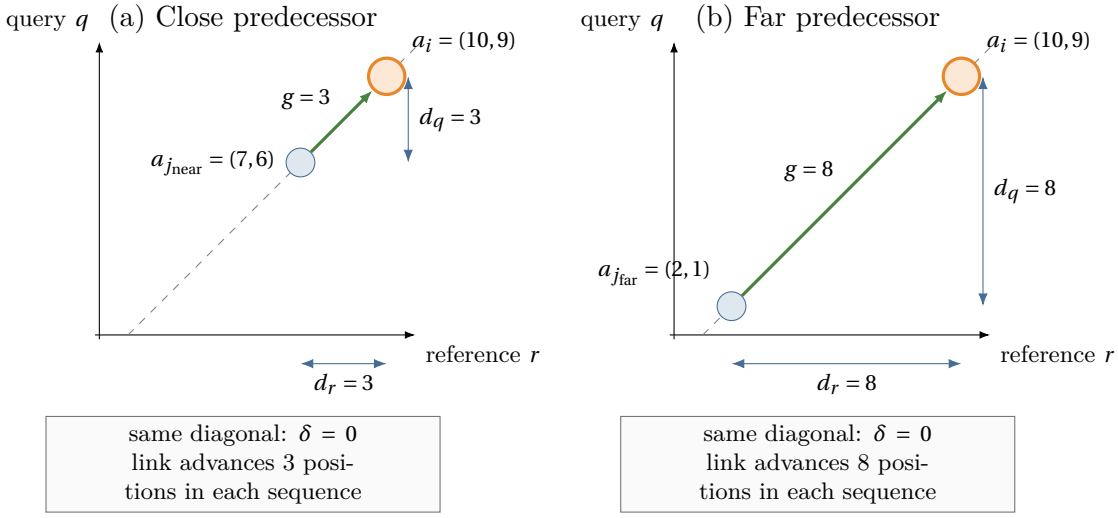


Figure 5.3: A close and a far predecessor for the same current anchor $a_i = (10,9)$. Both candidates lie on the diagonal $r-q=1$, so both have $\delta = 0$. Their link lengths are nevertheless different: the close link has $g = 3$, whereas the far link has $g = 8$. Thus zero diagonal deviation does not mean zero or equal distance.

It's worth noting that defined scoring is also symmetrical in terms of coordinates i.e the score doesn't take r_i, q_i positions as parameters so reversing coordinate systems doesn't change the outcome. However, in dynamic programming we often want to use the locally close predecessor as the best one. Consider for example situation in which we have two possible candidates for some anchor i: one is very close to our position while offering lower cummulative score for the chain, while the other one is very far away but offers high cummulative score. This scenario can happen if we have very good chain with narrow gaps and slightly worse chain with larger gaps, but the two are very far away from current position. We can have selection function that not only takes the best score, but also adds function dependent on current anchor position to make sure to formally express that preference:

$$\begin{aligned} f_j &= \max_{i<j}\{f_i + E(j,i)\}, \\ \text{Selection}(i) &= f_j + \phi(r_i, q_i) \end{aligned} \tag{5.6}$$

There shouldn't be any difference in behaviour for query and reference so ϕ should be symmetric. Additionally ϕ should be monotonic to express the fact that we prefer closer predecessors: $\phi(r_i,q_i) \le \phi(r_i+1,q_i), \phi(r_i,q_i+1) \le \phi(r_i+1,q_i+1)$. Please note that the difference here is that it's not a cummulative chain score. It's only used to select the best predeccesor for current anchor (if we tried to sum this score up we would get total score having $O(n^2)$ terms, which would severly alter the scoring behaviour discussed earlier).

In the later chapters we discuss selection function that uses artithmetical average of positions:

$$\begin{aligned} \text{Selection}(j) &= f_j + \phi(r_i, q_i) \\ \phi(r_i, q_i) &= \frac{r_i + q_i}{2} \end{aligned} \tag{5.7}$$

Selection

Adding the term with the coordinate's average can be interpretes as requirement that the chain score should increase linearly with the distance between the current anchor and the selected predecessor. If some chain is twice far away than other then it should offer at least twice better score. There are many otehr functions that could be placed here as ϕ, but simple average is the simplest one

5.5. B1: minimap2 dynamic programming

As our scoring model $E(i,j)$ depends only on two anchors we can define recursive equations for dynamic programming:

$$f_i = \max\left(\ell_i, \max_{j\in\mathscr{C}_i}\{f_j + E(j,i)\}\right), \qquad p_i = \arg\max_{j\in\mathscr{C}_i}\{f_j + E(j,i)\}. \tag{5.8}$$

Where $\mathscr{C}_i$ is the set of valid predecessors (selected by being close to diagonal - this is form of specifying cutoff condition so we don't need to check all possible predecessors all the time). Maximum function is used here to make sure that in worst case if our definition of $E(i,j)$ yield very low score we can say that we matched at least length of the minimizer span.

If no predecessor improves the base value of score ℓ_i, then $p_i = -1$.

We will call iteration over $\mathscr{C}_i$ a scan. Minimap2 does this scan by visiting anchors in order from newest to oldest.

After sorting, minimap2 numbers the anchors $0, 1, \ldots, n-1$. When the DP algorithm reaches anchor i, the scores $f_0, \ldots, f_{i-1}$ and predecessor indices $p_0, \ldots, p_{i-1}$ have already been calculated. It starts a new chain at i with

$$f_i = \ell_i \qquad \text{and} \qquad p_i = -1,$$

Minimap2 tests earlier anchors in the order $j = i-1, i-2, \ldots$ (larger index is tested first). For every pair that passes the distance checks, it calculates

$$c_j = f_j + E(j,i).$$

If $c_j > f_i$, anchor j becomes the new predecessor: $f_i \leftarrow c_j$ and $p_i \leftarrow j$ (strict equality is important here so we avoid using older anchors with the same score)

Trying every earlier anchor would make this dynamic program expensive. Minimap2 applies the following strategies to limit the work performed:

- `max_iter` limits how many previous anchors we scan: this is the number of highest-numbered array positions before i.
- `max_skip` is the limit on skip counter s. It starts at zero for every new i and is increased by one on marked anchor that fails to improve the score and decreased by one on marked anchor that improves the score. If skip counter exceeds limit we finish scan loop early and do not scan other previous anchors.
- `m` is an extra candidate that is tested after the scan loop. It is the anchor with the highest score among all previously tested anchors (this is a measure to prevent skipping good chains due to too aggressive skip counter limit)

- `CanScore(j,i)` checks the anchor distances to make sure the anchor lies within the band (close to diagonal). It uses D_x and D_y for limits on reference and query distances respectively and B for distance from diagonal.

Minimap2 contains separate rules for multi-segment and cDNA input. The key algorithm stays the same and differences are very subtle:

- There are differences in handling scoring case of $g_i = g_j$
- Multi-segment non-cDNA input adds special requirement $d_r \le D_y$
- cDNA input does not force $D_y \ge B$
- There are differences in penatly factors for scoring function $E(j,i)$

In description below I assume we talk about default case of one-segment, non-cDNA input.

Different kinds of input

Multi-segment input is in case of Minimap2 described as a situation in which single query maps to different fragments of the reference section. I do not support this use case in BioseqDB out of the box at this time. This situation is also known as split-read or supplementary alignment and normally occurs due to due to structural variations, translocations, or chimeric artifacts in the genome. cDNA in this case describes the same situation, but has slightly different scoring parameters. Even though that can be interesting, we don't want to describe these modes here since those are very specialized heuristics

5.5.1. Skipping with counters and marked anchors

As discussed above minimap2 scores only valid pairs of anchors:

$$0 < d_r(j,i) \le D_x, \qquad 0 < d_q(j,i) \le \min(D_x, D_y), \qquad \delta(j,i) \le B, \tag{5.9}$$

where $B = \texttt{bw}$ where all parameters like `bw` are minimap2's CLI arguments.

$$D_x = \max(\texttt{max_dist_x}, B), \qquad D_y = \max(\texttt{max_dist_y}, B).$$

After a candidate j passes this checks, we calculate $E(i,j)$ score and then update p_j if we see better score. We want to that p_j was updated in scan for i. This can be understood as building chains in parallel where we want to save information about predecessor for each anchor:

$$t_{p_j} \leftarrow i \qquad \text{when } p_j \ge 0.$$

If some time later in other scan we see that $t_{p_j} = i$ then we know that p_j was updated in scan for i (we already tested candidate with a larger anchor). If marked anchor x fails to improve f_i, then s increases by one. A failed unmarked anchor leaves s unchanged. An improvement subtracts one from a positive s. It does not necessarily reset the counter to zero. The loop stops only when $s > \texttt{max_skip}$

Interpretation

This marking with skip counter can be intuitevely understood as a situation in which we try to improve the best chain by adding new anchor. If repeatedly this chain fails to improve the score, we don't want to waste more time on it.

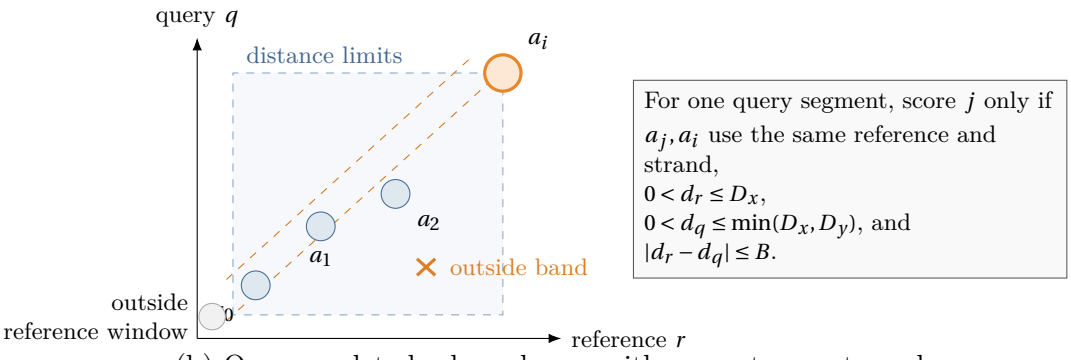
We don't perform this marking when this is candidate thatraises s above `max_skip` (in that case we stop the scan loop early).

Minimap2 keeps the loop index in e. If the skip test stopped the loop while it was processing j, then $e = j$. e is used to check if we can score anchor m (if $m < e$ then we can).

m is additional index saved by minimap2 on the side, which we call the saved high-score anchor. The saved value is an anchor index whose previously calculated f_m is large. If m is absent or has left the reference-distance window (is too far), minimap2 first rebuilds it by finding the largest f_j in the current `max_iter` set.

This saved anchor can then remain saved after $m < i - \texttt{max_iter}$. Each time we check $e < m$ (see if we already scored this anchor) and if not we calculate $E(m, i)$ and do update checks similar to other anchors. m is introduced to make sure to take into consideration anchor that could be potentially a little bit further away. Minimap2 doesn't use priority queue or list here, because chaining accuracy is already good enough and this approach is much simpler.

(a) Which earlier anchors can be scored



(b) One complete backward scan with concrete counter values

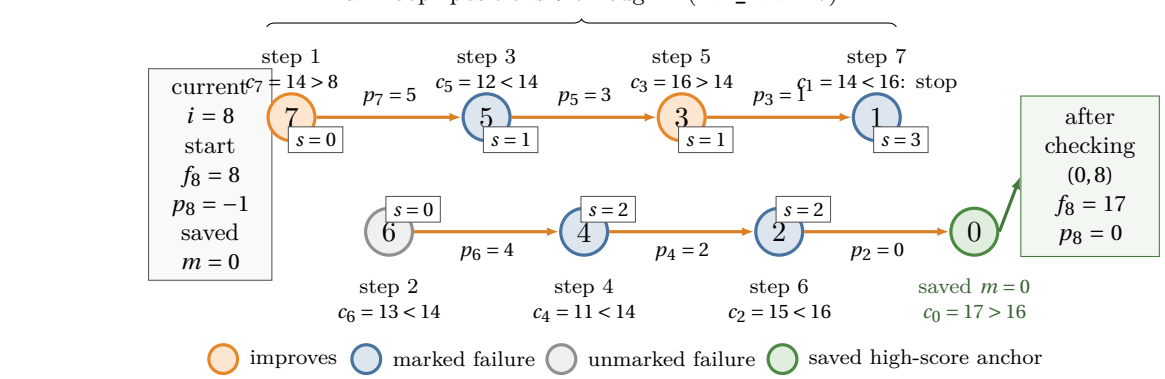


Figure 5.4: (a) The reference and query limits and diagonal band decide whether a pair can be scored or not. (b) If distance checks pass, then we perform scans in a loop. The picture shows one pass of the scan for the current anchor $i = 8$. The numbers inside the circles are anchor indices. The orange arrow from j to p_j shows the predecessor previously saved for anchor j. After j is scored without stopping, the arrow's destination receives the current mark $i = 8$. The numbered steps visit anchors $7, 6, \ldots, 1$. The small s boxes show the single skip counter for current anchor 8. Step 2 doesn't affect s as this anchor doesn't have a predecessor in this scan pass (it's unmarked). Step 5 improves the score and changes $s = 2$ to $s = 1$. Anchor 0 is already outside the main loop because `max_iter` $= 7$, so we don't scan it, but we check it later with $m = 0$. Step 7 stops with $e = 1$, so saved $m = 0$ satisfies $m < e$ and is checked once. The score of $m = 0$ equal to 17 wins, but in some other scan other anchor could have a better score.

Algorithm 10 Minimap2's ordered predecessor scan for one query segment

Require: sorted anchors $a_0, \ldots, a_{n-1}$; ordinary non-cDNA mode; configured distance limits, $B = \texttt{bw}$, `max_iter`, and `max_skip`
Ensure: chain scores f_i and predecessor indices p_i

$D_x \leftarrow \max(\texttt{max_dist_x}, B)$
$D_y \leftarrow \max(\texttt{max_dist_y}, B)$
function CanScore(j, i)
    $d_q \leftarrow q_i - q_j, \quad d_r \leftarrow r_i - r_j$
    if a_j, a_i use different references or strands then
        return false
    if $d_r \le 0$ or $d_r > D_x$ then
        return false
    if $d_q \le 0$ or $d_q > D_x$ or $d_q > D_y$ then
        return false
    if $|d_r - d_q| > B$ then
        return false
    return true
$t_j \leftarrow 0$ for every j; $m \leftarrow -1$; $st \leftarrow 0$ m is one saved anchor index
for $i \leftarrow 0$ to $n-1$ do
    while $st < i$ and (a_{st} has a different reference or strand from a_i, or $r_i - r_{st} > D_x$) do
        $st \leftarrow st + 1$
    $st \leftarrow \max(st, i - \texttt{max_iter})$ hard lower bound for the main scan
    $f_i \leftarrow \ell_i, \quad p_i \leftarrow -1, \quad s \leftarrow 0, \quad e \leftarrow st - 1$
    for $j \leftarrow i-1$ down to st do
        if CanScore(j, i) then
            $c \leftarrow f_j + E(j, i)$
            if $c > f_i$ then
                $f_i \leftarrow c, \quad p_i \leftarrow j$
                if $s > 0$ then
                    $s \leftarrow s - 1$
            else if $t_j = i$ then
                $s \leftarrow s + 1$
                if $s > \texttt{max_skip}$ then
                    $e \leftarrow j$
                    break
            if $p_j \ge 0$ then
                $t_{p_j} \leftarrow i$ mark one older anchor for this scan
    if $m < 0$, or a_m has a different reference or strand from a_i, or $r_i - r_m > D_x$ then
        $m \leftarrow -1$
        for $j \leftarrow i-1$ down to st do
            if $m < 0$ or $f_j > f_m$ then
                $m \leftarrow j$
    if $m \ge 0$ and $m < e$ then
        if CanScore(m, i) then
            $c \leftarrow f_m + E(m, i)$
            if $c > f_i$ then
                $f_i \leftarrow c, \quad p_i \leftarrow m$
    if $m < 0$, or (a_m, a_i use the same reference and strand, $r_i - r_m \le D_x$, and $f_i > f_m$) then
        $m \leftarrow i$

Algorithm details

We check if a_{st} has a different reference or strand from a_i, because anchors in minimap2 can originate from different refences. In BioseqDB this won't happen, but we still need to check for strand direction. Also please note that formally we use different values for marking $t_{p_j} \leftarrow i$, but it's main goal is just to detect visited anchors for the same scan pass (like in binary map, but assigning i just means that we don't have to clear it.)

5.6. BioseqDB SIMD DP: exact and bounded batch

For one target anchor a_i, scalar B1 tests possible predecessors one at a time. During the sacan it starts at a_{i-1} and walks towards the lower end of the search window. SIMD implementations both implement the same outer loop, but the scan itself is rewritten: We place each predecessor candidate in batches of size L (each batch is one SIMD vector) - one vector call can calculate as many as L values of $E(j,i)$. We have two kinds of SIMD algorithm:

1. Exact SIMD B1 called `exact_visited` It it the most naive implementation: it calculates up to L pair scores together in a SIMD vector, but F, P, and s are still single scalars as in original B1.

2. Bounded batch SIMD B1 called `bounded_batch_approximate`. Algorithm removes the mark array completely, while scalar skip counter in now counting batches. Lane λ keeps its own temporary best score H_λ and predecessor P_λ. A shared counter u records how many batches in a row failed to improve any lane (the algorithm stops when u reaches its batch limit U). We take the largest score across the lane winners and test the saved anchor m once.

We will discuss bounded batch SIMD B1 first, as it is more interesting one.

5.6.1. Candidates in SIMD vectors

The candidates are placed in batches. So now in loops we use $b = 0$ instead of j where 0 is the first batch, $b = 1$ is the next older batch, and so on. Within one batch, $\lambda \in \{0, \ldots, L-1\}$ is the SIMD lane number. The candidate placed in that lane is $j_{b,\lambda}$.

First we introduce the most boring thing - coordinates mapping:

$$\begin{aligned} \mathrm{end}_b &= i - bL, & \mathrm{begin}_b &= \max(st_i, \mathrm{end}_b - L), \\ a_b &= \mathrm{end}_b - \mathrm{begin}_b, & j_{b,\lambda} &= \mathrm{begin}_b + \lambda. \end{aligned} \tag{5.10}$$

Here end_b is the position immediately after the batch, begin_b is its first candidate, and a_b is its number of active lanes. As candidates are split into batches, the last one can be partial (so a_b will be used to mask out the inactive lanes).

For lane λ, the pair scorer first calculates $\epsilon_{b,\lambda} = E(j_{b,\lambda}, i)$. The code then loads the earlier chain score $f_{j_{b,\lambda}}$ and adds it to $\epsilon_{b,\lambda}$. The complete proposed chain score is $C_{b,\lambda}$, and $\mathbf{C}_b$ is the vector containing one such total per lane. A rejected candidate or an inactive lane receives $-\infty$.

Figure 5.5 depicts that now instead of single candidate being checked, we check batches:

- $C_{b,\lambda}$ holds total score for the candidates which corresponds to c in original B1 algorithm

- $\mathbf{j}_b$ holds position of the candidate which corresponds to j in original B1 algorithm
- $\mathbf{H}^{(b)}$ holds the best score for the batch which corresponds to f_i in original B1 algorithm (note that now we hold vector instead of scalar, so at the end we need to select minimum value)
- $\mathbf{P}^{(b)}$ is shown later on the next figure and holds the index for $\mathbf{H}^{(b)}$

Note that only $\mathbf{H}^{(b)}$ and $\mathbf{P}^{(b)}$ are presisted between scans. The superscript (-1) over H means that we got this value from previous scan.

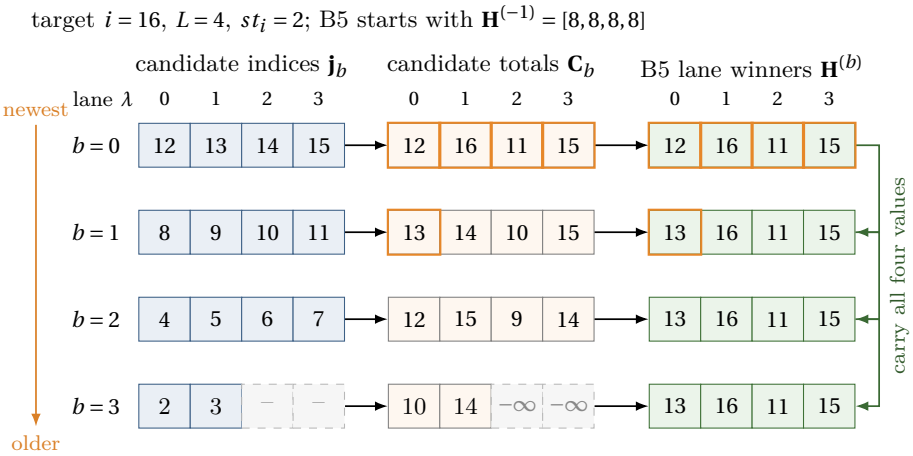


Figure 5.5: How predecessor candidates are placed in four SIMD lanes. The lane number λ selects the corresponding column in all three vectors. Blue boxes are candidate indices, orange boxes are calculated scores, and green boxes are best scores (the scores are improved vertically). When a lane improves its new maximum has an orange outline. Batch 3 is partial (we exhausted all candidates), so its two dashed lanes are inactive and their totals are $-\infty$.

We operate on batches so we need to adjust our definition of skip counter. Let $S =$ `max_chain_skip`:

$$U = \max\left\{1, \left\lfloor \frac{S}{L} \right\rfloor + 1\right\} \tag{5.11}$$

As a helper we introduce $I_{b,\lambda}$ which is just binary mask checking lane-wise which C value is better than H, so we update skip counter according to the following rules:

$$u_b = \begin{cases} 0, & \text{if any } I_{b,\lambda} \text{ is true,} \\ u_{b-1} + 1, & \text{otherwise.} \end{cases} \tag{5.12}$$

When $u_b \geq U$ the algorithm stops. In original B1 we had guarantee that we will stop after S skipped candidates, but here we stop after U such that: $S < LU \leq S + L$. This doesn't however bother us at all, as this is only very minor implementation detail user doesn't need to see.

The figure below depcits two kinds of SIMD B1 algorithms. The first one (a) wasn't discussed in detail as it only puts the candidates in a single SIMD vector to speed up only few operations like finding the maximum, where the rest of the code stays unchanged. The second parts of figure (b) is more interesting as it shows the approach we discussed with vector batching.

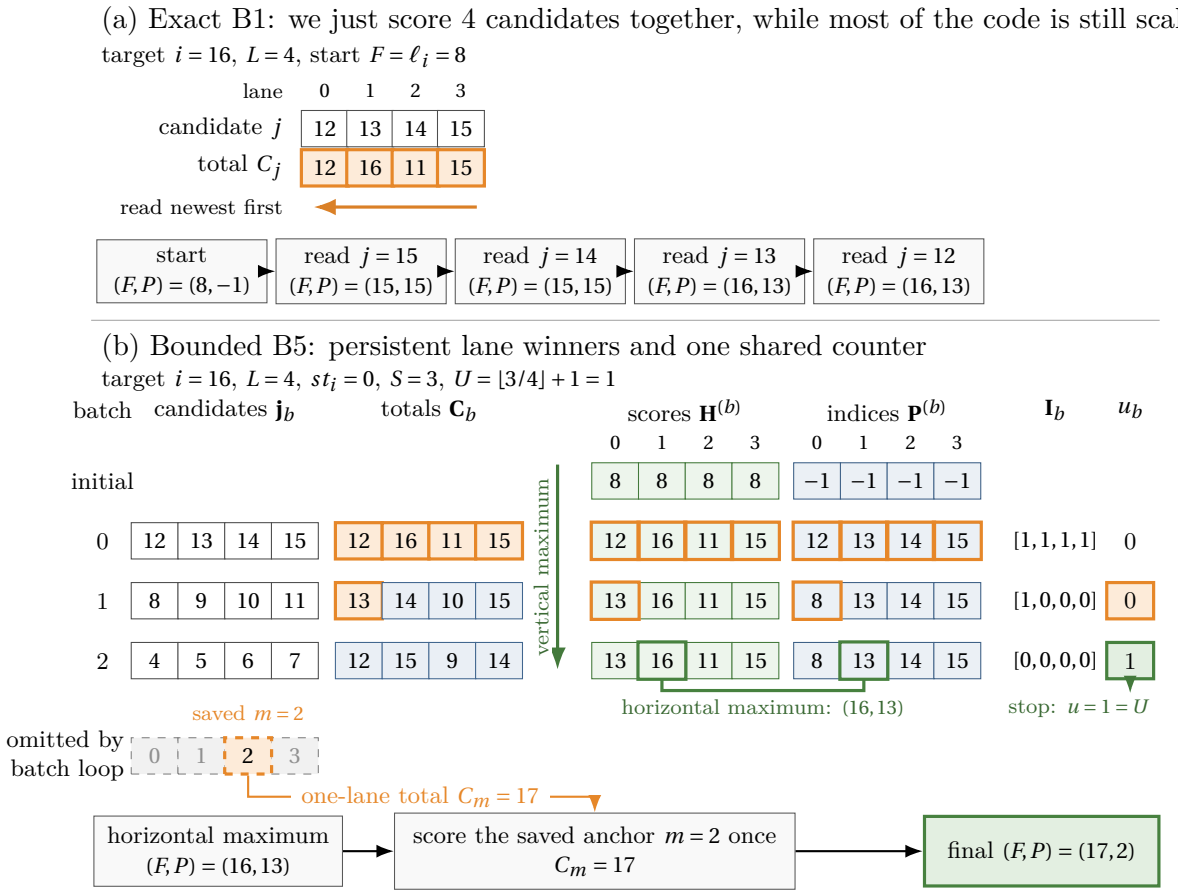


Figure 5.6: Two strategies for calculating SIMD B1 (a) Exact B1 calculates four totals together, then applies state changes in the scalar order whenever some lane can improve. (b) Bounded SIMD is more clever - it starts with $\mathbf{H}^{(-1)} = [8,8,8,8]$ and $\mathbf{P}^{(-1)} = [-1,-1,-1,-1]$. Each row shows both vectors after its batch. Green boxes hold scores and blue boxes hold their predecessor indices. An orange border around C marks a new improvement (maximum). Orange borders around H and P show the stored pair that changed. All four lanes change in batch 0. In batch 1, only lane zero changes, to $(H_0, P_0) = (13, 8)$. Batch 2 changes neither vector, so $u = 1 = U$ and candidates 0 through 3 (bottom gray area) are omitted. The downward arrow shows that each lane keeps its best (H, P) pair while the scan moves through older batches. The horizontal maximum then compares the four lane scores. Lane 1 supplies the pair selected by that horizontal maximum, $(H_1^{(2)}, P_1^{(2)}) = (16, 13)$, marked by the green bracket and outlines. The orange box marks saved candidate $m = 2$, whose separate one-lane total $C_m = 17$ replaces the result with $(17, 2)$.

After the last scored batch, the algorithm takes horizontal maximum of a vector once more:

$$f_i = \max_\lambda H_\lambda, \qquad p_i = \max\{P_\lambda : H_\lambda = f_i\}. \tag{5.13}$$

The algorithm is described in detail in Algorithm 11.

Algorithm 11 Bounded SIMD B1

Require: target i, window $[st_i, i)$, SIMD width L, $S = \texttt{max_chain_skip} > 0$, $\texttt{max_chain_iter} > 0$, saved index m
Ensure: f_i, p_i

$U \leftarrow \max\{1, \lfloor S/L \rfloor + 1\}$ whole-batch stopping limit
$\mathbf{H} \leftarrow [\ell_i]^L, \quad \mathbf{P} \leftarrow [-1]^L, \quad u \leftarrow 0$ one temporary winner per lane
$o \leftarrow st_i$ no omitted interval yet: $[st_i, o)$ is empty
for $b \leftarrow 0, 1, \ldots$ while $\mathbf{end}_b > st_i$ do
    calculate $\mathbf{begin}_b, \mathbf{end}_b, a_b, \mathbf{j}_b$ using Equation 5.10
    $\mathbf{A} \leftarrow [\lambda < a_b]_{\lambda=0}^{L-1}$ mask unused lanes in the oldest partial batch
    $\boldsymbol{\epsilon} \leftarrow \text{PairScoreBatch}(\mathbf{j}_b, i, \mathbf{A})$ $E(j, i)$, or $-\infty$ for a rejected pair
    $\mathbf{C} \leftarrow \text{Select}(\boldsymbol{\epsilon} \neq -\infty, \mathbf{f}[\mathbf{j}_b] + \boldsymbol{\epsilon}, -\infty)$ total candidate scores
    $\mathbf{I} \leftarrow (\mathbf{C} > \mathbf{H})$ strict lane-wise improvements
    $\mathbf{H} \leftarrow \text{Select}(\mathbf{I}, \mathbf{C}, \mathbf{H})$
    $\mathbf{P} \leftarrow \text{Select}(\mathbf{I}, \mathbf{j}_b, \mathbf{P})$ equal older candidates do not replace
    if $\text{Any}(\mathbf{I})$ then
        $u \leftarrow 0$
    else
        $u \leftarrow u + 1$
    if $u \geq U$ and $\mathbf{begin}_b > st_i$ then
        $o \leftarrow \mathbf{begin}_b$ batch loop omits $[st_i, o)$
        break
$f_i \leftarrow \text{HorizontalMax}(\mathbf{H})$
$\mathbf{T} \leftarrow \text{Select}(\mathbf{H} = f_i, \mathbf{P}, -1)$
$p_i \leftarrow \text{HorizontalMax}(\mathbf{T})$ largest predecessor index wins a score tie
if $f_i = \ell_i$ then
    $p_i \leftarrow -1$
if $m < st_i$ or m is outside the current reference and strand window then
    $m \leftarrow \arg\max_{st_i \leq j < i}(f_j, j)$, or -1 if this interval is empty compare score first, then index
if $st_i \leq m < o$ then
    $\epsilon_m \leftarrow \text{PairScoreOneSIMDLane}(m, i)$ $-\infty$ when the pair is rejected
    if $\epsilon_m \neq -\infty$ then
        $c \leftarrow f_m + \epsilon_m$
        if $c > f_i$ then
            $f_i \leftarrow c, \quad p_i \leftarrow m$
if $m < 0$ or $f_i \geq f_m$ then
    $m \leftarrow i$ B5 moves the cache to the newer anchor on a tie

5.6.2. Reverse chaining (backtracking)

The forward pass has already produced one score f_i and one predecessor p_i for every anchor. Backtracking follows the predecessor links and turns them into the chain records and flat anchor array A defined at the start of this chapter.

Formal definition for backtracking

Anchors have their score assigned: f_i. We would like to start by considering the highest scores as they are the best chains according to our score function. We sort the anchors in descending order of their scores: $f_{a_0} \geq f_{a_1} \geq \ldots \geq f_{a_{n-1}}$.

Now each anchor is the chain end so we need to find its start. We will traverse anchors according to p_i i.e $i \to p_i \to p_{p_i} \to \ldots \to -1$. This traversal doesn't have to go till the end of the chain. Usually there will be some part of the chain that is too low quality (not contributing to the score significantly).

We define function $\Delta_e(b)$ as the score assigned to the anchors followed from e up to, but not including, b:

$$\Delta_e(b) = \begin{cases} f_e, & b = -1, \\ f_e - f_b, & b \geq 0. \end{cases} \tag{5.14}$$

The value of $\Delta_e(b)$ will be used to determine the best cut point for each anchor:

$$\text{Cut}([\Delta_e(e), \Delta_e(p_e), \Delta_e(p_{p_e}), \ldots, \Delta_e(-1)]) \in [e, p_e, p_{p_e}, \ldots, -1] \tag{5.15}$$

The backtracking is then defined as:

$$\text{Backtrack}(e) = \{c = (a_i, p_{a_i}, \ldots \text{Cut}(a_i)) | a_i \in [a_0, a_1, \ldots, a_{n-1}] \ \wedge f_{a_i} \geq F_{\min} \wedge |c| \geq C_{\min}\} \tag{5.16}$$

The backtracking function finds best cutoff point and filters anchors for minimal score $F_{\min}$ and minimal chain length $C_{\min}$.

5.6.3. Minimap2 backtracking

Minimap2 uses two types of markings. The array t (similar to forward chaining), records which anchors have already been marked:

1. 0 means unmarked: we didn't visited this anchor yet
2. 1 means permanently marked: we won't consider this anchor as end of the chain, because it's already part of some found best chain. However anchors marked as 1 still can be part of the chain. This marking prevents a situation in which we would find chain, then it's exact subchain etc.
3. 2 means temporarily marked while one path is tested: this marking is used for Cut function and cleared after the path is tested. It purpose was to detect cycles (see cycle note below)

> **$t = 2$**
>
> Minimap2's implementation could remove usage of $t = 2$ markings, because under usual conditions there will be never a cycle of chains (anchor will always point to other anchor will lower position due to how forward chaining works). In listings below I provide the pseudocode corresponding to the actual current implementation, not some optimized one.

Minimap2 cut function marks visited anchors with $t = 2$ and stops on any anchor with non-zero t mark. We remember the best cut with the largest value of Δ_e. If a later cut is worse than the best cut by more than `max_drop`, the search stops and returns the best cut seen so far. After everything we clean the $t = 2$ temporary markings.

Figure 5.7 shows three interations of the main loop: we detect cuts for three anchors a_{11}, a_{10}, a_9. The example uses `min_sc` = 40 (this is minimap's minimum score $F_{\min}$), `min_cnt` = 2 (this is minimap's minimum chain length $C_{\min}$), and `max_drop` = 5 (maimum allowed score drop between best cut and current anchor in the chain). A Δ value inside anchor a_b is the

score that would result if a_b was the selected cut. Orange boxes are anchors where either cut happens or we stop the search for the cut. Additionally I added "marked" label to a_5 to depict the fact that a_5 was marked by first $e = 11$ chain, so it cannot be end of the chain (it's still selected for $e = 10$, because it's start of the chain) Green boxes are anchors that are copied into the resulting array (everything excluding the cut anchor itself).

The minimap2 backtracking uses the same technique as BioseqDB Vulkan minimizer seeding: in the first pass we only count the number of copied anchors, while in the second pass we store them in the resulting array. This $\mathsf{phase} \in \langle \mathsf{count}, \mathsf{write} \rangle$ iteration is depicted in (c) panel. The last step of minimap2's backtracking depicted in panel (d) is reversing the chains (so the start is on the left rather than on the right side) and sorting by reference coordinates of the chain's starts, so the chains are in natural left-to-right order.

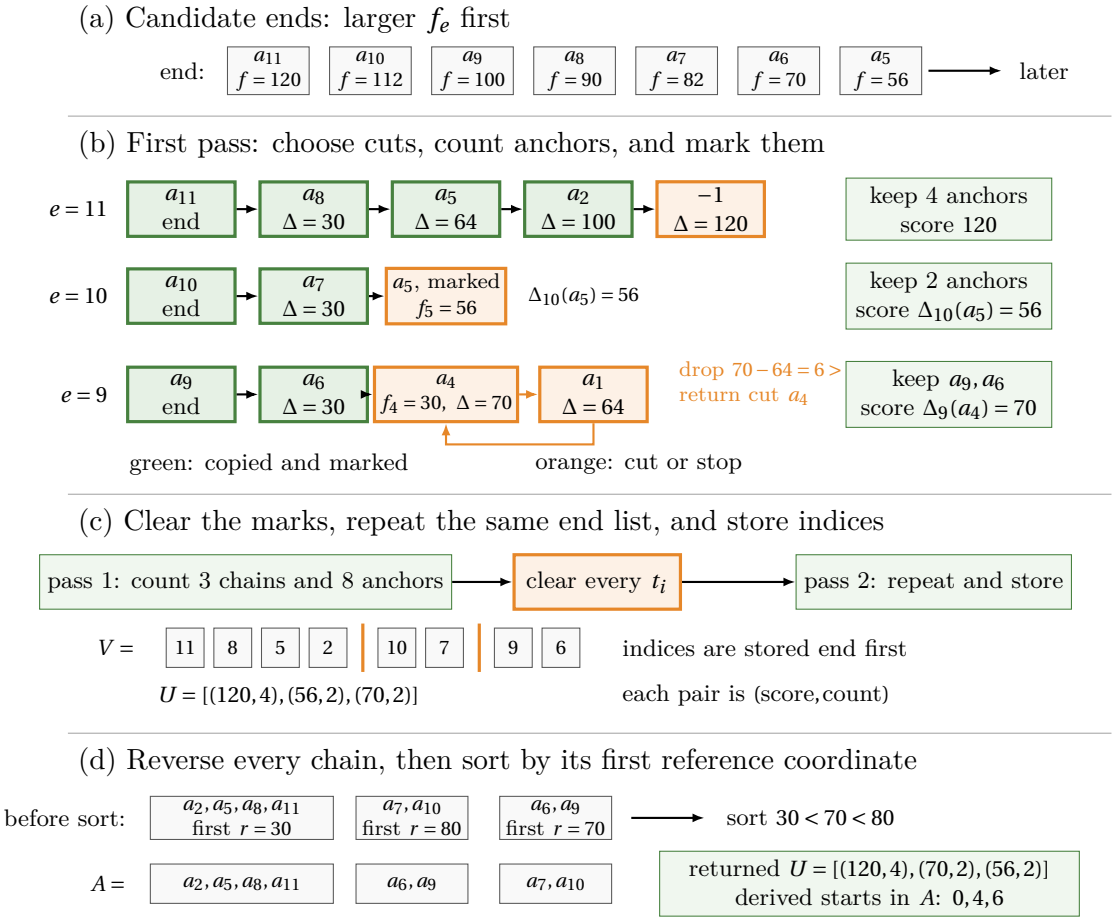


Figure 5.7: Minimap2 scalar backtracking on three anchors. Panel (b) shows that the anchor that triggers `max_drop` doesn't have to be the cut that is returned. Panel (c) shows the separate count and write passes. Panel (d) shows the final forward anchor order.

Algorithm 12 shows the code for the cut search loop.

Algorithm 12 Minimap2 cut selection function for anchor e

Require: unmarked end e, scores f, predecessors p, marks t

Require: drop limit $D = \texttt{max_drop}$
Ensure: cut b, which is excluded from the new chain

```
function ChooseCut(e, f, p, t, D)
    i ← e,   b ← e,   δ_best ← 0
    repeat
        t_i ← 2   temporary mark
        i ← p_i
        if i < 0 then
            δ ← f_e
        else
            δ ← f_e − f_i
        if δ > δ_best then
            δ_best ← δ,   b ← i
        else if δ_best − δ > D then
            break
    until i < 0 ∨ (i ≥ 0 ∧ t_i ≠ 0)
    h ← i,   j ← e   h is where this search stopped
    while j ≥ 0 and j ≠ h do
        t_j ← 0,   j ← p_j   clear only temporary marks
    return b
```

Algorithm 13 shows also how two passes for count/write are integrated. Minimap2 uses first pass to count all anchors i.e N_C (this value is used to allocate array U). Array U holds anchor count for a chain in lower 32 bits and score in higher 32 bits (this packing is defined by Pack function). U is then chain-reversed into B_γ array, which undergoes sorting by reference coordinates ReferencePos($B_\gamma[0]$) The final array A contains all anchors, while output array U contains packed `uint_64t` metadata for chain (count and score).

Algorithm 13 Minimap2 scalar backtracking and final chain order

Require: input anchors $a_0, \ldots, a_{n-1}$, scores f, predecessors p

Require: limits $S = \texttt{min_sc}$, $K = \texttt{min_cnt}$, and $D = \texttt{max_drop}$
Ensure: packed minimap2 records U and forward-order anchor array A

define $\mathrm{Pack}(s, c) = (\mathrm{uint64}(s) \ll 32) \mid c$
$Z \leftarrow [(f_i, i) \mid 0 \le i < n \text{ and } f_i \ge S]$
sort Z by increasing score only — read it backward below
use temporary array $V[0 \ldots n)$ — reuse an existing n-entry array
for phase $\in \langle \mathrm{count}, \mathrm{write} \rangle$ do
  $t[0 \ldots n) \leftarrow 0, \quad N_C \leftarrow 0, \quad N_V \leftarrow 0$ — accepted records and retained indices
  for $k \leftarrow |Z| - 1$ down to 0 do
    $e \leftarrow Z_k.\mathrm{index}$
    if $t_e = 0$ then
      $b \leftarrow \mathrm{ChooseCut}(e, f, p, t, D)$
      $N_V^{\mathrm{start}} \leftarrow N_V, \quad i \leftarrow e$ — cursor before tracing this end
      while $i \ne b$ do
        if phase = write then
          $V[N_V] \leftarrow i$
        $N_V \leftarrow N_V + 1, \quad t_i \leftarrow 1, \quad i \leftarrow p_i$
      $c \leftarrow N_V - N_V^{\mathrm{start}}$
      if $b < 0$ then
        $s \leftarrow f_e$
      else
        $s \leftarrow f_e - f_b$
      if $s \ge S$ and $c > 0$ and $c \ge K$ then
        if phase = write then
          $U[N_C] \leftarrow \mathrm{Pack}(s, c)$
        $N_C \leftarrow N_C + 1$
      else
        $N_V \leftarrow N_V^{\mathrm{start}}$ — discard indices but keep the marks
  if phase = count then
    allocate $U[0 \ldots N_C)$
$o \leftarrow 0$
for $\gamma \leftarrow 0$ to $N_C - 1$ do
  $c_\gamma \leftarrow \mathrm{uint32}(U[\gamma])$ — low 32 bits store the anchor count
  $B_\gamma \leftarrow [a_{V[o+c_\gamma-1]}, \ldots, a_{V[o]}]$ — reverse one chain
  $o \leftarrow o + c_\gamma$
$\pi \leftarrow \mathrm{argsort}_{0 \le \gamma < N_C} \mathrm{ReferencePos}(B_\gamma[0])$ — increasing ReferencePos
$U_{\mathrm{old}} \leftarrow U, \quad U \leftarrow [\,], \quad A \leftarrow [\,]$
for $m \leftarrow 0$ to $N_C - 1$ do
  $\gamma \leftarrow \pi_m$
  append $U_{\mathrm{old}}[\gamma]$ to U
  append every anchor in B_γ to A
return (U, A)

Start positions

Minimap2 stores only the score and anchor count, but it does not store an anchor start. The start of a chain is the sum of the counts of all earlier entries in U and later code uses that sum. I were initially very suprised and confused by summing coordinates, but this is the explanation.

5.6.4. BioseqDB SIMD backtracking

Exact SIMD B1 and bounded SIMD B1 call the same SIMD backtracking function. Its main differences from minimap2 are the following.

1. Minimap2 follows and marks one end at a time. BioseqDB handles the end with largest (f_e, e) alone, then collects up to L later paths together.
2. Minimap2 can cut a path with `max_drop`. BioseqDB follows a path until -1 or an anchor that is already marked.
3. Minimap2 repeats the complete backtracking pass so it can count chains before allocating U. BioseqDB runs one pass and appends accepted chains to output arrays that grow as needed.
4. Minimap2 does not specify an anchor order for equal scores. BioseqDB tries the higher end position first on a score tie.
5. Minimap2 finally sorts complete chains by ReferencePos of their first anchor. BioseqDB leaves accepted chains in the order in which it processes their ends.

In Bioseq chaining we consider 3 different steps:

1. First cut have empty marks so as microoptimisation this case is handled separately.
2. Path collection loads L top-scoring anchors and then follow one chain per lane.
3. Ordered marking works on transposed matrix, where each of L lanes contains consecutive indices for one chain. (In reality we don't transpose any matrix, it's here only as illustration as in the code we put chains directly into final path rows)

In BioseqDB we only have binary marksL

$$t_i = 0 \quad \text{means unclaimed,} \qquad t_i = 1 \quad \text{means claimed.}$$

There is no temporary value 2, which was obsolete. A mark is never cleared, including when the marked prefix later fails the score or count test.

The largest-ranked end is handled alone. If L consecutive links satisfy $p_j = j - 1$, one vector comparison checks the L links. Two vector stores then save and mark the L anchors. Otherwise the same Highway loop uses only lane zero for that link. This is a one-lane SIMD operation, not a call to the scalar backtracker.

Each later batch contains at most L ends. During collection, lane λ follows one end. At depth d, an indexed scatter writes that lane directly to

$$V_{\lambda,d} \quad \text{at address} \quad V + \lambda n + d.$$

Consequently each path is already one contiguous row. No second matrix and no later transpose pass are used. The L-by-n path matrix itself uses $4nL$ bytes. The ranked ends and marks are separate temporary arrays.

Figure 5.8 follows the example with $L = 4$, $S = 30$, and $K = 2$. The first path is $19, 9, 8, \ldots, 0$. It marks these anchors before the next four ends are collected. For the later ends, the relevant scores are

$$f_{19} = 100, \quad f_{18} = 90, \quad f_{17} = 85, \quad f_{16} = 80, \quad f_{15} = 75, \quad f_9 = 40, \quad f_{11} = 55, \quad f_{10} = 50.$$

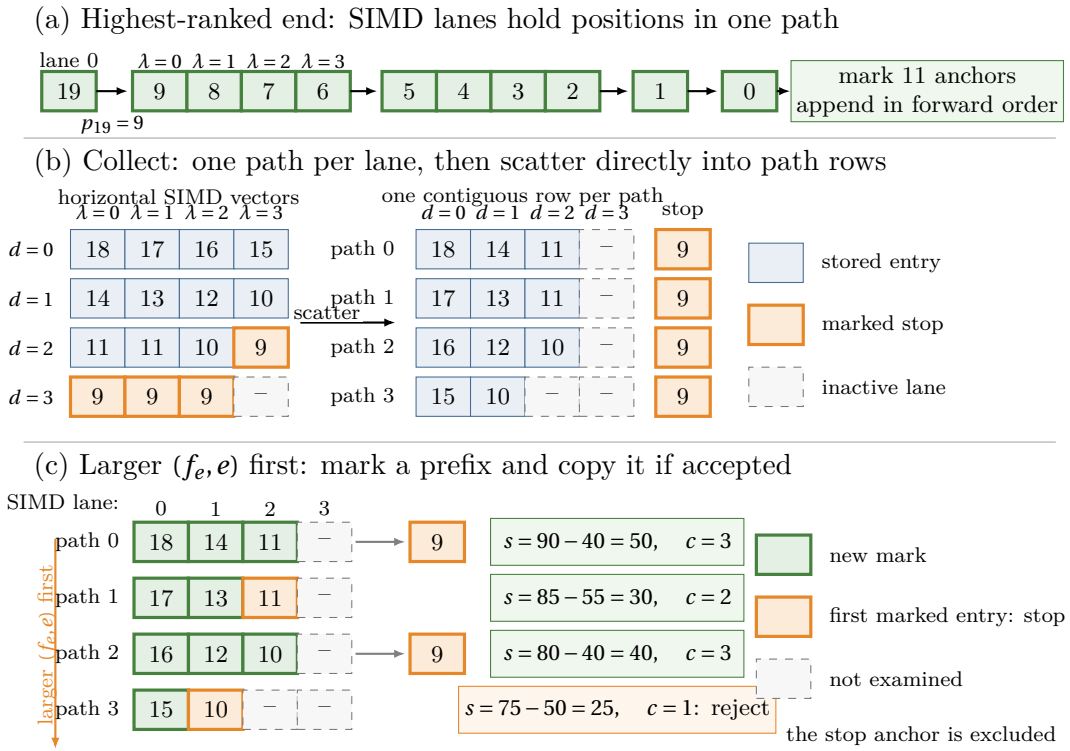


Figure 5.8: BioseqDB SIMD backtracking with four lanes. Every SIMD vector is a connected horizontal group. Panel (a) shows the special handling of the first path. Panel (b) shows horizontal collection vectors on the left and the path-row buffer on the right. Indexed scatter creates this layout directly. Panel (c) reads one row at a time, so the path with the larger (f_e, e) pair claims a shared anchor first.

The accepted output in this example is

$$C = [(100, 11, 0), (50, 3, 11), (30, 2, 14), (40, 3, 16)],$$
$$A = [a_0, \ldots, a_9, a_{19} \mid a_{11}, a_{14}, a_{18} \mid a_{13}, a_{17} \mid a_{10}, a_{12}, a_{16}].$$

Each record in C is (score, count, start). Path 3 marks a_{15} before testing its thresholds. Its rejected anchor therefore remains claimed but is not copied into A.

The following listing uses the same symbols as Algorithms 12 and 13. For one stored row, define

$$\mathrm{FirstMarked}(t, V_\lambda, o, g) = \min\big(\{u \mid 0 \le u < g \wedge t_{V_{\lambda,o+u}} = 1\} \cup \{g\}\big).$$

It returns g when none of the g entries is marked. The first-path vector run is used when

$$\mathrm{Run}_L(j, c) \iff j \ge L-1 \ \wedge \ c \le n - L \ \wedge \ \bigwedge_{u=0}^{L-1} p_{j-u} = j - u - 1.$$

Here V_λ means row λ of V. ℓ_λ is the number of stored entries in that row, b_λ is its stop, and α_λ says whether its collection lane is active. During marking, o is the next unread row offset and g is the number of entries in the current vector. Square brackets around a condition have value one when it is true and zero otherwise. Every equation indexed by λ below is evaluated for $0 \le \lambda < q$ in one SIMD vector. Indexed reads are made only in lanes whose stated condition is true. Algorithm 14 then gives the complete procedure. The notation $a_{V_{\lambda,c-1}}, \ldots, a_{V_{\lambda,0}}$ denotes an empty sequence when $c = 0$.

Algorithm 14 BioseqDB SIMD path collection and backtracking

Require: anchors $a[0\ldots n)$, scores f, predecessors p, thresholds S,K, width L, and empty outputs C,A

function ChooseCutsSIMD($\mathbf{e},p,t,V,n$)
  $q\leftarrow|\mathbf{e}|,\quad \mathbf{j}\leftarrow\mathbf{e},\quad \boldsymbol{\ell}\leftarrow\mathbf{0},\quad \mathbf{b}\leftarrow-\mathbf{1},\quad \boldsymbol{\alpha}\leftarrow\mathbf{1}$
  for $d\leftarrow0,\ldots,n-1$ while $\sum_{\lambda=0}^{q-1}\alpha_\lambda\neq0$ do
    $h_\lambda\leftarrow[\alpha_\lambda=1\wedge t_{j_\lambda}=1],\quad \eta_\lambda\leftarrow\alpha_\lambda\wedge\neg h_\lambda$
    $b_\lambda\leftarrow j_\lambda$ if $h_\lambda=1$, and $(V_{\lambda,d},\ell_\lambda)\leftarrow(j_\lambda,\ell_\lambda+1)$ if $\eta_\lambda=1$ scatter
    $j_\lambda\leftarrow\begin{cases}p_{j_\lambda}, & \eta_\lambda=1\\ -1, & \text{otherwise}\end{cases},\quad \alpha_\lambda\leftarrow[j_\lambda\geq0]$
  abort this call if $\sum_{\lambda=0}^{q-1}\alpha_\lambda\neq0$
  return $(\boldsymbol{\ell},\mathbf{b})$
function BacktrackSIMD(a,f,p,S,K,L,C,A)
  allocate $V\in\mathrm{int32}^{L\times n}$, and set $t[0\ldots n)\leftarrow0$ 0: unclaimed, 1: claimed
  $Z\leftarrow\mathrm{sort}_{\uparrow}[(f_i,i)\mid0\leq i<n\wedge f_i\geq S],\quad k\leftarrow|Z|-1$
  if $k\geq0$ then
    $e\leftarrow Z_k.\mathrm{index},\quad j\leftarrow e,\quad c\leftarrow0$
    while $j\geq0\wedge c<n$ do
      $g\leftarrow\begin{cases}L, & \mathrm{Run}_L(j,c)\\ 1, & \text{otherwise}\end{cases}$
      $(V_{0,c+u},t_{j-u})\leftarrow(j-u,1)$ for $0\leq u<g$, then $(j,c)\leftarrow(p_{j-g+1},c+g)$
    abort this call if $j\geq0$
    if $f_e\geq S\wedge c\geq K$ then append $(f_e,c,|A|)$ to C and $(a_{V_{0,c-1}},\ldots,a_{V_{0,0}})$ to A
    $k\leftarrow k-1$
  while $k\geq0$ do
    $q\leftarrow\min(L,k+1),\quad \mathbf{e}\leftarrow(e_0,\ldots,e_{q-1}),\quad e_\lambda\leftarrow Z_{k-\lambda}.\mathrm{index}$
    if $(S>0\vee K>0)\wedge\bigwedge_{\lambda=0}^{q-1}t_{e_\lambda}=1$ then $k\leftarrow k-q$ and continue
    $(\boldsymbol{\ell},\mathbf{b})\leftarrow$ ChooseCutsSIMD($\mathbf{e},p,t,V,n$)
    for $\lambda\leftarrow0$ to $q-1$ do
      $c\leftarrow0,\quad o\leftarrow0$
      while $o<\ell_\lambda$ do
        $g\leftarrow\min(L,\ell_\lambda-o),\quad r\leftarrow\mathrm{FirstMarked}(t,V_\lambda,o,g)$
        $t_{V_{\lambda,o+u}}\leftarrow1$ for $0\leq u<r$, and $c\leftarrow c+r$
        if $r<g$ then $b_\lambda\leftarrow V_{\lambda,o+r}$ and break, otherwise $o\leftarrow o+g$
      $s\leftarrow\begin{cases}f_{e_\lambda}, & b_\lambda<0\\ f_{e_\lambda}-f_{b_\lambda}, & b_\lambda\geq0\end{cases}$
      if $s\geq S\wedge c\geq K$ then append $(s,c,|A|)$ to C and $(a_{V_{\lambda,c-1}},\ldots,a_{V_{\lambda,0}})$ to A
    $k\leftarrow k-q$
  return (C,A)

Collection does not modify t. This lets several lanes collect a shared suffix before any of them claims it. The mark-and-copy step then handles rows with larger (f_e,e) first. It loads successive entries from one row into horizontal SIMD lanes, finds the first entry with $t=1$, and permanently marks only the prefix before that entry. The stop b_λ can therefore change from the stop found during collection to an anchor just marked by an earlier row in the same batch.

If $b_\lambda<0$, the row score is f_{e_λ}. Otherwise it is $f_{e_\lambda}-f_{b_\lambda}$. The boundary is excluded. Only the newly marked prefix contributes to the count and to A. The output records stay in decreasing (f_e,e) order, while anchors inside each record are copied in forward path order. The backtracker does not apply `max_drop`, does not repeat a count pass, and does not sort complete chains by coordinate.

Two checks handle already marked ends and predecessor cycles. If every start in a later batch is already marked and $S>0$ or $K>0$, the code skips the empty rows immediately.

When $S = K = 0$, this shortcut is disabled, and an already-marked end can produce a score-zero record with zero anchors. If an unmarked path is still active after n links, the whole backtracking call reports a predecessor cycle before that batch marks or copies anchors. Exact SIMD B1 and bounded SIMD B1 both call this same backtracking function. Their difference lies in how the forward pass produces f and p, not in the recovery step above.

5.6.5. Effect of repeated anchors on the first-path vector run

The first loop in Algorithm 14 can process L anchors at once only when $\mathsf{Run}_L(j, c)$ is true. The score and equal-score tie rule select the first chain end. The predicate then requires each of the next L predecessor links to move to the preceding slot in the coordinate-sorted anchor array.

A repeat-derived anchor is a retained match between a selected minimizer in one block and an occurrence in another block. A same-target anchor group contains retained anchors with the same target sequence, target coordinate, and strand. Its members have different query coordinates in these constructions. A predecessor path contains at most one member of such a group. Other group members still occupy array slots. They can make $p_j < j - 1$, even when consecutive path anchors are close in both sequences.

Reference and query structures

Figure 5.9 shows the three constructions used to measure this effect. The drawings show representative minimizer links and are not to scale.

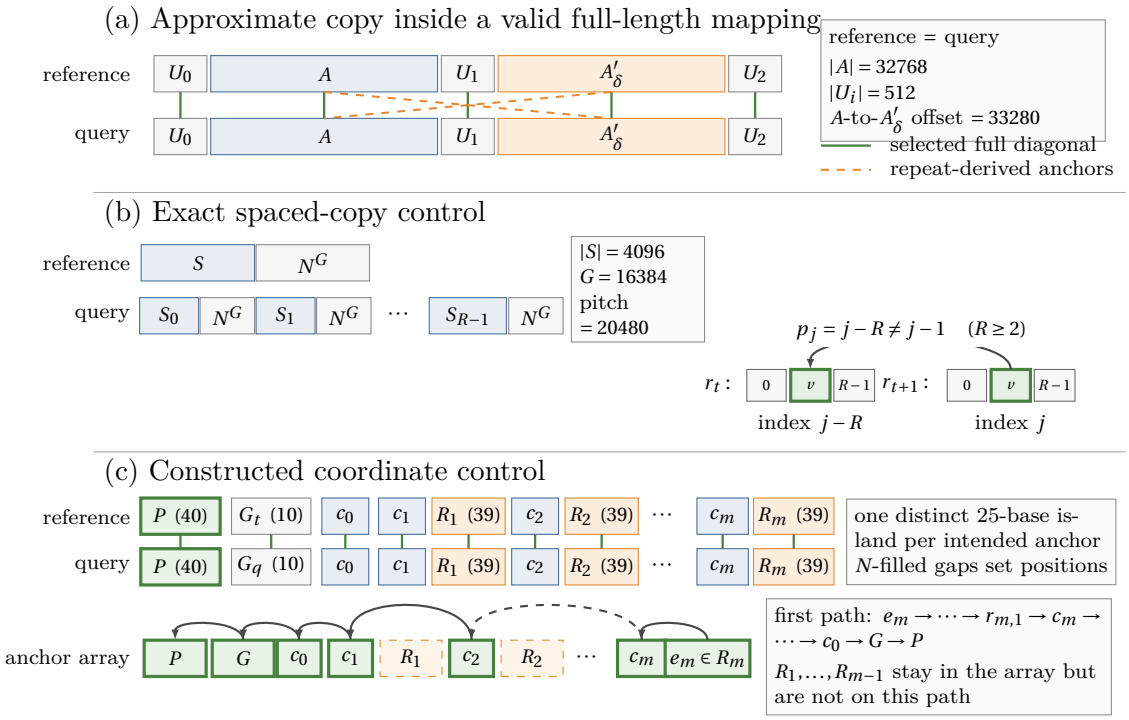


Figure 5.9: Reference and query structures used to measure the first-path Run_L branch. Green vertical lines connect matching blocks on the selected diagonal. Green boxes in the anchor-array row show the selected first path. Dashed orange links in panel (a) are repeat-derived. Dashed orange boxes in panel (c)'s anchor-array row are outside that path. Panel (a) keeps a full-length mapping while adding anchors from an approximate copy. Panel (b) isolates the array-index effect of exact copies. Panel (c) constructs fixed anchor coordinates from distinct minimizer islands.

The mosaic construction in panel (a) uses a reference and query with identical bytes:

$$T = Q = U_0\, A\, U_1\, A'_\delta\, U_2.$$

Here $|A| = 32768$, every U_i has length 512, and the complete sequence has length 67072. The configured parameter δ sets the substitution fraction between A and A'_δ. The generator applies the exact count

$$s_\delta = \left\lfloor \delta |A| + \frac{1}{2} \right\rfloor.$$

Every substituted base is changed to a different base. The realized fraction of positions at which the two blocks differ is therefore

$$\widehat{\delta} = \frac{s_\delta}{|A|}.$$

This value is fixed when the input is generated. The configured values are

$$\delta \in \{0.20, 0.10, 0.05, 0.02, 0.01, 0.005, 0.002, 0\}.$$

Within each sequence seed, the substitutions for a smaller δ form a subset of those used for every larger value. Both T and Q contain the same A and A'_δ blocks, so their query/reference error rate is zero. The quantities δ and $\widehat{\delta}$ describe only the internal difference between the two blocks inside each sequence.

An independently generated A' provides a separate no-copy control. Across the three tested inputs, its selected first path contained no same-target group larger than one. The full diagonal remains a valid highest-scoring path in every case. Table 5.3 lists the realized divergence and corresponding first-path measurements.

The exact-copy control in panel (b) uses

$$T = SN^G, \qquad Q = (SN^G)^R, \qquad |S| = 4096, \qquad G = 16384.$$

Here R is the number of query copies. The N-filled spacer blocks emit no minimizers and make the copy pitch 20480. In the generated inputs, each retained target minimizer produced one anchor for every query copy. Let $j_{t,v}$ be the array index for copy v, where $0 \le v < R$. Consecutive target minimizers satisfy

$$p_{j_{t+1,v}} = j_{t,v} = j_{t+1,v} - R.$$

For $R \ge 2$, the selected path contains no run of at least two unit predecessor relations. The construction therefore predicts zero vector batches for every tested $L \ge 2$.

Panel (c) is a constructed coordinate control. Every intended anchor is a distinct 25-base island that emits one minimizer for $k = 15$ and $w = 10$. The remaining positions contain N. This control does not estimate the frequency of the structure in biological sequence. Write $r_{k,\ell}$ for the ℓ-th anchor in R_k. In the equations below, c_k and $r_{k,\ell}$ denote their target-query coordinate pairs. Let $s_k = 1$ for odd k and $s_k = -1$ for even k. The following pairs give the target and query coordinate increments:

$$\begin{aligned} c_k - c_{k-1} &= (3500,\ 3500 - 100s_k), \\ r_{k,\ell} - c_k &= (500 + 38(\ell - 1),\ 500 - 450s_k + 38(\ell - 1)), \quad 1 \le \ell \le 39. \end{aligned}$$

The `map-ont` preset sets the short-range chaining bandwidth to 500. Every anchor in R_k and c_{k+1} differ by 550 between the target and query distances, so those anchors fail the bandwidth check. The preceding c_k remains valid, and the recorded predecessor array confirms $p(c_{k+1}) = c_k$. The 39 anchors of R_k lie between c_k and c_{k+1} in array order, which gives an index jump of 40.

Let h be the last anchor in P, and let $e_k = r_{k,39}$ be the last anchor in R_k. With the unmodified `map-ont` scoring coefficients, the construction gives

$$f(h) = 600, \qquad f(c_k) = 88, \qquad f(e_k) = 615.$$

All e_k values share the maximum score. The experiment orders equal-score ends by decreasing anchor-array index, matching BioseqDB's tie rule. This selects e_m. The first measured path passes through R_m and then follows $c_m, \ldots, c_0, G, P$. The worst-case argument for all minimap2 cut searches concerns the total work over every selected chain end and remains a separate result.

Metrics and results

Minimap2 2.31-r1302 was run with the `map-ont` preset. Filtering and scoring parameters were not changed. Instrumentation recorded the score and predecessor arrays. The only candidate-order change placed equal-score ends in decreasing anchor-array index, matching BioseqDB's tie rule. The tested widths were $L \in \{2, 4, 8, 16, 32, 64\}$. The branch counters were recomputed from the recorded arrays. Mosaic and spaced-copy paths were checked against coordinate-based oracles, while the coordinate control was checked against its predeclared

predecessor array. The mosaic and spaced-copy inputs used three deterministic sequence seeds.

Let q be the number of anchors on the first predecessor path. Let B_L be the number of evaluations for which Run_L is true, E_L the number of loop evaluations, and P_L the number that pass both array bounds. Three different ratios are useful:

$$\mathsf{Fire}_L = \frac{B_L}{E_L}, \qquad \mathsf{Contiguous}_L = \frac{B_L}{P_L}, \qquad \mathsf{Coverage}_L = \frac{LB_L}{q}.$$

The first ratio is the observed branch-firing rate. The second isolates the predecessor test after the bounds pass. The third is the fraction of path anchors consumed by vector batches. A successful evaluation consumes L anchors, so firing rate and anchor coverage are not interchangeable. An empty denominator gives an undefined value.

For maximal runs of unit predecessor relations with lengths r_i, the greedy loop gives

$$B_L = \sum_i \left\lfloor \frac{r_i}{L} \right\rfloor, \qquad E_L = q - (L-1)B_L.$$

In the coordinate control, the path ends at anchor-array index zero, so $p_0 = -1 = 0 - 1$ contributes one unit relation. The $m = 1$ path is one run of length 91. For $m \geq 2$, the only maximal runs have lengths 39 and 52. Thus

$$B_L = \left\lfloor \frac{39}{L} \right\rfloor + \left\lfloor \frac{52}{L} \right\rfloor, \qquad q = m + 90.$$

For fixed L with $B_L > 0$, both Fire_L and $\mathsf{Coverage}_L$ decrease as $\Theta(1/m)$. For $L = 64$, both values are zero for every $m \geq 2$.

Table 5.2: Observed first-path branch-firing rate $100B_L/E_L$, in percent. The mosaic rows and the $R = 1$ spaced-copy row report medians over three sequence seeds. The $R \geq 2$ spaced-copy row covers the tested powers of two from 2 through 64, all of which gave zero. Coordinate-control values are deterministic and agreed with the recorded counters.

Input	$L = 2$	$L = 4$	$L = 8$	$L = 16$	$L = 32$	$L = 64$
Mosaic, independent A'	100.00	99.94	99.74	99.24	97.01	91.12
Mosaic, exact $A' = A$	1.17	0.59	0.29	0.14	0.07	0.02
Spaced copies, $R = 1$	100.00	98.97	94.12	88.89	80.00	66.67
Spaced copies, $R \geq 2$	0.00	0.00	0.00	0.00	0.00	0.00
Coordinate control, $m = 2$	95.74	84.62	45.45	29.41	6.67	0.00
Coordinate control, $m = 1024$	4.21	2.10	0.96	0.48	0.19	0.00

Table 5.3 shows the intermediate mosaic cases. The grouped-path column gives the percentage of selected-path anchors whose same-target group contains more than one anchor. Such groups add off-path array entries that can interrupt unit predecessor steps. This percentage does not count the interrupted steps.

Table 5.3: Configured and realized internal divergence in the mosaic construction. Each entry in the last three columns gives the median followed by the observed minimum and maximum over three sequence seeds, in percent. The firing column is $100\,\mathrm{Fire}_4 = 100B_4/E_4$. The coverage column is $100\,\mathrm{Coverage}_4 = 100(4B_4)/q$. The independent block is a separate control. Its realized divergence is measured directly from the two generated blocks. The third column gives its median, whose range was 74.930 to 75.186 percent.

Case	s_δ	$100\widehat{\delta}$ (%)	Grouped path (%)	$100\,\mathrm{Fire}_4$ (%)	$100\,\mathrm{Coverage}_4$ (%)
$\delta = 0$	0	0.000	97.69 [97.64, 97.69]	0.59 [0.58, 0.59]	2.31 [2.29, 2.33]
$\delta = 0.002$	66	0.201	94.33 [94.12, 94.34]	0.97 [0.92, 1.10]	3.77 [3.60, 4.25]
$\delta = 0.005$	164	0.500	89.55 [89.47, 89.68]	1.56 [1.47, 1.79]	5.96 [5.64, 6.79]
$\delta = 0.010$	328	1.001	82.24 [82.19, 82.29]	2.61 [2.57, 2.80]	9.69 [9.55, 10.34]
$\delta = 0.020$	655	1.999	69.33 [68.93, 69.42]	5.14 [5.08, 5.30]	17.82 [17.64, 18.28]
$\delta = 0.050$	1638	4.999	41.41 [41.33, 41.63]	14.97 [14.97, 15.38]	41.32 [41.32, 42.09]
$\delta = 0.100$	3277	10.001	17.23 [16.81, 17.64]	37.99 [37.64, 38.72]	71.02 [70.71, 71.65]
$\delta = 0.200$	6554	20.001	3.04 [2.66, 3.36]	79.41 [77.58, 81.93]	93.91 [93.26, 94.77]
Independent A'	n/a	75.043	0.00 [0.00, 0.00]	99.94 [99.90, 100.00]	99.98 [99.98, 100.00]

To model seed preservation, let π denote the deterministic mutation ranking used by the generator. We model π as a uniform random permutation of the $M = |A|$ block positions, independent of A. Once a sequence seed is fixed, the generated input and the minimap2 run are deterministic. For a fixed span of r bases, the probability that none of its positions belongs to the fixed-size mutation set is

$$H_r(M, s_\delta) = \frac{\binom{M-r}{s_\delta}}{\binom{M}{s_\delta}} = \prod_{a=0}^{r-1} \frac{M - s_\delta - a}{M - a}.$$

An aligned, same-orientation k-mer is unchanged with probability H_k. For fixed k, M large, and $\widehat{\delta}$ bounded away from one,

$$H_k(M, s_\delta) = (1-\widehat{\delta})^k \exp\left(O\left(\frac{\widehat{\delta}k^2}{M}\right)\right).$$

Counting individual substituted positions gives the upper bound below. Subtracting their pairwise overlaps gives the lower bound:

$$k\widehat{\delta} - \binom{k}{2}\frac{s_\delta(s_\delta-1)}{M(M-1)} \;\le\; 1 - H_k(M, s_\delta) \;\le\; k\widehat{\delta}.$$

Consequently, the expected number of disrupted aligned k-mer spans is

$$(M-k+1)\left[1 - H_k(M, s_\delta)\right] = \Theta(Mk\widehat{\delta})$$

when $0 < k\widehat{\delta} \ll 1$. This count covers all aligned k-mer spans before minimizer selection.

A selected minimizer also depends on its surrounding w-window. Let I contain the selected minimizer occurrences in A for which one supporting w-window lies wholly inside A, and let $n_I = |I|$. Let Y_δ count members of I that remain selected at the corresponding block-relative coordinate and orientation with the same exact k-mer in A'_δ. In this experiment, $k = 15$ and $w = 10$, so a complete minimizer window spans $k + w - 1 = 24$ bases. An unchanged supporting window is sufficient to preserve the selected occurrence. An unchanged selected k-mer is necessary. Conditional on A, expectation over π therefore satisfies

$$n_I H_{24}(M, s_\delta) \;\le\; \mathbb{E}_\pi[Y_\delta \mid A] \;\le\; n_I H_{15}(M, s_\delta).$$

The selected windows overlap, so their preservation events are dependent. The calculation does not assume that neighboring minimizers change independently. For $k = 15$, minimap2's masked `hash64` transformation is one-to-one on canonical 30-bit codes. Two different canonical 15-mers therefore cannot share an encoded key. Requiring the same orientation also removes the canonical match between a k-mer and its reverse complement.

Figure 5.10 compares this local model with the measured path. The gray reference region is $100\alpha H_{24}$ to $100\alpha H_{15}$, where

$$\alpha = \frac{2M}{2M + 3|U_i|} = 0.9771$$

is the fraction of sequence bases in the two repeated blocks. Multiplication by α provides a simple length weight when comparing preservation probabilities with the grouped-path percentage. It does not account for variation in minimizer or anchor density. The gray region is a visual probability reference. It is not a confidence interval or a bound on the chaining metric.

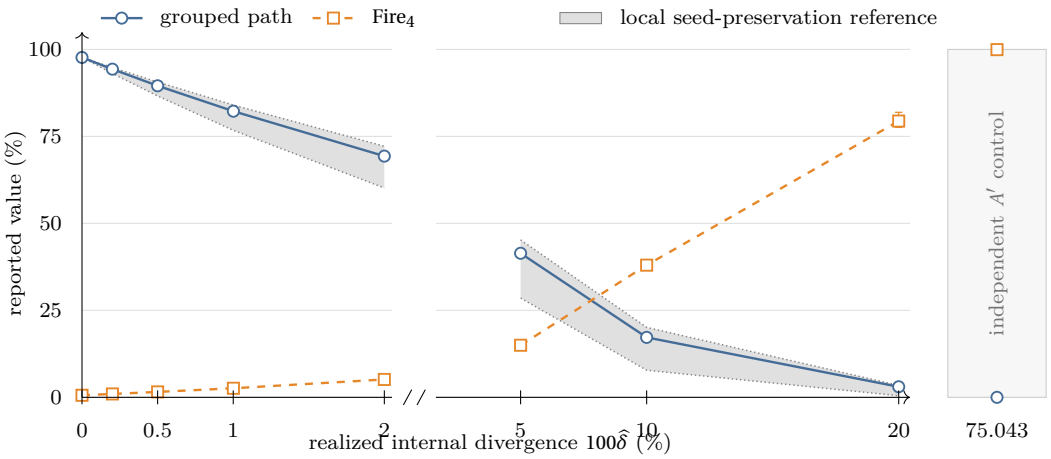


Figure 5.10: Internal divergence, grouped-path percentage, and $L = 4$ branch firing in the mosaic experiment. Markers report medians over three deterministic sequence seeds. Whiskers span their observed minimum and maximum. The gray region shows the local seed-preservation reference defined in the text. It is not a confidence interval or a bound on the chaining metric. The nested sweep uses separate linear scales from zero to two percent and from five to twenty percent. The axis break covers the interval with no tested case. The independent A' control appears in the separate shaded column.

In the exact mosaic, a median of 97.69 percent of selected-path anchors belonged to same-target groups larger than one. The independent control had no such group. For $L = 4$, their median firing rates were 0.59 percent and 99.94 percent. Their corresponding anchor coverages were 2.31 percent and 99.98 percent. Within the nested sweep, every increase in $\hat{\delta}$ was accompanied by a lower median grouped-path percentage and a higher median Fire_4. Every nonzero-divergence grouped-path median lies inside the local reference region in Figure 5.10. At zero divergence, the measured median is 0.02 percentage points below the reference. For the first three nonzero realized divergence levels, write their ascending values as d_1, d_2, d_3. Then

$$\left(\frac{\mathrm{Fire}_4(d_i) - \mathrm{Fire}_4(0)}{d_i}\right)_{i=1}^{3} \approx (1.89, 1.94, 2.02),$$

where firing rates and divergence are fractions. The three similar ratios show an approximately linear increase over these three low-divergence points.

A second local model shows why the response can contain first or higher powers of $\widehat{\delta}$. Suppose Run_L becomes true only after mutations affect $c \geq 1$ disjoint shared k-mer spans. Assume that every other path property stays fixed. Counting all ways to place substitutions in these spans gives

$$P_{\mathrm{break},c}(M, s_\delta) = \sum_{a=0}^{c} (-1)^a \binom{c}{a} H_{ak}(M, s_\delta).$$

This is the probability that every one of the c spans contains at least one substituted base. For fixed c and k, take M to infinity and $\widehat{\delta}$ to zero with $s_\delta = M\widehat{\delta} \geq c$ and $ck = o(M)$. Then

$$P_{\mathrm{break},c}(M, s_\delta) = \Theta\left((k\widehat{\delta})^c\right)$$

in this limit. For $c = 1$, the probability is proportional to $k\widehat{\delta}$. If $s_\delta < c$, then $P_{\mathrm{break},c} = 0$ because fewer than c substitutions cannot reach all c disjoint spans.

This calculation describes preservation of shared seed keys. Chaining scores are evaluated later. All 27 mosaic mappings had one unique maximum-score end. Query minimizer filtering removed none. Reference occurrence filtering removed no mosaic seed hits. The retained reference occurrence counts were one or two, below the cutoff of ten. Shared keys between A and A'_δ created cross-copy anchors in same-target groups. Those off-path anchors remained in the coordinate-sorted array and could make the selected predecessor skip an array slot. They became less frequent as divergence increased.

The local model does not give a universal formula for Fire_L. Minimizer reselection, overlapping windows, anchor coordinates, predecessor choices, and unit-run lengths also affect Run_L. The monotone result is an empirical property of this crafted $L = 4$ sweep. Some other widths had small departures because unit-run lengths are discrete. For fixed L, coverage is determined by firing rate:

$$\mathsf{Coverage}_L = \frac{L\,\mathsf{Fire}_L}{1 + (L-1)\,\mathsf{Fire}_L}.$$

This descriptive comparison uses three deterministic sequence seeds.

The spaced-copy control reached zero firing and zero anchor coverage at $R = 2$, and both remained zero through $R = 64$. In this construction, retained exact-copy anchors removed every tested vector opportunity. These data do not establish a scaling law or the frequency of this pattern in real reads.

The coordinate sweep accepted $m = 1$ and every tested power of two from 2 through 1024. Every measured value agreed with the closed form above. At $m = 1024$, anchor coverage was 8.08 percent for $L = 2$ and 7.90 percent for $L = 4$. It was 7.18 percent for both $L = 8$ and $L = 16$, 5.75 percent for $L = 32$, and zero for $L = 64$.

Random-sequence controls

This experiment measures how often Run_L is true for anchors obtained from random DNA. No anchor coordinates were predeclared. Each target began with independently sampled A, C, G, and T bases with equal probability. A random source segment was then copied into N non-overlapping target intervals. The query was a randomly oriented copy of that segment. The mutation model used a substitution probability of 0.001. The insertion and deletion tests each used probability 0.0005 at every source base visited by the generator. Candidate insertion and deletion lengths were sampled from a Gaussian distribution with a mean and standard deviation of one base. They were rounded to integers and clipped at zero.

Zero-length draws caused no edit. Deletions were clipped at the end of the oriented source. Minimap2 used its default $k = 15$, $w = 10$, occurrence filtering, and chaining scores. Mapping used one thread.

The instrumentation reported the branch-firing percentage $100B_L/E_L$ for the first predecessor path from the initial dynamic-programming call. Its starting anchor was selected by score and the equal-score tie rule. The experiment tested $L \in \{4, 8, 16\}$. Equal scores were ordered by decreasing anchor-array index to match BioseqDB. A value is defined only when minimap2 found a score-qualified chain end and the measured path had $E_L > 0$. Runs without such a path are omitted because no firing percentage exists when $E_L = 0$. A displayed zero means that a measured path existed and $B_L = 0$. Neither experiment runner saved minimap2's PAF output. These results describe the internal predecessor path. Whether minimap2 later emitted each path as an alignment is unknown.

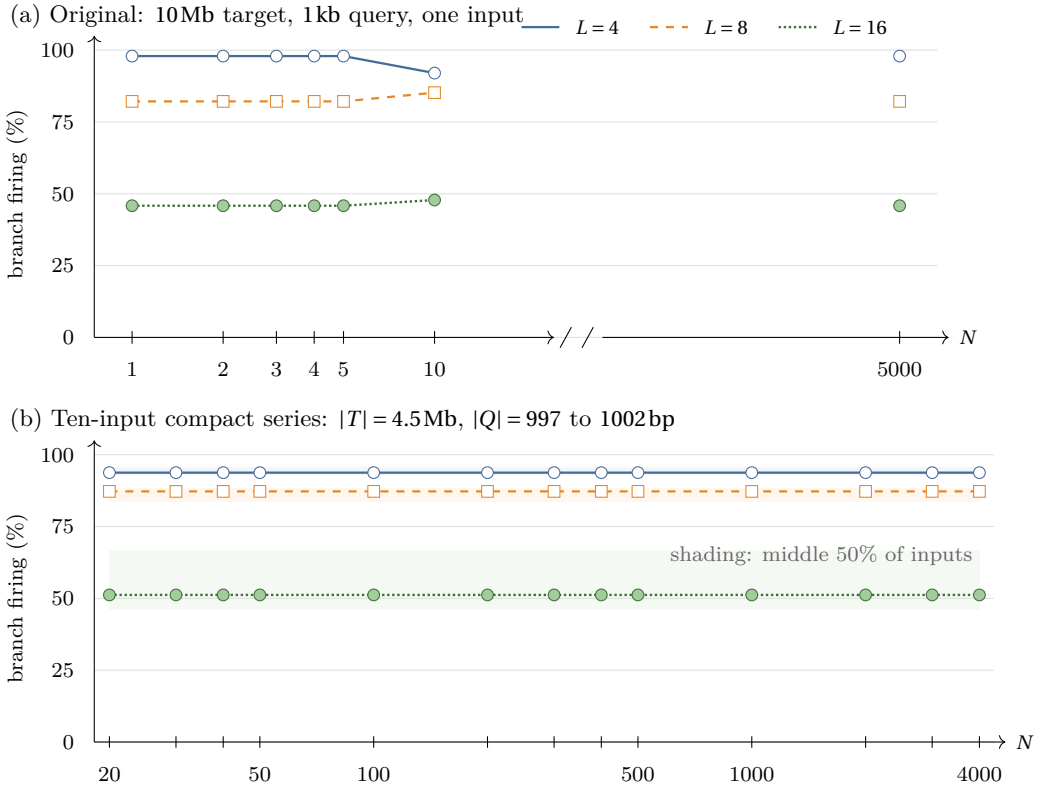


Figure 5.11: First-path branch firing on random-sequence occurrence sweeps. The plotted value is $100B_L/E_L$. Horizontal positions are logarithmic in N within each unbroken axis segment. Panel (a) shows every defined result from the original sweep over N with $k = 15$ and $w = 10$. The axis break spans the tested values between $N = 20$ and $N = 4000$, for which no score-qualified chain end was available. The $N = 5000$ point is plotted separately. Panel (b) shows the median over ten generated inputs at each tested N. Its bands span the middle half of those ten values. Every panel (b) run had a defined first path.

Panel (a) records the original run. Its missing middle prevents an estimate of the trend over N. The ten-input compact series produced a defined value for all 13 selected N values spanning 20 to 4000. Its median firing rates were 93.75 percent for $L = 4$, 87.23 percent for $L = 8$, and 51.19 percent for $L = 16$. For each generated input, all N values used the same sequence background. The measured path length and branch counters stayed constant as N

changed. The selected copy and absolute anchor-array positions could change. The recorded Run_L outcomes produced the flat lines. The earlier three-input compact pilot is not plotted.

The size controls kept k, w, mutation probabilities, and minimap2 parameters fixed. They tested $N = 20$ and $N = 4000$. Figure 5.12 shows the three target/query size pairs that produced defined percentages: approximately 4.5 Mb/1 kb, 40 Mb/4 kb, and 80 Mb/8 kb. The 10 Mb/1 kb, 20 Mb/2 kb, and 40 Mb/1 kb controls produced no defined percentage at either tested N.

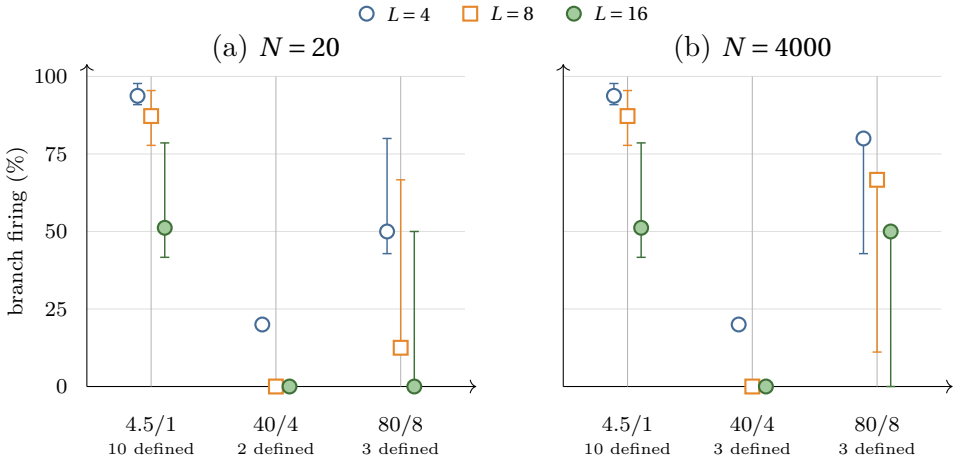


Figure 5.12: Median first-path branch firing for random-sequence size controls with a defined path. Each point is the median of the defined per-input percentages. The number of contributing inputs is printed below it. The horizontal labels give target length in megabases followed by query length in kilobases. Query lengths were 997 to 1002 bases, 3998 to 4004 bases, and 7999 to 8008 bases. Vertical whiskers span the smallest and largest per-input percentages. Within each size pair, markers and whiskers are offset horizontally only for visibility. A plotted zero means that a measured path existed and $B_L = 0$. Runs with no score-qualified chain end are omitted. At 40 Mb/4 kb and $N = 20$, one of three generated inputs had no defined percentage, so the median uses the other two. The omitted target/query size pairs are 10 Mb/1 kb, 20 Mb/2 kb, and 40 Mb/1 kb at both tested N values.

In the 40-Mb/4-kb and 80-Mb/8-kb controls, default occurrence filtering left short first paths. Query-side filtering removed none of their generated minimizers. For both sizes, the adaptive reference cutoff `mid_occ` remained 10. Most repeated source minimizers had approximately N reference occurrences. A few also matched elsewhere in the random target. These counts exceeded `mid_occ`. Reference occurrence filtering removed between 98.79 and 98.90 percent of the raw anchor hits. For each consecutive group above the cutoff, `occ_dist`=500 set the number of minimizers that could remain from its query-coordinate span. It kept those with the smallest reference occurrence counts. Every defined 4-kb input had $q = 8$, while the 8-kb inputs had $q \in \{15, 16, 17\}$. The longer query produced a larger limit under this rule. Target length also affects the adaptive cutoff, so this comparison does not isolate query length.

The 4.5-Mb/1-kb series used larger adaptive cutoffs. They were 21 for $N = 20$ and 4001 for $N = 4000$. Reference occurrence filtering retained every anchor hit. On each validated first path, every predecessor index was one less than the current index. These paths contained 163 to 187 anchors and gave the higher firing percentages shown in Figure 5.12.

After filtering, Run_L is evaluated from the predecessor indices on these short paths. All recorded bounds tests passed. A displayed zero therefore means that the predecessor-index

equalities in Run_L failed at every loop evaluation. The observed first-path profiles were

q	Fire_4	Fire_8	Fire_{16}
8	$1/5 = 20\%$	$0/8 = 0\%$	$0/8 = 0\%$
15	$3/6 = 50\%$	$1/8 = 12.5\%$	$0/15 = 0\%$
16	$3/7 = 42.86\%$	$1/9 = 11.11\%$	$0/16 = 0\%$
17	$4/5 = 80\%$	$2/3 = 66.67\%$	$1/2 = 50\%$

Every defined 40-Mb/4-kb input had the first profile. It provided one four-wide batch and no eight-wide or sixteen-wide batch. For the 80-Mb/8-kb controls, the path lengths at $N = 20$ were 16, 15, and 17. At $N = 4000$ they were 16, 17, and 17. A 17-anchor path was the only observed profile with one sixteen-wide batch. That batch was followed by one scalar evaluation, so $B_{16} = 1$, $E_{16} = 2$, and the firing percentage was 50 percent. The other profiles gave zero. These discrete outcomes produce the whisker from 0 to 50 percent in Figure 5.12.

The 40-Mb and 80-Mb conditions contained only two or three inputs with a defined percentage. Their whiskers are descriptive ranges. The results do not establish a length trend. Minimap2 required a chaining score of at least 40. The omitted 40-Mb/4-kb input at $N = 20$ had no anchor reaching that threshold.

These measurements describe opportunities for the first-path vector branch. They do not measure BioseqDB execution time. The cost of failed checks and the speed of successful vector batches must be evaluated by a BioseqDB runtime benchmark. Initial dynamic-programming and later RMQ calls were recorded separately. Table 5.2 and Figures 5.11 and 5.12 report only the initial dynamic-programming call used by the special first-path loop.

5.6.6. BioseqDB SIMD performance comparison

This test configuration used a 500,000,000-base reference and one query fragment whose configured mean length is 5,000,000 bases (with two samples).

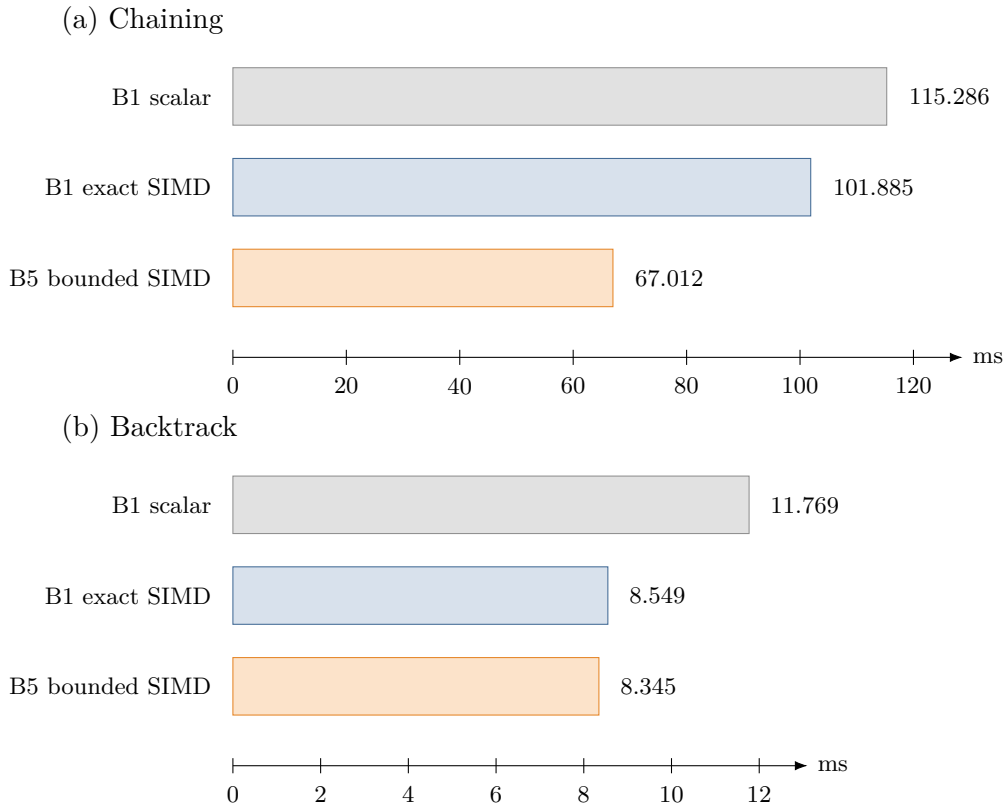


Figure 5.13: Recorded times for the three CPU modes

The figure above depicts only two runs and presents only quick prototyping test measurements to judge if we are going in right direction.

5.7. How minimap2 and G3SA split the dynamic program

In minimap2 B1, the algorithm fixes one target i and scans earlier sources j. Minimap2 RMQ also fixes one target, but obtains its main candidate from a tree query instead of scanning the earlier sources one by one. The G3SA short kernel and the triangular part of a tile instead fix one completed source j and update several later targets i in parallel. The rectangular part of a G3SA tile switches back to one target per thread. Both implementations fill a score f_i and predecessor information for each sorted anchor. Minimap2 stores the predecessor index j. G3SA stores the positive distance $i - j$. A possible link from earlier anchor j to later anchor i is one cell of the matrix in Figure 5.14. The figure compares only the direction in which work is assigned. The tested cells and scoring rules differ.

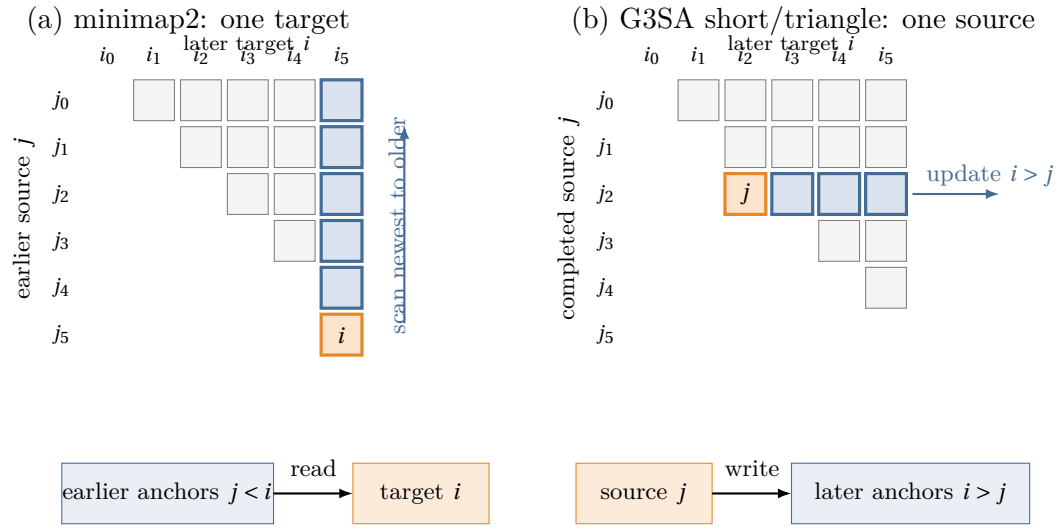


Figure 5.14: Two ways to assign predecessor pairs. Blue cells are processed in the shown step. Minimap2 fixes target i and reads earlier sources. The G3SA short kernel and the triangular part of a tile fix completed source j and update later targets. In the rectangular part, each thread instead fixes one later target and scans the completed source tile, as shown in Figure 5.16. The orange box marks the fixed anchor.

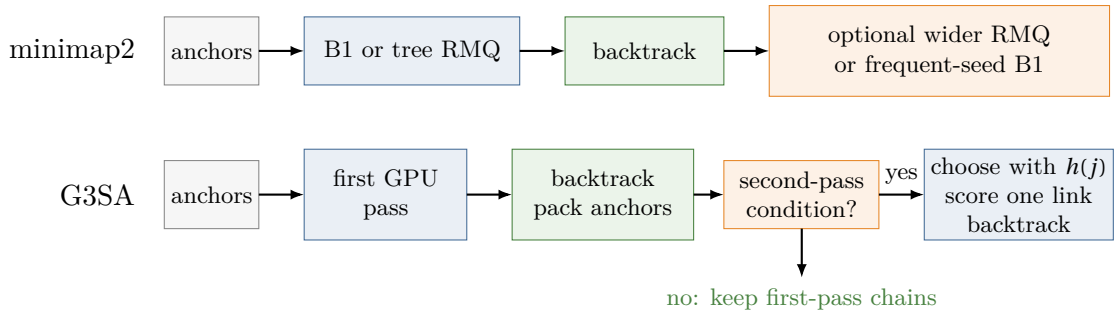


Figure 5.15: The chaining calls made by minimap2 and G3SA. G3SA always performs its first pass. Only reads satisfying the condition below enter its second pass.

Minimap2 chooses its first chaining call from the RMQ flag. With the flag disabled, it runs B1. With the flag enabled, it runs tree RMQ. When the first chains leave a long region unjoined, minimap2 may sort the retained anchors again and run RMQ with a wider bandwidth. This is always a second chaining call, but it is a second RMQ call only when the first call also used RMQ. A separate branch can restore seeds omitted as too frequent and rerun B1, but only when the RMQ flag is disabled.

Minimap2 chaining choices
mm_map_frag_core 273

G3SA always runs its first GPU pass. For each read separately, it backtracks and packs the retained anchors at the start of the part of the anchor array assigned to that read. Let n_u be the number of retained chains, Q the query length, and $R = q_{\text{last}} - q_{\text{first}}$, the query-coordinate distance from the first to the last anchor of the first retained chain. The second pass runs exactly when

$$n_u > 1 \quad \wedge \quad \big(Q - R > 1{,}000 \ \vee \ R > 0.10Q\big).$$

Other reads keep the chains produced by the first pass. The second pass uses the preliminary value $h(j)$, defined below, to choose one predecessor before it calculates the complete link score. Selected reads are then backtracked a second time.

5.8. G3SA first pass for one seed segment

For two anchors from the same seed segment, the first G3SA pass uses bandwidth zero, fixed span 15, maximum coordinate distance 5,000, and $\lambda_s = 0$. Every same-segment predecessor that G3SA stores satisfies

$$0 < i - j \le 5{,}000, \qquad 0 < d_r(j,i) = d_q(j,i) \le 5{,}000.$$

The complete link score from Equation 5.4 then reduces to

$$E(j,i) = \min\{15, d_r(j,i)\}.$$

Within one seed segment, this pass can link only anchors whose reference and query distances are equal. It is not the general minimap2 B1 scan. Links between different seed segments use another scoring branch and are not restricted by the zero bandwidth.

G3SA sets $f_i = 15$ and $p_i = -1$. In the short kernel and the triangular part of a tile, sources are used in increasing index order and the block waits after every source. In the rectangular part, all source scores are already complete, so one thread scans the tile for one later target without waiting after each source.

Algorithm 15 G3SA short-kernel exact-diagonal first pass for one single-segment range

Require: one sorted single-segment range $a_0, \ldots, a_{n-1}$
Ensure: scores f_i and predecessor distances $p_i = i - j$, or $p_i = -1$

```
for i ← 0 to n − 1 do
    f_i ← 15,   p_i ← −1
for j ← 0 to n − 1 do
    for all i ∈ {j + 1, ..., min(n − 1, j + 5000)} in parallel do
        d_r ← r_i − r_j,   d_q ← q_i − q_j
        if 0 < d_r = d_q ≤ 5000 then
            c ← f_j + min{15, d_r}
            if c ≥ f_i and c > 15 then
                f_i ← c,   p_i ← i − j
    WaitForThreads   all updates from source j are complete
```

The tiled kernel may test sources outside the 5,000-index limit that controls whether a predecessor is stored. A full tile starting at s loads sources s through $s + 1023$. A partial tile stops at the last available source. In a sufficiently long range, the rectangular scan can reach target $s + 6024$. It stores the tile result only when the selected predecessor satisfies $0 < p_i \le 5000$. A source more than 5,000 indices behind can therefore become the temporary winner. If it remains the winner, the kernel does not store that tile result, even when a lower-scoring predecessor inside the 5,000-index limit was tested.

The stored predecessor is a positive distance, so G3SA backtracking moves from i to $i - p_i$. The triangle checks source indices in increasing order. For the rectangular part, a later-target thread starts with the source at its own thread number and wraps at 1024. Equal scores can therefore select a different predecessor in the two loops.

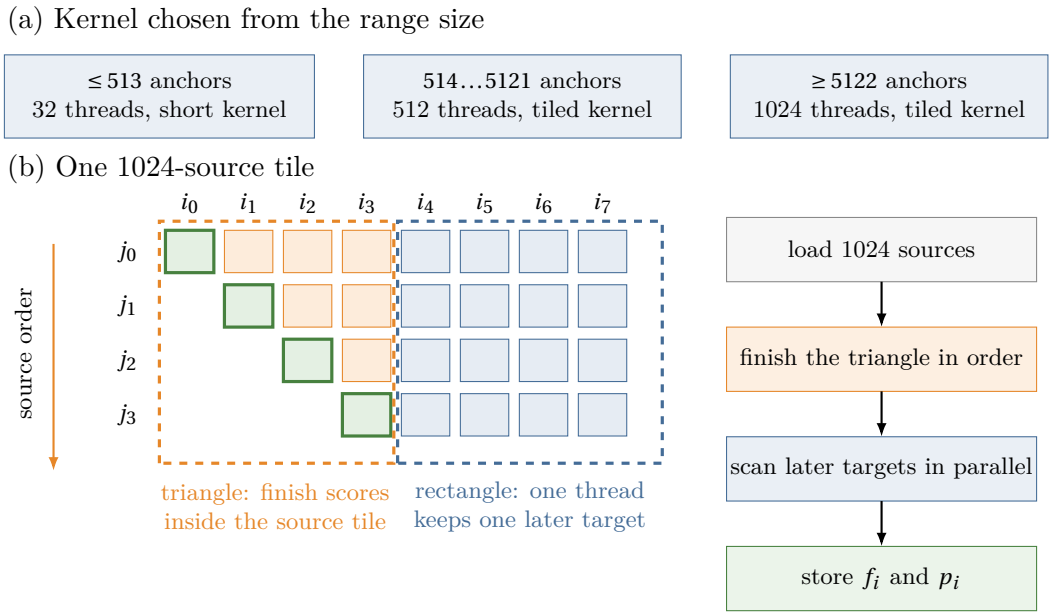


Figure 5.16: G3SA changes the number of threads with the range size. A tiled kernel loads up to 1024 sources. Green diagonal boxes are completed source scores. Orange cells are links among anchors in the tile and must be finished in source order. During each rectangular iteration, one thread keeps one later target in registers and scans the blue cells. Threads repeat this step for the remaining target columns.

In the triangle, one source row is completed before the next row starts. In the rectangle, each thread holds one later target at a time and scans the completed source tile. It performs at most one score and predecessor write after that tile has been scanned. A later source tile can update the same target again.

G3SA direct tiled chaining
mm_compute_chain_long_tiled 333

After this pass, G3SA considers ends with $f_i \geq 40$. It follows predecessor paths from higher score to lower score. For each read, it retains paths with at least three anchors and score at least 40, using maximum drop 0, then packs their anchors into a contiguous prefix of that read's part of the anchor array.

5.9. Choosing one predecessor before computing its full score

The complete candidate score is $f_j + E(j,i)$. Calculating $E(j,i)$ needs both coordinates, the span, and the gap penalty. Minimap2 first calculates the following preliminary value for every earlier anchor:

$$h(j) = f_j + \frac{\lambda_g}{2}(r_j + q_j). \tag{5.17}$$

A larger $h(j)$ is chosen. Once f_j is finished, $h(j)$ can be stored and reused for later targets. This comparison chooses one candidate, but does not give the final chain score. The selected link still needs $f_j + E(j,i)$.

The name range minimum query, abbreviated RMQ, means the following operation: among stored items whose keys lie in a given interval, return the item with the smallest stored priority. Minimap2 stores $\pi(j) = -h(j)$, so its minimum priority is the largest $h(j)$. Figure 5.17 contrasts this tree query with G3SA's second pass. G3SA's function names con-

tain `_rmq`, but the running code performs no range query. It compares $h(j)$ for every source and target pair that its GPU kernels test.

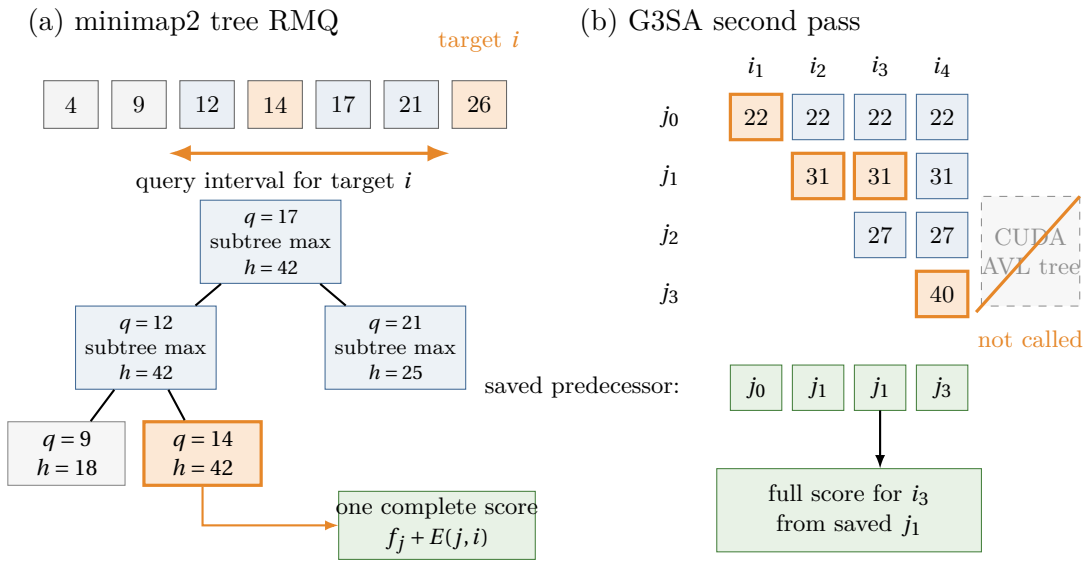


Figure 5.17: Minimap2 saves the largest h for every tree subtree. Its range query returns one earlier anchor, shown in orange. G3SA compares $h(j)$ in every blue cell that it tests and saves one predecessor per target. In panel (b), each number is the $h(j)$ value of its source row. The crossed AVL tree in G3SA's `krmq_cuda.cu` is not called.

5.9.1. Minimap2 RMQ

Minimap2 stores eligible earlier anchors in a height-balanced binary search tree, called an AVL tree. The search key is

$$K_j = (q_j, j).$$

Including j keeps anchors with equal query coordinates as separate nodes. The tree stores

$$\pi(j) = -h(j).$$

Each node also points to the smallest π in its subtree. That pointer already identifies the subtree's best candidate, so a range query need not inspect its anchors one by one.

Minimap2 first sets

$$D = \max\{\texttt{max_dist}, \texttt{bw}\}.$$

Anchors at one reference coordinate are inserted only after the algorithm moves to a larger reference coordinate. At query time, every stored anchor is on the same reference and strand as i, has $r_j < r_i$, and satisfies $r_i - r_j \le D$. Old anchors are removed when their reference distance exceeds D. If the tree is larger than `cap_rmq_size`, the oldest anchors are also removed until the cap is met.

The tree searches the closed interval between these composite keys:

$$K_i^{\text{lo}} = (q_i - D, \text{INT32_MAX}), \qquad K_i^{\text{hi}} = (q_i, 0). \tag{5.18}$$

The tree returns one

$$j^\star \in \arg\max\{h(j) : j \text{ is stored and } K_i^{\text{lo}} \le (q_j, j) \le K_i^{\text{hi}}\}.$$

For $0 < j <$ INT32_MAX, this reduces to

$$q_i - D < q_j < q_i.$$

Equal $h(j)$ values have no explicit index tie rule.

The target starts with $f_i = \ell_i$ and $p_i = -1$. Minimap2 calculates

$$c = f_{j^\star} + E(j^\star, i)$$

for the returned anchor. It updates the target only when

$$\delta(j^\star, i) \le \texttt{bw} \quad \wedge \quad c > f_i.$$

When both conditions hold, it sets $f_i \leftarrow c$ and $p_i \leftarrow j^\star$.

Minimap2 treats this link as an exact extension when

$$\delta(j^\star, i) = 0 \quad \wedge \quad g(j^\star, i) \le \ell_{j^\star}.$$

Equation 5.5 gives such a link no penalty. A local repair scan has four requirements:

1. the main tree returned $j^\star$,
2. that link is not an exact extension,
3. the local tree is non-empty, which also requires a positive configured local distance, and
4. $q_i > 0$.

Let D_{in} be `max_dist_inner` clamped to the interval $[0, D]$. The local tree keeps earlier anchors on the same reference and strand whose reference distance is at most D_{in}, subject to the tree-size cap. The scan visits the retained anchors with

$$q_i - D_{\text{in}} \le q_j \le q_i - 1$$

from larger to smaller query coordinate. It calculates the complete score for each visited anchor. Only candidates with $\delta(j, i) \le \texttt{bw}$ take part in the score update and the B1 mark and skip rule. The main tree is never asked for its second choice. If $j^\star$ is exact, no local repair scan runs.

One tree range query costs $O(\log M)$, where M is the number of stored anchors. Starting the local iterator also costs $O(\log M)$, and visiting k_i local anchors adds $O(k_i)$. Across n targets, tree queries, insertions, and deletions cost $O(n \log M_{\max})$, where $M_{\max}$ is the largest active-tree size over all targets. The local scans add

$$O\left(\sum_i k_i\right).$$

The parameter `max_chain_iter` is not used by RMQ. `max_chain_skip` can stop only the local repair scan.

Minimap2 RMQ chaining
mg_lchain_rmq 258

Augmented AVL range query
__KRMQ_RMQ 110

5.9.2. G3SA second pass: comparing $h(j)$ on the GPU

G3SA runs this pass only for reads selected after the first pass and its backtracking step. It sorts the retained anchors again and splits them into per-read ranges. The short kernel assigns later targets to 32 threads. The medium and long kernels use the 1024-source tiles shown in Figure 5.16.

For every tested pair (j, i) that passes the coordinate checks, G3SA recalculates $h(j)$ from Equation 5.17. Target i keeps the largest value seen so far. It saves the corresponding predecessor as $j_i^\star$, represented by the distance $p_i = i - j_i^\star$. An equal value replaces the predecessor saved earlier. After this selection, one thread calculates

$$c_i = f_{j_i^\star} + E(j_i^\star, i).$$

If $c_i > 15$, it sets $f_i = c_i$. Otherwise it restores $f_i = 15, p_i = -1$. The complete calculation uses the span stored in predecessor anchor $j_i^\star$, not a fixed span of 15.

G3SA still compares every source and target pair tested by its kernels. It only avoids calculating $E(j, i)$ for most pairs. Minimap2 instead uses its main tree to choose one candidate in $O(\log M)$. When that link is not exact, minimap2 may also calculate complete scores for candidates from its local repair tree.

Each new range starts with score 15. The wrapper passes 20,000 as both coordinate-distance arguments, with $\lambda_g = 0.12$ and $\lambda_s = 0$. The short kernel compares a source with at most the next 5,000 anchor indices and checks both coordinate-distance limits. The tiled kernels use the wider scan described above and apply $p_i \le 5000$ only when writing the selected predecessor. Their pair comparison checks increasing query coordinate, query distance at most 20,000, the same reference and strand group, and different reference coordinates. It does not check the 20,000 reference-distance bound in every tiled cell. None of the called kernels rejects a pair by comparing its diagonal width with the supplied bandwidth. They do not perform minimap2's local repair scan and do not use `max_chain_skip` or a tree-size cap.

G3SA backtracks this second result with minimum score 40, at least three anchors, and maximum drop 20,000. These differences mean that the second pass is not an exact GPU copy of minimap2 RMQ.


G3SA second-pass comparison
mm_compute_chain_short_rmq 970


G3SA second-pass launches
chaining_kernel_wrapper_rmq 595

The file `krmq_cuda.cu` contains CUDA versions of minimap2's AVL operations. The entire `gpu_chain_rmq` definition is commented out. The header still declares it, but nothing calls it. As written, one CUDA thread would process one read and perform the tree operations in sequence. The local repair code is disabled. This function does not backtrack or pack anchors. It does not run in current G3SA.


Dormant CUDA AVL prototype
gpu_chain_rmq 406

5.10. BioseqDB implementation

BioseqDB exposes four chaining choices through C++ and PostgreSQL. The PostgreSQL setting is

`bioseq.aligner.chaining.algorithm`.

The `chaining_algorithm` field inside per-call `aligner_options` uses the same values $0, \ldots, 3$. Choice 0 is the default in both interfaces. The chosen value determines how f_i, p_i are calculated. In the generic C++ chain-stage implementation, backtracking runs on the CPU

except for Vulkan queries that request coordinates only. PostgreSQL choice 3 uses the complete Vulkan pipeline described below.

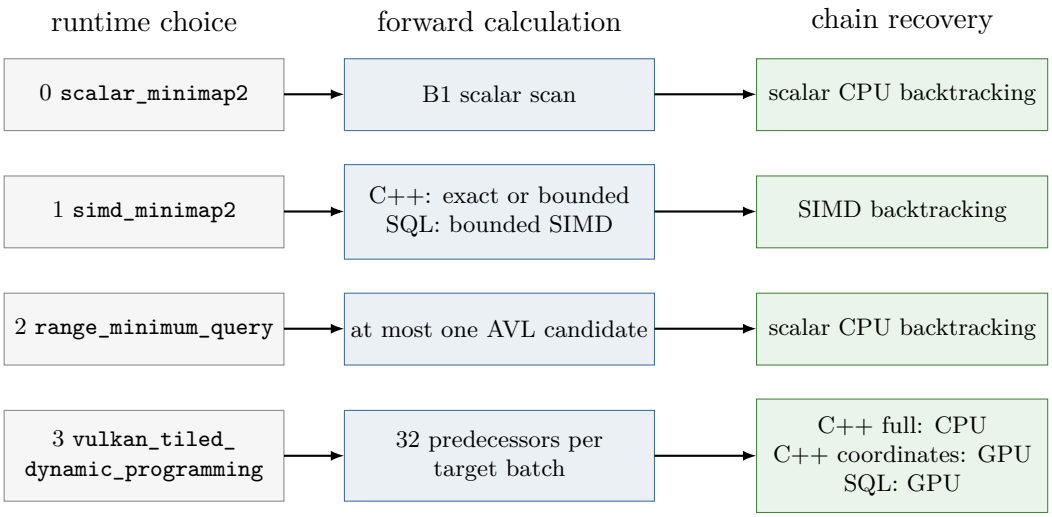


Figure 5.18: The four BioseqDB chaining choices. Choice 1 has exact and bounded internal modes. PostgreSQL always uses its bounded mode. Choice 3 shows the generic C++ recovery route. PostgreSQL performs its chain recovery on the GPU.

`scalar_minimap2`, value 0. This is the PostgreSQL and high-level C++ default. It runs the scalar B1 predecessor scan and BioseqDB scalar backtracking.

`simd_minimap2`, value 1. The C++ API lets a caller choose one of two internal modes:

- `exact_visited` reproduces BioseqDB scalar B1 predecessor choices.
- `bounded_batch_approximate` is bounded SIMD B1. For SIMD width L, it may stop after $\lfloor \texttt{max_chain_skip}/L \rfloor + 1$ consecutive batches in which no lane improves its saved lane score.

PostgreSQL exposes choice 1 but not these two mode names. It always uses the bounded mode. Both modes use the SIMD backtracker described above. Bounded mode requires positive `max_chain_skip` and `max_chain_iter`. Otherwise choice 1 returns an error.

`range_minimum_query`, value 2. This selects BioseqDB's CPU AVL implementation. The current route uses only the primary tree. For each target, that tree returns at most one candidate. If one exists, BioseqDB calculates $f_j + E(j,i)$. Scalar CPU backtracking follows.

`vulkan_tiled_dynamic_programming`, value 3. This selects the Vulkan implementation. Its work split and chain recovery are described below.

A selected SIMD or Vulkan algorithm does not silently change to scalar chaining when it cannot run. It returns an error. Dispatch ignores the internal field `chaining_mode`. The four-value choice shown in Figure 5.18 does. None of these choices runs G3SA's two-pass source sweep.

5.10.1. How BioseqDB RMQ differs from minimap2

BioseqDB stores anchors in an AVL tree keyed by query coordinate and records the best $h(j)$ in each subtree. The current choice-2 route leaves the optional local-repair distance at zero, so no repair scan runs. Neither `max_chain_skip` nor `max_chain_iter` changes this forward calculation.

For configured main distance D, the BioseqDB query uses inclusive keys

$$K^{\mathrm{lo}}_{i,\mathrm{Bio}} = (q_i - D, \mathrm{INT64_MIN}), \qquad K^{\mathrm{hi}}_{i,\mathrm{Bio}} = (q_i, \mathrm{INT64_MAX}).$$

These keys permit anchors on both query-coordinate boundaries. BioseqDB calculates the selected link's diagonal width but does not compare it with the configured bandwidth. It runs RMQ once and omits minimap2's optional wider second chaining pass.

Choice 2 therefore does not reproduce minimap2 RMQ.

5.10.2. BioseqDB Vulkan work split

Like minimap2, the Vulkan algorithm fixes one target i and tests earlier predecessors j. It does not fix one source and update many targets as G3SA does. One contiguous anchor range assigned to the shader is handled by one 32-lane workgroup. After a workgroup finishes one range, it may advance by the total number of workgroups and process another range. Within one range, it finishes targets one after another. Its 32 lanes calculate up to 32 complete candidate scores. A reduction across the lanes selects the batch winner, then updates scalar f_i, p_i. When `max_chain_skip` is positive, it stops after

$$\left\lfloor \frac{\texttt{max_chain_skip}}{32} \right\rfloor + 1$$

consecutive batches do not improve the target. A zero value disables this early stop. It also limits the predecessor scan to at most 3,000 anchors, even when `max_chain_iter` is larger. These are not the scalar B1 stopping rules.

For a range with $n > 131{,}072$ anchors, the implementation sets

$$s_{\mathrm{thin}} = \left\lceil \frac{n}{65{,}536} \right\rceil,$$

keeps anchors $0, s_{\mathrm{thin}}, 2s_{\mathrm{thin}}, \ldots$, and always keeps the last anchor. Chaining uses only this retained set. In the generic C++ entry point, full-result queries calculate f, p on Vulkan and backtrack on the CPU. Coordinate-only queries examine at most 16 unvisited chain ends in descending score-and-index order and return at most one accepted path.

PostgreSQL allows choice 3 only as part of the complete Vulkan pipeline. That pipeline performs chaining and trace selection on the GPU. It can also keep a bounded CIGAR calculation on the GPU when Vulkan extension is enabled. It does not enter the generic CPU backtracker. If its requirements are not met, PostgreSQL returns an error rather than retrying with a CPU or SIMD algorithm.

Chapter 6

Final alignment and extension

6.1. From a seed chain to a base-level alignment

Chaining identifies an ordered set of anchors with plausible query and target coordinates. It does not determine the substitutions, insertions, and deletions between those anchors. Final alignment resolves this remaining ambiguity. It refines the outer coordinates, assigns a dynamic-programming score, and reconstructs an edit path when the query requests a CIGAR or an exact edit count.

BioSeqDB treats final alignment as an optional pipeline stage. Coordinate-only queries can return regions derived from seeds and chains. A request for a base-level path enables traceback, while an exact edit predicate also requires the traced path because a seed-derived divergence estimate is insufficient. Avoiding dynamic programming when these outputs are absent is important for database workloads that project many candidate coordinates.

The stage has three execution routes. KSW2 supplies two-piece affine scoring, a static band, Z-drop, and minimap-style endpoint rules. Block Aligner uses a single affine model and an adaptive SIMD block. A restricted GPU route aligns windows produced by GPU seeding and chaining. These routes do not have identical scoring or endpoint semantics. Route selection is explicit, and each route has a distinct scoring and result contract.

The alignment modes in Section ?? determine which matrix borders are fixed and which endpoints are free. BioSeqDB applies these conditions at the level of subproblems. Inter-anchor rectangles are global fills with fixed endpoints. The two outer rectangles are extension problems whose endpoint may be selected before the end of the available flank. This distinction is more accurate than describing an entire chained region as one global or local matrix.

6.2. Affine-gap dynamic programming

Let $Q = q_1 \dots q_n$ be a query and $T = t_1 \dots t_m$ a target interval. For substitution score $s(q_i, t_j)$, an affine gap of length $k \geq 1$ has penalty

$$g(k) = q + ke,$$

where $q > 0$ is the gap-open penalty and $e > 0$ is the extension penalty. The first gap symbol therefore costs $q + e$. With query positions on the vertical axis and target positions on the

horizontal axis, the global Needleman, Wunsch, and Gotoh recurrence is

$$I(i,j) = \max\{H(i-1,j) - q - e,\ I(i-1,j) - e\}, \tag{6.1}$$
$$D(i,j) = \max\{H(i,j-1) - q - e,\ D(i,j-1) - e\}, \tag{6.2}$$
$$H(i,j) = \max\{H(i-1,j-1) + s(q_i,t_j),\ I(i,j),\ D(i,j)\}. \tag{6.3}$$

The insertion and deletion states record whether a path already ends in a gap. A global fill initializes both axes with cumulative gap costs. An extension changes the boundary and endpoint rules while retaining the same cell recurrence. A full matrix takes $O(nm)$ time and $O(nm)$ trace memory when every decision is retained Gotoh, "An improved algorithm for matching biological sequences."

The default nucleotide model uses match score $a = 2$, mismatch penalty $b = 4$, and two affine families

$$(q_1, e_1) = (4, 2), \qquad (q_2, e_2) = (24, 1).$$

KSW2 assigns a gap the lower of the two costs,

$$g(k) = \min\{4 + 2k,\ 24 + k\}.$$

The short family is cheaper below length 20, both are equal at 20, and the long family is cheaper above 20. The recurrence keeps two insertion states and two deletion states:

$$I_r(i,j) = \max\{H(i-1,j) - q_r - e_r,\ I_r(i-1,j) - e_r\}, \tag{6.4}$$
$$D_r(i,j) = \max\{H(i,j-1) - q_r - e_r,\ D_r(i,j-1) - e_r\}, \qquad r \in \{1,2\}, \tag{6.5}$$
$$H(i,j) = \max\{H(i-1,j-1) + s(q_i,t_j), I_1, I_2, D_1, D_2\}. \tag{6.6}$$

This model charges short sequencing errors and long gaps differently. Block Aligner instead uses only the first affine family. Selecting Block Aligner is therefore a scoring choice as well as a performance choice.

6.2.1. Banding and stopping criteria

Anchors suggest an expected diagonal d_0. KSW2 can restrict evaluation to cells satisfying

$$|(i-j) - d_0| \le w,$$

where w is the half-width of the band. Work is then approximately $O(w(n+m))$, but an optimum that leaves the band cannot be recovered. The ordinary alignment width is derived from the chaining width as

$$w_{\text{align}} = \lfloor 1.5\, w_{\text{chain}} + 1 \rfloor.$$

Z-drop terminates an extension whose score has deteriorated sufficiently from the best cell. If $S^\star$ is the best score at $(i^\star, j^\star)$, termination is permitted when

$$S^\star - S(i,j) > Z + e_z \left| (i - i^\star) - (j - j^\star) \right|. \tag{6.7}$$

The displacement term allows a long gap to move away from the best diagonal without receiving the same stopping penalty as a score loss along that diagonal. In the two-piece kernel, e_z is the long-gap extension penalty. Block Aligner does not use this rule. Its outer extensions apply X-drop and require two consecutive steps below the threshold before stopping.

For mandatory KSW2 fills used by an exact edit predicate, banding and Z-drop are disabled. This prevents the path between two fixed anchors from ending early. The outer candidate window and the selected chain still constrain the search, so this setting does not turn the complete mapping task into an unbounded full-matrix comparison.

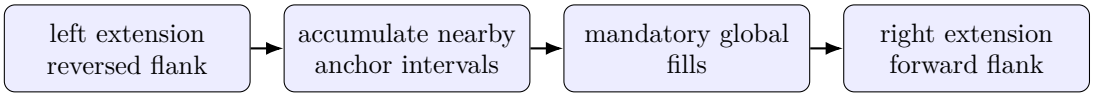
6.3. Alignment around a chain

BioSeqDB follows the ordinary DNA extension structure of minimap2 Li, "Minimap2: pairwise alignment for nucleotide sequences." It solves three kinds of subproblem rather than one matrix over the complete region. The left flank is reversed and aligned as an extension. Rectangles between selected anchors are global fills. The right flank is a forward extension.

Table 6.1: Boundary conditions of the three alignment phases.

Phase	Orientation	KSW2	Block Aligner
Left flank	reversed	extension endpoint, band, and Z-drop	native X-drop extension
Between anchors	forward	global two-piece affine fill	global single-affine adaptive block
Right flank	forward	extension endpoint, band, and Z-drop	native X-drop extension

Short distances between consecutive anchors are accumulated. A fill is started only after both the query and target spans reach a minimum length, or when the last anchor forces the remaining rectangle. This reduces a long region to a sequence of moderate subproblems. It also prevents the overhead of running dynamic programming separately for every closely spaced seed.



ordered anchors preserve the query and target coordinate direction

Figure 6.1: Decomposition of a chained region into two extensions and a set of global inter-anchor fills. Several short anchor intervals may form one fill.

Algorithm 16 Final alignment of one chained region

Input. Query Q, target T, ordered anchors A, parameters P, and a selected backend.

1. Derive inner anchor bounds and bounded outer extension intervals.
2. If both left intervals are nonempty, align them in reverse orientation and update the left endpoint.
3. Place a cursor at the first anchor.
4. Visit each later anchor in order.
   (a) Accumulate query and target distance while either span is below *P.minLen*, except at the final anchor.
   (b) Solve each resulting positive rectangle as a global fill.
   (c) Append its CIGAR and move the cursor to the current anchor.
5. If both right intervals are nonempty, run the permitted forward extension and update the right endpoint.
6. Merge adjacent equal CIGAR operations and project reverse-strand coordinates when required.

Each phase returns a score, consumed lengths, endpoint information, and an optional trace. A non-dropped global fill consumes both slices. Ordinary KSW2 fills may instead stop under Z-drop and use their maximum-scoring positions as truncated endpoints. Exact-edit KSW2 disables Z-drop. An extension may consume only a prefix of the available oriented slices, which moves the reported outer coordinate to its selected endpoint. The region result is formed by adding phase scores and concatenating phase traces in coordinate order. Reversing the left input changes the direction of computation, but not the final order of the reported path.

Stopping state is also phase-specific. A KSW2 Z-drop in an earlier phase can suppress a later right extension because the minimap-style control flow treats the region as dropped. Block global fills do not use Z-drop. Its left and right X-drop extensions are separate searches, so the right extension remains eligible after an earlier Block phase. This is a material behavioural difference and is retained as a backend rule rather than normalised through a shared option.

The port differs from minimap2 in its supported policy surface. It implements the ordinary DNA seed-gap and end-extension structure, but not the splice, RNA, inversion-retry, or long-join policies. It also obtains slices from lazy sequence representations, including reverse-oriented slices, without first copying a complete region. These differences concern the surrounding pipeline. The KSW2 recurrence remains recognisable, while Block Aligner uses a separate phase contract instead of imitating KSW2 options that it does not support.

6.4. KSW2 with portable SIMD

6.4.1. Difference representation

A direct SIMD implementation stores absolute H, I, and D scores. Absolute values grow with sequence length and may require wide lanes. KSW2 uses the Suzuki and Kasahara observation that differences between neighbouring scores remain bounded by the scoring parameters even when the absolute score is large Suzuki and Kasahara, "Introducing difference recurrence relations for faster semi-global alignment of long sequences." Representative differences are

$$U(i,j) = H(i,j) - H(i-1,j), \qquad V(i,j) = H(i,j) - H(i,j-1).$$

The main recurrence operates on signed byte values derived from these differences. Wider scores are reconstructed where exact maxima and endpoints are needed. More cells therefore fit in each SIMD vector.

BioSeqDB expresses this recurrence with portable SIMD operations. Lanes hold cells from one alignment problem, usually along an anti-diagonal. This is intra-sequence parallelism. It differs from placing several independent sequence pairs in the lanes of one vector. A single-affine specialization is used when the two penalty pairs are equal. The default parameters select the two-piece specialization, with four gap states and their continuation bits. Trace decisions are stored for cells inside the admitted band.

6.4.2. Endpoint and traceback rules

KSW2 tracks the best internal cell, the best score at each sequence end, and the bottom-right score when that cell is reached. A global fill traces from the bottom-right cell. An extension normally traces from the best admitted endpoint, with the query-end score and end bonus considered by the selected mode. A left extension is computed on reversed slices, after which its CIGAR runs are restored to source order.

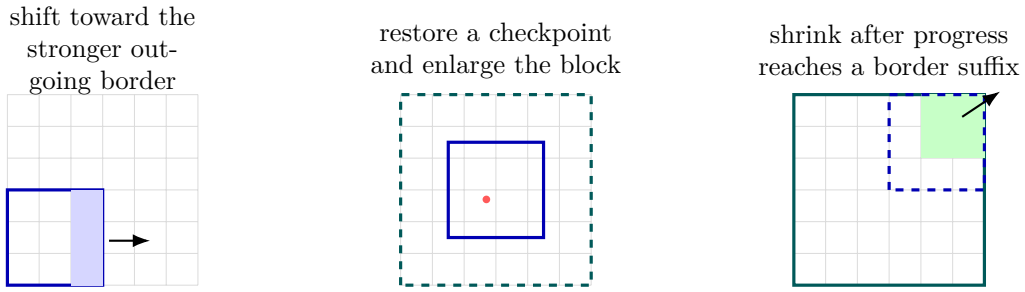
The BioSeqDB port changes data access and allocation rather than the scoring model. It encodes only each requested subrange from a lazy view, reuses operation-local scratch, and dispatches the same recurrence to supported CPU targets. The second affine family, band, Z-drop, endpoint bonus, and tie rules remain reasons to select KSW2. They are not projected onto Block Aligner.

6.5. Adaptive block alignment

6.5.1. Block movement and growth

Block Aligner evaluates the single-affine recurrence in a moving $B \times B$ square rather than a fixed diagonal band Liu and Steinegger, "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices." Each step computes a strip of width S on the right or lower edge. The maximum over the two outgoing border prefixes chooses whether the block moves right or down. The block can therefore follow a path that moves far from the original diagonal.

Greedy movement can miss a path after a long gap or a low-scoring interval. When several steps fail to find a new maximum, the algorithm restores a checkpoint and increases B. The larger region admits more alternatives. After progress resumes near an outgoing border suffix, the block may shrink and move diagonally. The minimum and maximum block sizes bound this adaptation.



(a) move (b) grow (c) shrink

Figure 6.2: Adaptive block movement. Border scores guide each shift. Lack of progress causes replay with a larger block, while later progress permits a smaller block.

6.5.2. SIMD recurrence within a block

Vectors are laid out vertically or horizontally inside the block. One gap state then has a dependency between lanes. For vector values $x_0, \ldots, x_{L-1}$, the required affine prefix operation is

$$y_i = \max_{0 \le k \le i} \{x_k + (i-k)g\}, \qquad g = -e. \tag{6.8}$$

Shift-and-maximum stages at distances $1, 2, 4$, and then higher powers of two compute this scan in $O(\log L)$ vector operations. A carry connects the end of one vector with the next. The diagonal transition and the other gap state use ordinary vector dependencies.

Score computation retains only the block borders and checkpoint copies, so its working score memory is $O(B)$. Traceback needs additional strip records along the adaptive trajectory. A checkpoint includes the trace position, which allows speculative strips to be discarded when the algorithm grows and replays a region.

BioSeqDB keeps the reference step geometry and admits SIMD lane shapes that preserve it. Global mode fills the fixed rectangles between anchors. X-drop mode handles the outer extensions. Input ranks are obtained from the same lazy views as KSW2. SIMD masks encode direction and gap continuation decisions, and traceback decodes these masks directly into packed runs. The main material difference from KSW2 remains the adaptive search with one affine family.

Algorithm 17 Adaptive block alignment

Input. Encoded slices, affine scores, block bounds, and either global or X-drop mode.

1. Place a minimum-size block at the initial matrix corner and save a checkpoint.
2. Compute the next outgoing strip with the vector prefix recurrence.
3. Record score and trace borders, then update the best score.
4. If progress continues, move toward the stronger outgoing border.
5. If progress stalls and growth is possible, restore the checkpoint, enlarge the block, and recompute that region.
6. If the best score reaches a suitable border suffix, shrink the block and advance diagonally.
7. Repeat until the global endpoint or the X-drop condition is reached.
8. Trace backward through the retained strips and emit packed runs.

6.5.3. Exact diagonal certificate

An equal-length fill can sometimes be resolved without constructing a block. Let its common length be n, let D be the exact score of the coordinate diagonal, and let M be the largest substitution score. Any different global path must contain the same positive number of inserted and deleted symbols. Its score is at most

$$U = (n-1)M - 2(q+e).$$

It follows that

$$D > U \quad \Longrightarrow \quad \text{the coordinate diagonal is the unique optimum.} \tag{6.9}$$

Strict inequality excludes a competing path with an equal score. When the condition holds, BioSeqDB can return the diagonal score, a single match run, and the exact mismatch count. Otherwise it runs the adaptive dynamic program. The certificate proves only this specific geometry and does not make Block Aligner exact for every admitted input.

6.6. Traceback, CIGAR, and edit counting

All alignment routes cross their low-level boundary using packed CIGAR runs of the form

$$\mathit{word} = (\mathit{length} \ll 4) \mid \mathit{opcode}.$$

The common opcodes include M for a diagonal step, I for an insertion, D for a deletion, and the more explicit equality and mismatch variants = and X. A run records one opcode and its length. Adjacent equal runs are merged within a traceback and across phase boundaries.

The forward calculation and backward trace have separate roles. The forward pass selects scores and endpoints. The trace reconstructs the path through diagonal and gap states. KSW2 records decisions for its admitted band. Block Aligner records decisions for computed strips. The GPU route either retains a bounded direction matrix or reconstructs directions tile by tile. All three then produce the same run-oriented interface.

A generic M run does not reveal whether its diagonal symbols are equal. The edit count is therefore computed from the emitted path and the corresponding sequence symbols:

$$E_{\text{path}} = \#\{\text{unequal diagonal steps}\} + \sum_{I \text{ runs}} \mathit{length} + \sum_{D \text{ runs}} \mathit{length}.$$

Block Aligner counts unequal diagonal steps during traceback, while KSW2 can replay diagonal runs against the sequence ranks. These CPU routes return the unit-cost edit count of the chosen path. It is distinct from the affine score and from a seed-derived estimate.

The CIGAR also defines coordinate consumption. A diagonal run consumes query and target symbols, an insertion consumes query symbols, and a deletion consumes target symbols. Replaying these rules checks the reported span and allows CPU routes to derive aligned lengths and edit count. The complete GPU route currently returns CIGAR and match counts, but it does not expose the chosen-path edit-count field or accept an exact edit predicate. Reverse-strand coordinates are converted only after the phase CIGARs have been assembled in alignment order.

6.7. Accelerator boundary and gridded traceback

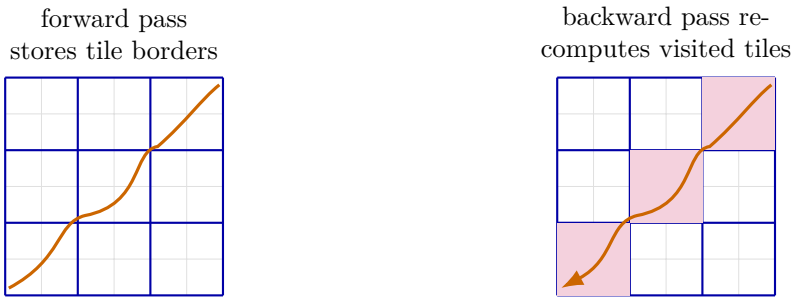
The GPU route begins after candidate windows have been chosen by GPU seeding and chaining. If a query needs coordinates only, those windows can be returned without base alignment. If it needs CIGAR, each selected query and target window is packed as symbol ranks and submitted as one job in a batch. The result contains score, endpoints, status, and a slice of packed CIGAR runs.

This route has an explicit ownership boundary. Its configuration must select the complete GPU frontend and an admitted base-alignment identity. The latter does not mean that the CPU Block Aligner recurrence runs on the device. It is a route selection condition. The general CIGAR kernel uses its own two-piece affine global recurrence over the selected coordinate window. Unsupported geometry or an unavailable GPU component rejects the route instead of continuing with a CPU alignment under the same label.

Conventional GPU traceback writes one direction value for every computed cell. G3SA reduces this traffic by retaining score states at tile boundaries and recomputing directions only for tiles reached by the final path Han et al., "G^{3}SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment." BioSeqDB applies the same memory principle in Vulkan. The forward pass stores five affine score states along tile borders. Starting from the global endpoint, traceback reloads a border, recomputes one tile in fast memory, traces to the next boundary, and discards the temporary directions. For tile dimensions b_q and b_t, border storage is proportional to

$$\left\lceil \frac{n}{b_q} \right\rceil \left\lceil \frac{m}{b_t} \right\rceil (b_q + b_t),$$

apart from the constant number of affine states. One reusable tile needs $O(b_q b_t)$ direction storage. The scheme replaces a full $O(nm)$ direction matrix with tile borders and additional arithmetic during traceback. It is most useful when writing the full trace matrix would dominate device memory traffic.



(a) boundary state (b) local replay

Figure 6.3: Gridded traceback. Blue tile borders persist after the forward pass. Only the purple tiles crossed by the selected path are recomputed during traceback.

Short or structurally simple windows first use specialized scripts, including greedy gap scripts that are not guaranteed to maximize the affine score. Moderate windows can retain a bounded direction matrix. Longer admitted windows use gridded traceback. The current GPU CIGAR recurrence is global over the chosen window. It does not apply the KSW2 band, Z-drop, or end bonus even if related values are present in the broader request. It should therefore be described as a GPU-specific alignment contract, not as a device implementation of KSW2 or Block Aligner.

6.8. Runtime selection and design limits

The alignment route is selected explicitly before region alignment. The selected CPU kernel is then specialized for KSW2 or Block Aligner, so the inner recurrence does not branch on the backend for each cell. Output needs are resolved separately. A coordinate-only query may omit base alignment, while CIGAR or exact edit output makes traceback mandatory.

The GPU route additionally requires the complete accelerated frontend and an admitted feature tuple before alignment begins.

Several limits follow from these contracts. First, KSW2 and Block Aligner can return different scores because their gap models and search regions differ. Second, the ordinary DNA extension port does not implement every specialised minimap2 mode. Third, the GPU route uses a global coordinate window and lacks the current CPU endpoint rules. Fourth, adaptive search and finite bands can both omit cells that an unrestricted matrix would examine.

These differences are exposed through route selection rather than hidden behind a single claim of algorithmic equivalence. KSW2 is appropriate when two-piece affine scoring and minimap-style stopping rules are required. Block Aligner is appropriate when its single-affine adaptive search matches the requested semantics. The GPU path is appropriate only when the complete accelerated route and its global window contract are admitted. When available, CIGAR and edit count describe the path produced by the selected route.

Chapter 7

Alignment prefiltering

7.1. Why filter after chaining but before alignment?

Seed matches and chains identify sequence intervals that may align well. They do not establish that the complete intervals satisfy an edit-distance bound. Repeated sequence, short seeds, hash collisions, and sequencing errors can all produce plausible chains whose intervening gaps are expensive to align and ultimately fail the requested bound. Applying affine-gap dynamic programming to every candidate is correct, but it spends time on candidates that a cheaper procedure may be able to disprove.

A pre-alignment filter addresses this case. It runs after chaining has defined candidate regions and before the base aligner processes chain-defined fills. Its result is deliberately limited. The filter may show that a candidate cannot meet the requested threshold, or it may leave the question unresolved. It does not produce an alignment, a CIGAR string, or an edit script.

This distinction is required by the SQL interface. Consider a predicate of the form

```
WHERE mmsearch(sequence, query)
  AND bioseq.edit(sequence) < 10
```

The strict bound $B = 10$ means that the unit-cost edit count reconstructed from every returned alignment traceback must be below ten. The prefilter receives the equivalent inclusive threshold

$$E = B - 1 = 9.$$

For mandatory fill i, let D_i be the minimum global Levenshtein distance between its query and target strings. Let C_i be the edit count on the traceback selected by the base aligner. A safe filter computes L_i such that

$$0 \le L_i \le D_i \le C_i.$$

The second inequality holds because no particular edit script can use fewer edits than the minimum over all scripts. For disjoint mandatory fills,

$$\sum_i L_i \le \sum_i C_i \le C_{\text{region}},$$

where C_{region} is the edit count exposed by `bioseq.edit`. A region may therefore be rejected when $\sum_i L_i > E$. A retained candidate still proceeds to the selected aligner, which constructs

a traceback before the original strict predicate is applied. The approximation can remove work, but it cannot replace exact evaluation.

The terms false positive and false negative are easy to misread in this setting. Here, the positive filter outcome means that a candidate is retained for alignment. The two errors have different effects.

Outcome	Meaning	Effect
False positive	A pair above the threshold is retained	Additional alignment work, with no change to the query result
False negative	A pair within the threshold is rejected	A valid result is lost, which violates the SQL predicate

Table 7.1: Consequences of prefilter errors. Safe filtering permits false positives but not false negatives.

BioSeqDB applies this contract to the geometry created by chaining. For a multi-seed chain, mandatory fill windows extend from the first anchor start to successive anchor ends. These windows include the ranks covered by anchors. An exact or hash-supported seed is not assigned zero edit cost merely because it formed an anchor. Its symbols are compared within the mandatory fill, so a hash collision can loosen candidate generation but cannot remove edits from the filter bound. Single-seed spans are handled separately by exact edit accounting.

The terminal windows have different semantics. Local extension and X-drop may omit an expensive prefix or suffix. A large global distance in such a window does not prove that the permitted local alignment fails. The prefilter therefore examines mandatory fill windows and leaves optional end extensions to the aligner. Reverse-strand fills are first expressed in the orientation in which they will be aligned.

Unequal lengths also require a restricted rule. Every transformation from Q to T needs at least

$$\Delta_{\text{len}} = ||Q| - |T||$$

insertions or deletions. BioSeqDB uses this exact lower bound for unequal lengths rather than extending an equal-length approximation beyond its stated domain. Unsupported sizes, thresholds, and execution geometries are retained conservatively. Resource or device failures are ordinary query errors, since silently changing the selected execution route would make performance and diagnostic behavior difficult to interpret.

7.2. SneakySnake chip-maze filtering

SneakySnake treats approximate matching as routing through a chip maze Alser, Shahroodi, et al., "SneakySnake: a fast and accurate universal genome pre-alignment filter for CPUs, GPUs and FPGAs." For equal-length strings of length m and an inclusive threshold E, the maze has $2E+1$ rows. Row d, where $d \in [-E, E]$, represents one displacement between the query and target. At target position j, its cell is

$$Z_d(j) = \begin{cases} 0 & 0 \le j + d < m \text{ and } Q[j+d] = T[j] \\ 1 & \text{otherwise.} \end{cases}$$

A zero is a match on that diagonal and a one is an obstacle. Starting at a checkpoint, the algorithm measures the matching prefix on every available diagonal. It follows the longest

run, crosses the obstacle at its end when the right boundary has not yet been reached, and starts another search from the new checkpoint. The number of crossed obstacles is the filter value. If it exceeds E, the pair cannot satisfy the threshold in the safe equal-length domain.

Figure 7.1 shows three escape segments. The highlighted route stays on the unshifted row. It follows exact runs at positions 0–2, 4–6, and 8–10, crossing obstacles at positions 3, 7, and 11. The third crossing raises the count above $E = 2$, so this example is rejected. The maze abstracts away the edit script. In particular, it records neither the operation responsible for a change of row nor the vertical distance between selected segments.

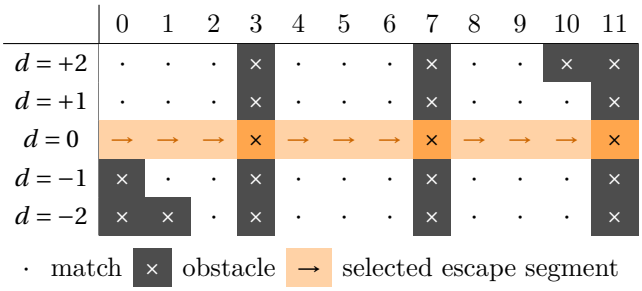


Figure 7.1: A rejecting SneakySnake chip-maze walk for Q = `AAAAAAAAAAAA`, T = `AAACAAACAAAC`, and $E = 2$. Each row represents a query-to-target displacement. Amber cells show the successive escape segments and three crossed obstacles on the unshifted row.

The corresponding procedure is compact.

Algorithm 18 Unpartitioned SneakySnake core for one equal-length submaze

$c \leftarrow 0$
$r \leftarrow 0$
while $c < m$ do
$b \leftarrow \max_{d \in [-E,E]} \text{matchingPrefix}(Q, T, c, d)$
if $c + b \geq m$ then return retain
$r \leftarrow r + 1$
if $r > E$ then return reject
$c \leftarrow c + b + 1$
end while
return retain

The lower-bound direction follows from the relation between a maze route and an edit script. Between two edits, an alignment follows one diagonal through a run of equal symbols. A substitution ends a run on the same diagonal. An insertion or deletion moves the next run to another diagonal. An edit script of cost k therefore induces matching segments separated by at most k obstacles. The chip maze also permits transitions whose cost an edit script would represent more precisely, so it is a relaxation of edit distance.

SneakySnake chooses the furthest reachable checkpoint under this relaxed model. The greedy escape rule minimizes the number of obstacles in the routing model Alser, Shahroodi, et al., "SneakySnake: a fast and accurate universal genome pre-alignment filter for CPUs, GPUs and FPGAs." If $D(Q, T) \leq E$, the edit script induces a maze route with at most E obstacles, and the greedy route cannot use more. Consequently, a greedy count above E proves that the global distance also exceeds E. The converse does not hold because the relaxation can omit costs that an edit script must pay.

The maze is less expressive than edit-distance dynamic programming. This loss of information is intentional. The obstacle count can be smaller than the true distance, which retains some pairs that later fail alignment. It must not be larger in the domain used for rejection. Partitioning a long gap into bounded submazes may weaken the result further, since a new submaze forgets some preceding routing state. That change can add false positives, but it does not create false negatives. Bounds from non-overlapping mandatory pieces can still be accumulated because those pieces require separate edits.

BioSeqDB compares normalized alphabet ranks rather than DNA characters. The same equality operation can therefore handle nucleotide and amino-acid alphabets once both operands have a common representation. A scalar CPU path scans matching runs directly. A vector CPU path compares contiguous groups of ranks and locates the first mismatch. Both execute the same checkpoint rule. On a GPU, separate candidates can be assigned to independent invocations at small thresholds. Larger thresholds expose more diagonal work, so a small cooperating group can evaluate shifts for one candidate. Fixed submaze sizes bound memory use and keep dispatch geometry predictable.

Long gaps are divided into consecutive bounded submazes, and Algorithm 18 is applied to each one. The lower-bound counts are added. Restarting at a submaze boundary discards routing state and can reduce the resulting count. It cannot increase the bound above the value for the complete maze, so partitioning weakens filtering rather than changing accepted query results.

At each checkpoint, an exact pruning rule can reduce prefix scans. Suppose the best run found so far has length b. Another diagonal can improve the maximum only if its first b positions match, including the position at offset b. Testing that position first discards only a diagonal that cannot win. Every possible improvement still receives a complete matching-prefix comparison.

These mappings change how comparisons are scheduled, not the rejection criterion. If a vector lane, work group, or bounded representation cannot express a job, the job is retained for exact alignment. This rule keeps backend selection separate from query correctness.

7.3. GateKeeper shifted-mask filtering

SneakySnake must repeatedly search diagonals at each checkpoint. Very short gaps with small thresholds admit a more regular bit-vector computation. GateKeeper derives such a filter from Shifted Hamming Distance methods Alser, Hassan, et al., "GateKeeper: a new hardware architecture for accelerating pre-alignment in DNA short read mapping"; Bingöl et al., "GateKeeper-GPU: Fast and Accurate Pre-Alignment Filtering in Short Read Mapping." For equal-length strings, the unshifted mismatch mask is

$$H_0[j] = [Q[j] \neq T[j]].$$

For each displacement $d = 1, \ldots, E$, two further masks compare the target with shifted query positions.

$$H_{-d}[j] = [j \geq d \wedge Q[j-d] \neq T[j]]$$
$$H_{+d}[j] = [j + d < m \wedge Q[j+d] \neq T[j]].$$

The complete representation contains $2E + 1$ masks. A zero denotes a possible match. A run of true matches displaced from the main diagonal can therefore appear in one of the shifted masks even when H_0 contains mismatches.

Isolated zeros are often accidental matches rather than evidence of a useful run. GateKeeper amends each mask before the masks are combined. BioSeqDB fills an internal zero run of length one or two only when ones bound it on both sides. Longer runs and runs touching an edge remain unchanged. For an unbounded bit vector M, with positions outside the valid interval treated as zero, this operation can be written as

$$\begin{aligned} S &= \neg M \wedge (M \ll 1) \wedge (M \gg 1) \\ P &= \neg M \wedge (\neg M \gg 1) \wedge (M \ll 1) \wedge (M \gg 2) \\ \mathsf{amend}(M) &= M \vee S \vee P \vee (P \ll 1). \end{aligned}$$

The amended result is restricted to the valid sequence positions. When a mask spans several machine words, neighboring bits must be carried across word boundaries. Otherwise an identical two-zero run would receive different treatment depending on its position in memory.

A shift also creates a vacant position at one end. If this position remained zero, the later intersection could interpret it as a match and hide an end error. After amendment, BioSeqDB sets the boundary-adjacent bits

$$\widehat{H}_{-d}[d-1] \leftarrow 1, \qquad \widehat{H}_{+d}[m-d] \leftarrow 1.$$

The index identifies the position next to the vacant prefix or suffix for displacement d. The ordering is significant because compensation before amendment could turn an edge run into a bounded run and fill it.

All amended and compensated masks are intersected. The result is divided into four-bit groups. A zero group contributes no cost. The patterns

$$0101, \quad 0110, \quad 1001, \quad 1010, \quad 1011, \quad 1101$$

contribute two, while every other nonzero group contributes one. Accumulation can stop at $E+1$, since larger values have the same threshold outcome. This grouped estimate is not a popcount and it is not an exact edit distance.

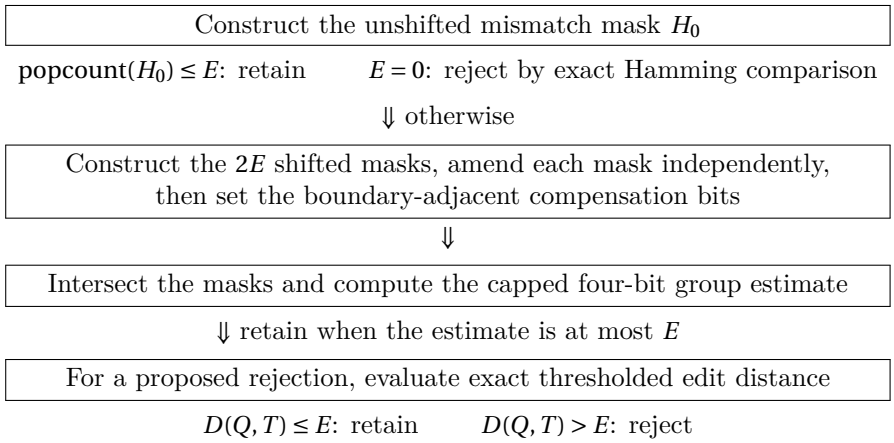


Figure 7.2: GateKeeper decision flow in BioSeqDB. The unshifted mask provides an exact rejection certificate when $E = 0$. For positive thresholds, shifted masks can retain a candidate or propose rejection, and the exact threshold guard confirms the proposal.

7.3.1. The exact rejection guard

Mask amendment changes zeros to ones, and the grouped estimate is an approximation. An observed absence of false negatives on a data set would not prove safety for every sequence. BioSeqDB therefore treats an estimate above E as a rejection proposal. A thresholded exact Levenshtein calculation confirms whether $D(Q,T) > E$. If it does not, the candidate is retained.

The exact guard changes the role of the published filter. GateKeeper was designed to predict whether a pair is likely to satisfy an edit threshold, and its filtering accuracy is evaluated empirically Alser, Hassan, et al., "GateKeeper: a new hardware architecture for accelerating pre-alignment in DNA short read mapping"; Bingöl et al., "GateKeeper-GPU: Fast and Accurate Pre-Alignment Filtering in Short Read Mapping." An empirical false-negative rate is not a sufficient contract for a database predicate, where one rejected valid pair changes the result set. BioSeqDB therefore uses shifted masks to decide whether exact threshold evaluation is needed. The masks can retain a candidate immediately, but for $E > 0$ they cannot reject one without the guard. When $E = 0$, a nonzero unshifted mismatch mask is itself an exact certificate because no insertion or deletion can produce a zero-edit alignment.

Algorithm 19 Guarded GateKeeper decision for one equal-length gap

$H \leftarrow \mathsf{unshiftedMismatchMask}(Q, T)$
if $\mathsf{popcount}(H) \le E$ then return retain
if $E = 0$ then return reject
$A \leftarrow \mathsf{amend}(H)$
for $d \leftarrow 1$ to E do
$H_{-d} \leftarrow \mathsf{shiftedMismatchMask}(Q, T, -d)$
$H_{+d} \leftarrow \mathsf{shiftedMismatchMask}(Q, T, +d)$
$M_- \leftarrow \mathsf{amend}(H_{-d})$
$M_-[d-1] \leftarrow 1$
$M_+ \leftarrow \mathsf{amend}(H_{+d})$
$M_+[m-d] \leftarrow 1$
$A \leftarrow A \wedge M_- \wedge M_+$
end for
if $\mathsf{groupedCost}(A) \le E$ then return retain
if $\mathsf{exactDistanceExceeds}(Q, T, E)$ then return reject
return retain

The exact guard evaluates only the threshold question. The scalar and generic GPU routes use banded dynamic programming. A path that leaves the band already requires more than E insertions or deletions. Values are capped at $E+1$, and only the preceding and current rows are needed. Specialized GPU variants instead use an exact bounded wavefront with longest-common-prefix extension. Both forms answer whether $D(Q,T) > E$, use bounded memory, and omit traceback. They run only when the mask calculation proposes rejection.

This design changes how GateKeeper results may be combined. A retained mask estimate is not a proved lower bound, so it must not be added across fills. A confirmed rejection is sufficient on its own because one mandatory fill already exceeds the region threshold. The mask remains useful as a cheap screening step that limits calls to the exact guard.

The rank representation again removes assumptions about nucleotide encoding. On CPUs, masks fit into one or a small number of machine words for the intended short-gap domain. Boolean operations then evaluate many sequence positions at once. On GPUs, each invocation handles a candidate with regular register operations. Specialized kernels are useful for common short lengths and low thresholds, while a generic kernel covers other supported

mask geometries. Longer gaps or larger thresholds are routed to SneakySnake or retained for the base aligner.

7.4. Dispatch and query-pipeline integration

Algorithm selection depends on both candidate geometry and execution backend. The scalar and vector CPU routes use SneakySnake across their supported range. A GPU batch uses GateKeeper when all jobs are short, equal in length, and have small thresholds. Other supported GPU batches use SneakySnake. Resolving the algorithm once for a batch avoids separate transfers and result merging. A mixed batch that does not satisfy GateKeeper's domain follows the broader SneakySnake route.

The mapping is summarized in Table 7.2. The table describes policy rather than query semantics. Every route must satisfy the same rejection contract.

Execution route	Preferred filter	Reason
Scalar CPU	SneakySnake	Direct prefix scans have low setup cost and cover general supported gaps
Vector CPU	SneakySnake	Contiguous rank comparisons accelerate matching prefix detection
GPU, short regular gaps	Guarded GateKeeper	Bit masks map to regular per-candidate operations
GPU, broader supported gaps	SneakySnake	Parallel diagonal evaluation covers larger thresholds without large mask state
Unsupported geometry	No rejection	Exact alignment preserves query semantics

Table 7.2: High-level mapping of prefilter algorithms to execution routes.

The relevant stage order is

$$\text{seeding} \rightarrow \text{chaining} \rightarrow \text{preliminary classification} \rightarrow \text{mandatory-fill prefiltering} \rightarrow \text{base alignment} \rightarrow \text{exact edit predicate} \rightarrow \text{final result classification}$$

This order has two important consequences. First, rejected regions do not consume base-alignment work. Retained regions are unchanged, including scoring parameters, gap costs, local-extension rules, and traceback. Second, the exact predicate runs before final primary and secondary classification. A region that violates the SQL bound cannot alter the final classification or mapping quality of a region that satisfies it.

Each fill result is associated with its owning region. SneakySnake lower bounds for disjoint mandatory fills are added with saturation at $E{+}1$. A region is removed when that total exceeds E. A GateKeeper mask estimate is not composable and contributes no numeric lower bound. GateKeeper rejects a region only through the exact $E = 0$ Hamming certificate or an exact-guard confirmation. Survivor compaction retains region order so downstream classification sees the same ordering it would have seen without prefiltering.

The prefilter is activated only when the query asks for exact edit accounting and supplies a usable positive strict bound. Bounds that cannot be represented are not converted by unsigned wraparound. The exact predicate still handles zero, negative, and otherwise unsupported cases according to SQL semantics. Backend choice follows the query's execution plan. Merely having a GPU available does not move an otherwise CPU query to a device.

7.5. Limitations

Retention is not evidence of similarity. SneakySnake may undercount edits because it omits vertical movement between escape segments and resets state at bounded submazes. GateKeeper may combine zeros from different shifted masks. Both effects can retain a pair whose true distance exceeds E. Exact alignment and the final predicate remain necessary.

Only mandatory fills are filtered. The method does not reject from a costly local prefix or suffix because the aligner may legally omit it. This restriction reduces the amount of work that can be filtered, but it preserves local-extension semantics.

Unequal lengths receive a weaker bound. The length difference can reject an impossible threshold, but it says little about substitutions within unequal strings. Such candidates often proceed to alignment even when their true distance is large.

Algorithm domains are finite. Bit masks, bounded submazes, and GPU work groups impose length and threshold limits. Inputs outside a proved and representable domain are retained. A new kernel or encoding cannot be used for rejection until it satisfies the same lower-bound or exact-guard contract.

Filtering has a break-even point. A filter adds rank comparisons, batching, and dispatch overhead. If nearly all candidates survive, this cost may exceed the saved alignment work. GPU launch latency is particularly relevant for small batches. Dispatch policy therefore uses candidate geometry, but performance selection never weakens the exact post-alignment check.

The edit model is specific. The filter bounds minimum unit-cost Levenshtein distance, while the SQL predicate counts unit-cost edits on the selected traceback. The latter count can be larger, which is the safe direction established by $D_i \le C_i$. The aligner may select that traceback using affine gap scores or other scoring choices. A bound for another distance model must not be treated as a proof for this predicate.

7.6. Conclusion

BioSeqDB places prefiltering between chaining and base alignment, where it can remove regions whose mandatory fills cannot satisfy an exact SQL edit bound. SneakySnake provides a general lower-bound route through shifted matching diagonals. GateKeeper provides a compact shifted-mask calculation for short regular gaps, with an exact threshold guard before rejection. CPU and GPU implementations schedule these calculations differently while retaining the same contract. Unsupported inputs and inconclusive estimates proceed to alignment, and every survivor is checked by the exact predicate. This asymmetry permits extra work from false positives while preventing result loss from false negatives.

Chapter 8

Sequence representations and lazy evaluation

8.1. One logical sequence, several physical forms

The algorithms described in the preceding chapters are easiest to express over an ordered array of nucleotide ranks. PostgreSQL does not always present a sequence in that form. A value may be an in-memory packed object, a compressed or out-of-line PostgreSQL value, a set of columnar fragments, or a view over a sequence source. It may also be the result of slicing, concatenation, reversal, or complementation. These cases have the same logical content but different costs and lifetime constraints.

Converting every input to one byte per symbol would give algorithms a simple interface. It would also decode complete values when a query needs only a small interval. For a sequence of length N and a requested interval of length m, that choice changes the immediate work from $O(m)$ to $O(N)$. It also creates a second representation that may be several times larger than the packed source. This cost is particularly undesirable for repeated SQL expressions and for alignment against long references.

BioSeqDB separates four properties of a sequence:

1. The logical sequence is a length and an ordered stream of alphabet ranks.

2. The packing geometry maps ranks to bits and physical storage units.

3. The representation owns or locates the data and defines its lifetime.

4. The access requirement states whether a consumer can stream symbols or requires a contiguous packed interval.

The same logical interface can therefore serve minimizer extraction, alignment, SQL operators, index construction, and accelerator uploads. A lazy value may allocate metadata and retain a source handle. It is still lazy when it postpones decoding and repacking the complete symbol payload until a consumer requests a representation that requires them. The resident interface is exposed by `compact_sequence_range`. The non-owning logical facade is `lazy_sequence_view`. Figure 8.1 shows this separation.

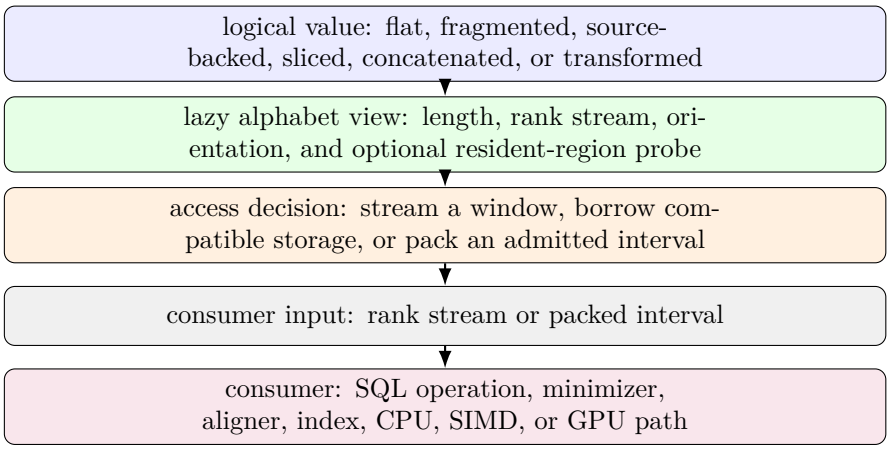


Figure 8.1: Sequence abstraction layers. Ownership remains with the value or an associated read session. Packed storage is acquired only when required by the consumer.

8.2. Alphabet-generic compact ranges

8.2.1. Logical and physical traits

The compile-time compact range represents the resident end of the design. Its alphabet type determines rank cardinality and biological operations such as complementation. A separate storage-unit type and bit width determine packing. Algorithms can therefore operate on DNA4, DNA5, or another registered alphabet without embedding a particular code table or machine-word width.

Listing 8.1 gives a conceptual interface. It is an API sketch rather than an implementation listing.

Listing 8.1: Conceptual compact-range interface.

```
compact range traits
    alphabet type
    storage unit type
    bits per symbol
    symbols per unit
    unused high bits per unit

compact range operations
    logical size
    physical unit count
    rank at logical position
    symbol iterator
    copy ranks from [first, last) into destination
```

The distinction between logical size and physical unit count is essential. If a unit contains L symbols and the last unit contains P empty symbol slots, then the logical length is

$$N = UL - P,$$

where U is the number of units. Code that treats U as a symbol count is correct only for unpacked storage with $L = 1$.

Packing width must also remain separate from algorithmic arithmetic. A 32-bit DNA4 storage unit holds 16 ranks, but a 27-symbol rolling key requires more than that unit can

represent. Hash state, score types, and coordinates are selected from their mathematical ranges. They are not inferred from the physical load width.

The range family has a common vocabulary but more than one ownership model. Table 8.1 summarizes the principal forms.

Table 8.1: Principal compact-range forms.

Input form	Physical form	Ownership rule
ordinary alphabet range	one value per symbol	borrows a stable lvalue from the caller
packed owning range	bit-packed vector	owns the packed payload
validated PostgreSQL value	typed trailing payload	borrows the resident datum and its established lifetime

A borrowed range must not be constructed from a temporary. Likewise, a range over a PostgreSQL value is valid only while the detoasted datum or read session remains live. The shared interface does not erase these obligations.

8.2.2. Ranks, symbols, and orientation

Rank iteration returns compact integers suited to inner loops. Symbol iteration converts each rank to the alphabet type and is useful when an operation needs alphabet-aware semantics. Keeping both forms avoids repeated symbol construction in numeric kernels while retaining a typed interface for validation and text conversion.

Interval extraction accepts logical coordinates $[from, to)$, source orientation, and destination order. Source orientation determines coordinate mirroring and complementation for a reverse-complement view. Destination order determines how the selected ranks are written. These are independent choices. A canonical minimizer cursor, for example, can inspect reverse-complement coordinates while retaining the order expected by its rolling state.

An allocating full conversion remains available at byte-oriented boundaries. It is not the default for alignment. The interval form seeks to the first relevant packed unit and decodes only the selected symbols. For resident data, its work is $O(to - from)$. The corresponding compact-range operation is `to_encoded_into`. Lazy and fragmented representations expose the same bounded rank-copy contract.

A logical slice has its own length but does not change the coordinate system of the underlying payload. If a forward view covers $[a, b)$, view position j maps to source position $a + j$. In a reversed view it maps to $b - 1 - j$. Nested slices first normalize their bounds against the current view, then compose this mapping with the parent. This rule avoids accumulating rounding or padding adjustments at each SQL operation. Padding belongs to the physical final unit, while slice bounds remain exact symbol coordinates.

Iterator categories reflect what the representation can actually provide. A resident compact range supports direct rank access. A source-backed view may offer the same logical operation through a bounded read rather than through a stable reference. Generic consumers should therefore express traversal in terms of ranks and windows. They should request a resident range only when their kernel truly depends on typed contiguous storage.

8.3. Packing and resident PostgreSQL values

8.3.1. Packing geometry

For an alphabet with σ ranks, the default bit width is

$$b = \lceil \log_2 \sigma \rceil.$$

A registered packing policy may choose a wider code, provided that every rank is representable. For a unit of W bytes, BioSeqDB stores

$$L = \left\lfloor \frac{8W}{b} \right\rfloor \quad \text{symbols and leaves} \quad r = 8W - Lb$$

unused high bits. For logical position i,

$$u(i) = \lfloor i/L \rfloor, \qquad s(i) = b(i \bmod L),$$

$$rank(i) = \big(unit[u(i)] \gg s(i)\big) \,\&\, (2^b - 1).$$

The first logical symbol occupies the least significant b bits of the first unit. Figure 8.2 shows the layout for eight DNA4 symbols in a 16-bit unit.

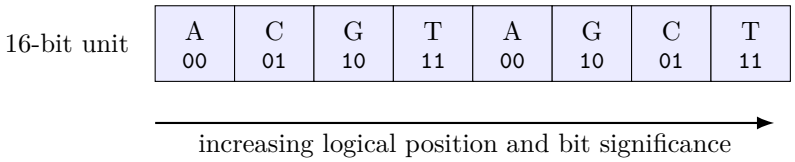


Figure 8.2: Least-significant-symbol-first packing for DNA4. Sequential extraction masks the low rank and shifts toward higher positions.

DNA4 uses two bits per rank and fills all standard unit widths. DNA5 needs three bits, so each unit can contain high bits that are not symbol slots. Those unused bits differ from the empty symbol slots in the final unit. Both values must be recorded correctly when computing length or converting widths.

A resident rank access uses one unit load, one shift, and one mask. Repacking an interval of m symbols takes $O(m)$ work. Cross-width views require more care. Matching symbol widths and divisible unit sizes are not enough to justify a typed pointer cast. Alignment, object lifetime, permitted type access, and byte order must also agree. BioSeqDB consequently distinguishes native typed borrowing from byte-compatible upload. A byte-compatible descriptor may be copied to an accelerator buffer, but it does not authorize a CPU to dereference the bytes through another unit type.

8.3.2. A flat trailing-payload representation

A C++ container is not a PostgreSQL datum. BioSeqDB therefore has a standard-layout packed object whose fixed header is followed by aligned data. The header records the alphabet code, storage-unit width, logical length, and padding in the last unit. Runtime metadata is checked before it is dispatched to an alphabet and width specialization. The inner rank loop then receives compile-time masks, complement rules, and lane types.

The trailing payload permits a zero-copy compact range when the PostgreSQL value is resident, uncompressed, aligned, and already validated. The adapter does not create a vector.

This statement does not mean that access to every SQL value is copy-free. PostgreSQL may first detoast or decompress a value according to its normal storage rules PostgreSQL Global Development Group, PostgreSQL 17 Documentation: TOAST. It also does not mean that a pointer remains valid after its memory context or read session ends.

Boundary validation checks header metadata, payload size, unit count, padding, and logical length for consistency. Only after this check does runtime metadata select a typed compact range. This places dynamic dispatch outside the symbol loop while preserving one SQL sequence type for several alphabets and storage widths.

8.4. A lazy view over flat and fragmented storage

8.4.1. The reader-facing facade

A packed pointer cannot describe a sequence that crosses fragments or draws its symbols from a sequence source. Joining all pieces before reading the first rank would defeat bounded access. BioSeqDB instead exposes a small alphabet-specific view whose backing representation is selected at run time. Its public behavior consists of:

- reporting the logical length
- writing a requested rank interval into caller-provided storage
- iterating bounded chunks in either orientation
- probing whether a compatible packed interval is already resident
- reporting whether the backing state is eligible for worker sharing

The facade stores a state pointer and a table of representation operations. It does not contain or own the complete sequence. A flat value, a fragmented scanner, a source-backed value, and a composition tree can therefore present the same rank-stream contract. Algorithms depend on that contract rather than on scanner or source types.

When a requested window lies inside one fragment, a backend can read it directly. A request that crosses boundaries is assembled in bounded scratch storage. Reverse traversal maps the logical window to source coordinates before changing write order and applying complement semantics. Thus a terminal slice does not require a prefix scan through earlier fragments. Source lookup and I/O costs still depend on the source, but the interface limits the requested coordinate range.

The resident probe is deliberately stricter than rank streaming. It succeeds only when the requested interval has stable storage, compatible packing, and a valid lifetime. A source cache page is not exposed as a resident region if the facade cannot retain its pin. This prevents a local optimization from creating a dangling pointer.

Listing 8.2 summarizes a window read. The pseudocode places coordinate normalization before backend access so that each backend receives one valid half-open interval.

Listing 8.2: Bounded rank extraction through the lazy facade.

```
READ_WINDOW(view, first, last, orientation, output)
    require 0 <= first <= last <= view.length
    mapped = map_to_source(first, last, orientation)
    for each intersecting piece in mapped order
        if piece is a literal
            decode only the intersecting packed ranks
```

```
    otherwise
        request only the intersecting source ranks
    apply output order and complement mapping
require output.count = last - first
```

The output buffer is supplied by the caller, which controls temporary memory. The backend may divide a large interval into smaller chunks, but it must preserve rank order and report the same logical result independently of piece boundaries. A consumer can therefore choose a chunk size suited to its cache or vector kernel without changing sequence semantics.

8.4.2. Ownership and failure handling

The lazy facade is non-owning. A call owner or PostgreSQL read session retains its state and releases source handles, detoasted values, arenas, and admitted allocations. Code that invokes the facade must finish before that owner is destroyed. Callback interfaces restrict results so that pointers, iterators, references, and range views cannot be returned accidentally. A callback must also avoid storing the view through a side effect.

This rule matters because PostgreSQL errors may transfer control without C++ stack unwinding. A read session registers cleanup actions and drains them on the PostgreSQL error path before rethrowing the database error. Relying only on ordinary destructors would leave resources live when an error crosses C++ frames PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Using C++ for Extensibility.

Worker eligibility is a capability, not an ownership token. A flat resident value may be worker-shareable. A composition is shareable only when every source leaf permits it. Any CPU task or accelerator submission that can outlive the current call first obtains a stable resident owner. The owner and its memory permit remain live until task completion or the accelerator fence.

8.5. Persistent slices and lazy composition

8.5.1. Immutable piece trees

SQL values must survive nested slicing, concatenation, and transformation. A raw pointer into another PostgreSQL value cannot supply that lifetime. It also cannot describe a slice that begins inside a packed unit. BioSeqDB represents such expressions as expanded PostgreSQL objects backed by immutable balanced piece trees.

There are three node kinds. A source leaf stores a retained source handle and stable logical coordinates. A literal leaf stores owned bytes with validated type information. A branch joins two subtrees and caches their combined length, piece count, and height. Reverse and complement states are attached to edges rather than stored as payload-bearing nodes.

Figure 8.3 illustrates the representation. Existing subtrees can be shared by later expressions because their nodes are immutable and their arenas retain dependencies.

Concatenation joins two roots and rebalances the tree. Slicing splits at the two boundary coordinates, retains the middle subtree, and shares unaffected nodes. Reversal changes an edge transform and visits branch children in the opposite logical order. At a leaf, the same transform maps coordinates, changes output order, and complements ranks when required.

Transform composition follows the biological operation rather than the tree shape. Two reversals cancel. Two complements cancel for alphabets whose complement mapping is an involution. Reverse-complement can therefore be recorded as a small edge state even when it is applied to a deeply nested concatenation. During traversal, the accumulated state

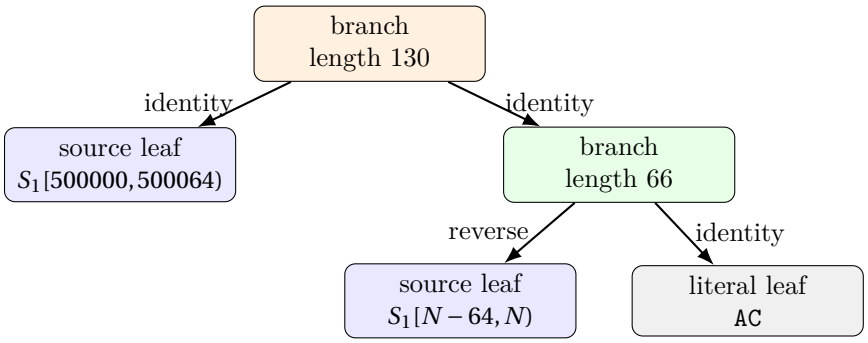


Figure 8.3: Persistent composition of source slices and a literal. Reversal is edge metadata. Neither source interval is copied into the tree.

determines child order and leaf decoding. This keeps repeated transformations independent of sequence length and avoids inserting long chains of unary nodes.

Joins of already compatible trees and all splits follow height-bounded paths. Concatenation that binds an unbound literal must validate and pack it. The SQL boundary may perform additional structural and size checks. The first expansion of a flat or out-of-line value can copy its packed bytes once without decoding ranks. Reading a selected interval later costs $O(m)$ for m emitted ranks plus fragment or source overhead.

The tree's arenas make sharing compatible with PostgreSQL memory contexts. They retain older arenas and source dependencies, and they release those dependencies through registered cleanup. As a result, a value returned by one SQL operator remains valid for the next operator without copying the original symbol payload.

8.5.2. Bound and unbound values

An expanded value records whether its alphabet is bound. A bound sequence has an alphabet and packing width. An unbound SQL literal contains canonical text but has not adopted an alphabet. Length, slicing, reversal, comparison, and unbound concatenation do not require complement semantics. Combining a bound value with an unbound literal validates the literal and adopts the bound alphabet. Two distinct bound alphabets are not silently recoded.

Element access returns a letter value containing both alphabet code and rank. This preserves identity in arrays and prepared statements. An in-range element of an unbound sequence is an error because its physical alphabet is unknown. An out-of-range element yields SQL `NULL`.

Equality and hashing follow logical content. For a bound value, the key contains a bound-state tag, alphabet code, and rank stream. Storage width, padding, flat or expanded form, source backing, and piece boundaries do not affect the result. An unbound value uses a different state tag and canonical text. It is therefore not equal to a bound sequence merely because both print the same characters.

8.5.3. SQL views and transforms

Sequence subscripting uses zero-based, half-open coordinates. Negative indices are relative to logical length. Omitted bounds select the corresponding end, and bounds are clamped to the valid range. If the normalized upper bound is below the lower bound, the result is an empty typed sequence. Missing elements produce `NULL`. Null bounds propagate `NULL`. Assignment, multiple dimensions, and non-unit steps are rejected.

Listing 8.3: Examples of logical sequence operations.

```
s[0]        returns the first letter
s[-1]       returns the last letter
s[:2]       selects [0, 2)
s[1:-1]     removes the first and last letters
s[4:2]      returns an empty typed sequence

concat(left, right)
reverse(sequence)
reverse_complement(bound_sequence)
```

Concatenation accepts sequence and textual operands as well as arrays of text or letter values. Reverse is defined for bound and unbound values because it only changes order. Reverse-complement requires a bound alphabet with a complete complement mapping.

A cast that keeps the alphabet and storage width can remain metadata-only. A same-alphabet width change records the target geometry and delays repacking until a flat consumer requests it. Binding an unbound literal validates each symbol. Relabeling a bound sequence as another alphabet fails. These rules are the same for flat, out-of-line, expanded, and source-backed values.

8.6. Access planning and materialization

8.6.1. Consumer requirements

Not every algorithm can stream. A scalar scan needs sequential ranks. A windowed algorithm may need bounded random access. A SIMD kernel or accelerator upload may require one contiguous packed interval. BioSeqDB represents this choice with three access categories. Their resulting routes are summarized in Table 8.2.

Table 8.2: Access routes selected from consumer requirements.

Route	Typical case	Copy and lifetime rule
stream symbols	forward scan over a tree, source, or fragment set	decodes bounded chunks into caller scratch with no complete packed allocation
borrow resident	aligned interval in a compatible flat value	performs no payload copy while the datum owner remains live
materialize admitted	unaligned interval or resident-only consumer	packs only the selected interval and retains its allocation permit through use

For sequential access, the consumer supplies bounded scratch space and reads chunks from the facade. For contiguous access, the planner first asks whether the representation can lend a compatible resident interval. The interval must begin at a boundary represented by the receiving range. A start in the middle of a packed unit cannot be passed as an ordinary compact range because the consumer would interpret the wrong first symbol.

If borrowing is impossible, the planner computes the packed size of the selected interval and obtains a memory permit. Materialization then packs only $[from, to)$. The returned owner holds both the allocation and its permit. Moving the owner into asynchronous work also moves the accounting lifetime.

The term zero-copy therefore applies to specific transitions, not to the sequence system as a whole. A validated flat value can be adapted by reference. A slice or transformation of an expanded tree can share payload nodes. A fragmented interval can be streamed without a

complete packed result. In contrast, PostgreSQL may detoast a value, an unaligned interval may require repacking, and text or binary output must create its destination form.

Durable storage is another explicit boundary. A source view is scoped to its transaction and cannot be placed directly into a heap or index tuple. A tree must be flattened before it crosses that boundary. If its logical size exceeds the flat PostgreSQL representation, the operation reports a size error rather than truncating the value.

Figure 8.4 follows the access decision from a bound SQL expression to a consumer.

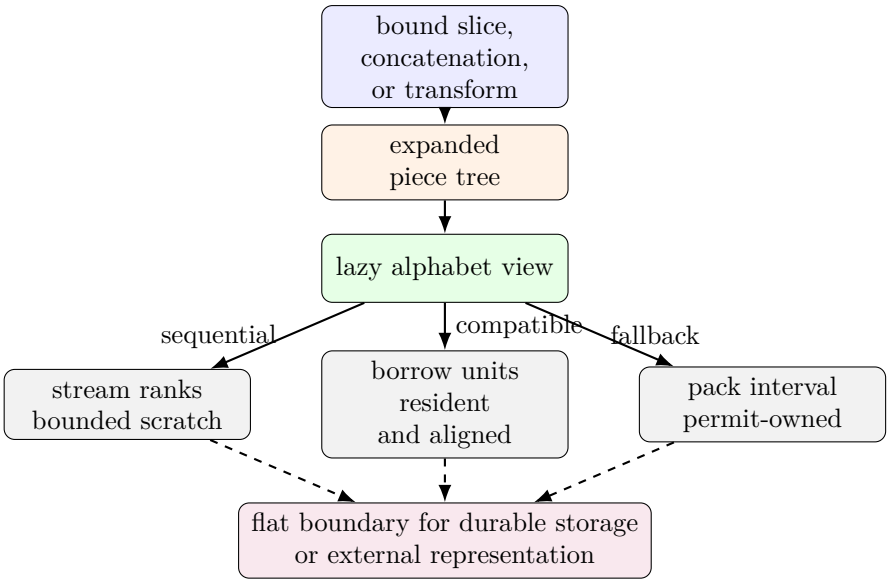


Figure 8.4: A bound lazy expression can finish through any access route. Flattening is performed only when the receiving boundary requires a durable or alternate representation.

8.6.2. Why eager conversion is avoided

Eager conversion has three effects beyond its immediate copy. First, it makes small slices pay for the full source. Second, it expands compact DNA ranks into byte or text buffers that place additional pressure on memory bandwidth. Third, it hides allocation inside a convenience operation, which makes memory admission and asynchronous lifetime difficult to account for.

Lazy composition changes the unit of work. Metadata operations preserve source coordinates and transforms. Streaming consumers request bounded chunks. Resident-only consumers name the interval they need and receive an owner for that interval. Conversions for external byte-oriented APIs remain valid, but they occur at a declared boundary and can be moved outside repeated calls when their input is invariant.

This design does not assume that laziness is always faster. A consumer that reads the complete value repeatedly may benefit from one admitted packed copy. The access planner makes that choice visible. The important property is that a slice or source-backed value is not forced into a complete resident vector before the actual consumer requirement is known.

8.7. Contract for sequence consumers

The sequence model can be summarized by the following rules:

1. Write alphabet-generic algorithms against compact-range traits or the lazy rank facade. Derive rank masks and complement behavior from the selected alphabet.

2. Keep storage width separate from hash, score, and coordinate width. Physical packing does not define an algorithm's numeric range.

3. Request the weakest access form that supports the computation. A forward scan should stream. A resident-only kernel should request and own the selected packed interval.

4. Convert only the required logical interval at a byte-oriented or durable boundary. Do not hide complete conversion in an iterator or convenience wrapper.

5. Treat every borrowed pointer as a capability tied to a named owner. Do not persist pointers into PostgreSQL values, source cache pages, or scratch buffers.

6. Preserve logical equality across flat, fragmented, source-backed, and composed forms. Representation details must not change rank content.

The earlier resident-vector model made one algorithm-friendly layout mandatory for every input. The current model makes logical ranks and ownership mandatory, while packed contiguity is an explicit request. This permits database-resident slices and fragmented sources to enter the alignment pipeline without treating all PostgreSQL and source storage as one kind of memory. The next chapter applies these representation and lifetime rules to PostgreSQL planning and custom scan execution.

Chapter 9

PostgreSQL API and custom scans

9.1. Alignment as a relational operation

The alignment pipeline described in the preceding chapters is not naturally expressed as a scalar SQL function. Its efficient unit of work is a batch of sequence pairs, and one pair may produce several accepted alignments. A scalar predicate instead receives one row, returns one Boolean value, and is normally evaluated independently at each call site. It cannot retain a group of inputs for vector or accelerator execution, and it cannot expand one input row into a set of match rows.

BioSeqDB nevertheless presents search through ordinary typed SQL. A typical query is

```
SELECT t.id,
       bioseq.score(t.id),
       bioseq.target_start(t.id),
       bioseq.target_end(t.id),
       bioseq.cigar(t.id)
FROM targets AS t
WHERE mmsearch(
        t.sequence,
        $1::bioseq.seq('dna5', 64))
```

The projected CIGAR value records the edit path of the current alignment SAM/BAM Format Specification Working Group, Sequence Alignment/Map Format Specification. The argument of each projection helper identifies the row lineage to which the value belongs. It does not contain the score or coordinates. These fields are supplied by the custom executor that owns the corresponding `mmsearch` expression.

Let input row i produce m_i accepted matches. The result cardinality is

$$N_{\text{out}} = \sum_{i=1}^{N_{\text{in}}} m_i, \qquad m_i \geq 0.$$

An input without a match produces no output. An input with one match behaves like a conventional filtering result. An input with several matches is projected several times, once for each match. Table 9.1 summarizes the resulting difference in execution semantics.

The SQL expression therefore acts as a rendezvous between PostgreSQL planning and BioSeqDB execution. It gives the parser a normal function signature and gives the planner a recognizable operation. It is not the location at which alignment is computed.

Table 9.1: Scalar evaluation and custom-scan execution.

Property	Scalar SQL function	BioSeqDB owner
Work unit	One tuple and one call	A bounded batch of sequence pairs
Cardinality	One scalar value per call	Zero, one, or many rows per input
Alignment fields	Independent calls have no shared match identity	All projected fields come from one match record
Execution route	PostgreSQL invokes the function directly	The scan selects the CPU or GPU pipeline for the batch

9.2. SQL surface and ownership contract

BioSeqDB provides a two-argument `mmsearch` form that uses session settings and a three-argument form that accepts typed alignment options. Both return `boolean` at the SQL type level. Match projections include the dynamic-programming score, query and target coordinates, edit count, mapping quality, strand, primary status, divergence, seed counts, and CIGAR. The usual rules for SQL types and null values remain applicable.

The Boolean return type permits `mmsearch` to appear in a `WHERE` or `JOIN ON` clause, but the ordinary function body is a failure boundary. If execution reaches it without a BioSeqDB owner, it reports that the query shape is unsupported. Returning `true` would be unsafe because any plan that missed the custom rewrite would accept every row. A cheap scalar fallback would also lose batching and multiple-match semantics.

Projected helpers follow the same ownership rule. During planning they remain ordinary expressions, so PostgreSQL can place them below an aggregate or on the nullable side of a join. During custom execution they are replaced with typed values associated with one match. A helper attributed to another target is not filled from the current alignment. This check prevents a score from one search from being combined with the coordinates of another search in the same query.

The requested projections also define the work required from the alignment pipeline. For example, CIGAR output requires traceback data, while a query that projects only coordinates need not format a CIGAR string. BioSeqDB snapshots the per-call or session options during planning. At scan initialization, it combines them with a field mask derived from the planned expressions. The batch then uses one consistent configuration.

9.3. Planner support, custom paths, and custom plans

The `mmsearch_support` function participates in PostgreSQL's planner support interface PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Function Optimization Information. It assigns a high startup cost to direct scalar evaluation. This cost discourages a plan that treats alignment as an inexpensive row filter, but it does not by itself create an owner. Correctness cannot depend on a custom path winning a cost comparison against an executable scalar path.

Relation and join path hooks perform the structural work. They locate the exact function expression and its `RestrictInfo`, then check argument types, relation lineage, parameterization, option shape, and expression placement. A base-table predicate can become a `BioseqScan`. A search between two relations can become a `BioseqBatchJoin`. A transformed or lazy sequence expression may require a `BioseqSliceProject` below the owner so that the evaluated sequence has a stable source identity.

Only after these checks does the planner construct a `CustomPath`. Its cost estimates source reading, operand evaluation, and alignment. Join ownership also includes materializing the query side and reading the target side. These are finite nonnegative costs. The large scalar cost identifies an undesirable unowned call, while the custom cost describes the actual work of the owning plan.

Figure 9.1 shows the division of responsibility.

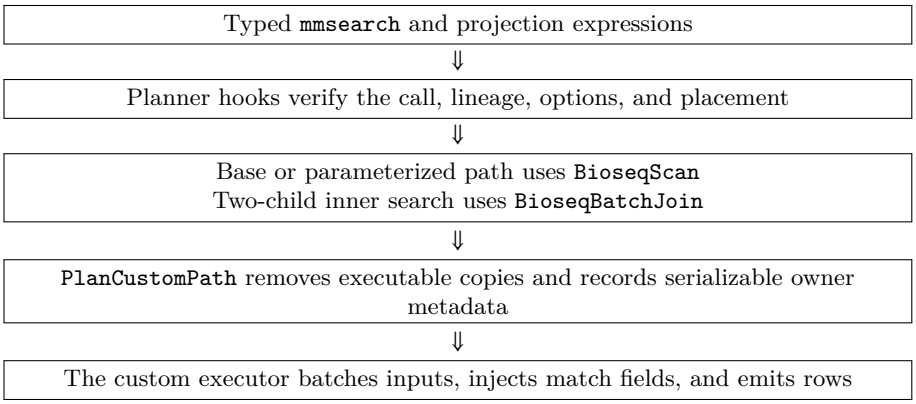


Figure 9.1: Planning and execution of an owned sequence search. The scalar function is retained for typing and planning, not as an alignment route.

PostgreSQL separates a `CustomPath` from its executable `CustomScan` PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Writing a Custom Scan Provider. BioSeqDB uses this boundary to keep process-local state out of the plan. Child paths are carried as PostgreSQL paths. Expression metadata that must pass through the setrefs stage is stored as expression nodes. Relation identities, source recipes, and version tags are stored in copyable plan data. Pipeline objects, buffers, and sequence providers are created later by the executor.

`PlanCustomPath` removes the owned `mmsearch` from ordinary, vectorized, and storage-level qualification lists. Leaving any copy in those locations would provide a second execution site for the same operation. The planner also publishes hidden child outputs before setrefs when an upper node will need an evaluated slice or a parameter. This allows PostgreSQL to bind the value through the normal plan tree rather than through hidden global state.

Plain `EXPLAIN` initializes the scan only for description and submits no alignment work. Prepared statements likewise store expressions and source descriptions, then bind their values when the plan is executed.

9.4. Typed placeholder injection

PostgreSQL evaluates a projection from an expression tree. After planner setrefs have established the final expression locations, a tree mutator replaces each attributable helper with a typed PostgreSQL `Const` node.

The node address is stable throughout execution. Before one output row is projected, the scan writes the current match fields into the node's value and null flag. Coordinates, lengths, and edit count use `BIGINT`. Scores and counters whose source representation is 32 bit use `INT4`. Status fields use `BOOLEAN`, divergence uses a floating type, and CIGAR uses `TEXT`. All placeholders for a row are populated from the same match record.

The injected node is an executor data channel, not a constant in SQL semantics. Injection occurs after planning, so PostgreSQL cannot fold the changing value into the plan. For each

emitted alignment, the custom scan updates the stable nodes, rebuilds the projection state, and then calls `ExecProject`.

Joins require an additional placement mechanism. PostgreSQL may move a helper into a child target list, and an outer join may later null-extend that child. The projection support function can wrap the helper in a `PlaceHolderVar`, preserving its evaluation level, relation set, and nulling relations PostgreSQL Global Development Group, PlaceHolderVar in the PostgreSQL 17.10 source tree. The custom scan replaces the owned expression inside that placement structure. A stable owner identity distinguishes otherwise identical helper functions that belong to different searches.

9.5. Bounded batch execution

The executor alternates between collecting input and draining output. The batch has a fixed capacity, so the complete relation need not be retained. When the capacity is reached, or when the source ends, the current batch is submitted to the selected alignment pipeline. Each input obtains a cursor over its accepted matches. PostgreSQL then pulls projected rows until these cursors are empty.

Algorithm 20 Conceptual execution of an owned `mmsearch`

1. Read source rows, apply ordinary residual conditions, and capture stable sequence references for non-null operands.
2. When the batch is full or the source ends, run the selected alignment pipeline and create a match cursor for each input.
3. For the next match, restore its input row, populate the requested placeholders, and call Project.
4. Return one tuple, then continue step 3 until all match cursors are empty. Release the batch and resume step 1 if input remains.

This order reconciles batching with PostgreSQL's pull executor. The parent still asks for one tuple at a time, but expensive processing occurs once for a group of inputs. An input with no cursor disappears. An input with three cursor entries is restored and projected three times with three coherent sets of match fields.

Tuple lifetime is important during this process. A table scan may reuse its active slot on the next call, and PostgreSQL resets short-lived memory between returned rows. BioSeqDB therefore copies the row representation needed by the batch and retains stable sequence-source descriptors. Large or fragmented sequences can remain locator backed while the provider acquires the regions required for the current dispatch. If a lazy value must be materialized for projection, the stable source is restored before another match from the same input is emitted.

Scalar and vectorized table access converge at the same submission boundary. A vectorized filter may first produce a keep mask, but every retained row is then evaluated for alignment operands and submitted to the common batch. This separation allows ordinary relational filtering to remain vectorized without turning alignment into a Boolean vector predicate.

An optional prefilter can reject candidates only under its stated lower-bound condition. Survivors still pass through alignment. A condition on an exact match field, such as an edit-count limit, is applied to the resulting match record before projection. The prefilter therefore does not replace the user's SQL condition with an approximation.

9.6. Join execution and rescans

A search predicate below a join can remain owned by its base `BioseqScan`. This is the simplest case when the query sequence is a constant or comes from the same row. For correlated nested execution, the query operand arrives as a PostgreSQL execution parameter. On rescan, the node clears retained rows and match cursors, rebinds the parameter, and starts a new batch. Rebinding is required because each outer row may provide a different query sequence.

A join-level search between target and query relations uses `BioseqBatchJoin`. The query child is materialized into stable sequence references, while the target child is streamed. The owner forms batches of target and query pairs, performs alignment once, and emits the ordinary columns of both children together with the current match fields. Residual join conditions are evaluated at the placement required by PostgreSQL.

When an upper owner needs a lazy slice or another supported transformation, a projection provider evaluates it. The provider records the base lineage and the transformation recipe. The batch join and its projection helpers can then refer to the same evaluated value without repeating the transformation.

The join-level rewrite is intentionally limited to supported inner-join shapes with one attributable search operation. The planner declines ownership when it cannot preserve row security, volatility, nulling, or expression order. Such a call reaches the explicit unsupported-query boundary if no lower owner exists. This behavior is preferable to executing a function that cannot produce the required match rows.

9.7. User-visible plan information

The owning node is visible in PostgreSQL `EXPLAIN`. A base search is reported as `Custom Scan (BioseqScan)`, while a two-child search is reported as `Custom Scan (BioseqBatchJoin)`. A lazy transformation may appear as `BioseqSliceProject`. The owned `mmsearch` must not remain as an ordinary `Filter` or `Join Filter`, because either label would indicate an executable scalar copy.

Plain `EXPLAIN` reports planned information such as projected columns, operand ownership, source policy, and the planned pipeline architecture. `EXPLAIN ANALYZE` adds the executor's observed row counts and the architecture that actually ran. It can also report batch submissions, index read or recomputation decisions, storage acquisition, and fallback reasons. Text, JSON, XML, and YAML output use the same structured description rather than separate algorithm names maintained by hand.

Ordinary residual conditions can still appear below the custom node, provided that they do not contain the owned search. Under `ANALYZE`, the custom-node row count is the number of emitted matches rather than the number of source rows. It can therefore exceed the table row count when one target has several accepted alignments. Conversely, a scanned target with no match does not contribute an output row. This interpretation follows directly from the cardinality equation at the start of the chapter.

A useful plan review therefore asks three questions. First, is the expected BioSeqDB custom node present at the correct relation or join level. Second, is the semantic `mmsearch` absent from descendant filters and join filters. Third, under `ANALYZE`, do emitted rows and the executed pipeline agree with the query and configuration. Cost alone does not answer these questions.

The custom interface has deliberate limits. Batching delays the first result until at least one dispatch completes. A join owner may need to materialize one child. Unsupported

security or join shapes are rejected. Each new match projection requires a SQL type, an ownership mapping, and a population rule. These restrictions follow from the requirement that one search operation has one executor owner and that every emitted field refers to one match.

9.8. Conclusion

BioSeqDB keeps a conventional SQL surface while assigning alignment to a custom relational operator. Planner support marks direct scalar evaluation as expensive. Structural hooks build an owning custom path, and plan construction removes every executable copy of the search predicate. Typed placeholders then connect one match record to ordinary PostgreSQL projection expressions.

The executor collects bounded batches and emits zero, one, or many rows for each input. Base scans, parameterized rescans, and inner batch joins share this ownership model. EXPLAIN exposes the selected owner and distinguishes planned from executed pipeline information. The next chapter describes the physical sequence representation that allows these nodes to retain small locators and acquire packed sequence regions only when alignment requires them.

Chapter 10

Physical sequence storage in PostgreSQL

10.1. Storage as part of query execution

A biological sequence can be represented as a PostgreSQL variable-length value, but this representation alone is insufficient for large reference collections. A sequence may exceed the size of one PostgreSQL allocation. An alignment may need only several bounded intervals from that sequence, while a metadata query may not need sequence bytes at all. Storing every sequence as one eagerly reconstructed value would therefore couple the cost of a query to the total payload size even when the requested work is small.

BioSeqDB separates ordinary columns from externalized sequence payloads. The table access method stores scalar values and sequence locators in a columnar layout. Packed sequence fragments reside in relations selected by sequence length and shard capacity. A locator in the parent row identifies the fragment range and the immutable source version. Readers resolve that locator only when an operator requests sequence data. Persistent minimizer indexes form a third relation family, described in the next chapter. They are derived from the sequence source and are not part of the primary payload layout.

This organization makes storage an execution property rather than a passive serialization choice. Projection determines which column streams are read. Scalar predicates can discard chunk groups before a sequence locator is loaded. A lazy scanner bounds the fragment data retained by one execution. PostgreSQL locks, snapshots, write-ahead logging (WAL), and publication rules govern when newly written stripes and fragments become visible.

The columnar layer was adapted from Citus Columnar Storage. BioSeqDB retains its page mapping Citus Data contributors, Citus Columnar storage implementation, stripes, chunk groups, per-column streams, compression metadata, and min/max pruning. It adds sequence-specific indirection and execution interfaces. Table 10.1 summarizes the resulting distinction.

10.2. Columnar relation layout

10.2.1. Logical byte space and reservations

The main relation fork begins with two non-data blocks. Block 0 holds the column-store metapage and block 1 is reserved. Logical stream data begin after the PostgreSQL page header in block 2. Let

$$P = \mathrm{BLCKSZ} - \mathrm{SizeOfPageHeaderData}$$

Table 10.1: Generic column-store functions and BioSeqDB additions.

Concern	Generic column store	BioSeqDB behavior
Physical organization	Stripes, chunk groups, compressed column streams	The same organization for scalar data and compact locators
Large values	One serialized value, possibly managed by PostgreSQL TOAST	Fixed-size packed fragments addressed by a validated locator
Filtering	Min/max pruning over ordered scalar values	Scalar pruning occurs before sequence access, while sequence values are excluded from min/max pruning
Reading	Reconstruct projected rows from selected streams	Publish scalar or batch slots and acquire sequence fragments on demand
Reuse	Backend-local decompression may avoid repeated decoding	Separate scopes for fragment residency, decompressed chunks, persistent indexes, and shared query results

be the usable payload of one database page. Let $x \geq 0$ be an offset relative to the start of the stream-data area. It maps to

$$\mathsf{block}(x) = 2 + \left\lfloor \frac{x}{P} \right\rfloor, \qquad \mathsf{inpage}(x) = \mathsf{SizeOfPageHeaderData} + (x \bmod P).$$

Higher layers can consequently reserve a continuous byte interval without splitting every compressed stream at page boundaries.

The metapage records the format and storage identities together with the first unused stripe number, row number, and byte offset. A writer reserves non-overlapping ranges while holding the relation extension lock. It then serializes its stripe into the reserved range. Metapage changes use PostgreSQL's generic WAL support, so crash recovery follows the database page and buffer model. Initialization and truncation require stronger relation locks because they replace the shared allocation state.

A reservation is not equivalent to a visible stripe. Completion records the actual row count, byte length, and stream metadata in the storage catalogs. Readers use published metadata under their PostgreSQL snapshot. They do not infer valid data from a byte range that a writer reserved but did not finish. This distinction permits concurrent writers to allocate disjoint ranges while preserving transactional visibility.

10.2.2. Stripes, chunk groups, and streams

A stripe is the large append unit. Each stripe is partitioned into chunk groups, and every chunk group covers the same logical row interval for all columns. The default configuration places up to 150,000 rows in a stripe and 10,000 rows in a chunk group. Both values are table options. Their purpose is to balance metadata granularity, compression ratio, and the amount of data handled in one scan step.

For each column in a chunk group, the writer produces two streams. An existence bitmap represents NULL values at one bit per row. A value stream contains aligned serialized values for present rows. The writer may compress the value stream with Zstandard, LZ4, or PostgreSQL PGLZ, depending on the build and table options. It retains the compressed form only when it is smaller than the original. The chunk metadata records the algorithm that was actually used, including the uncompressed case.

Figure 10.1 shows the hierarchy and its relationship to a projected scan. The figure separates physical page mapping from the logical units used by the reader.

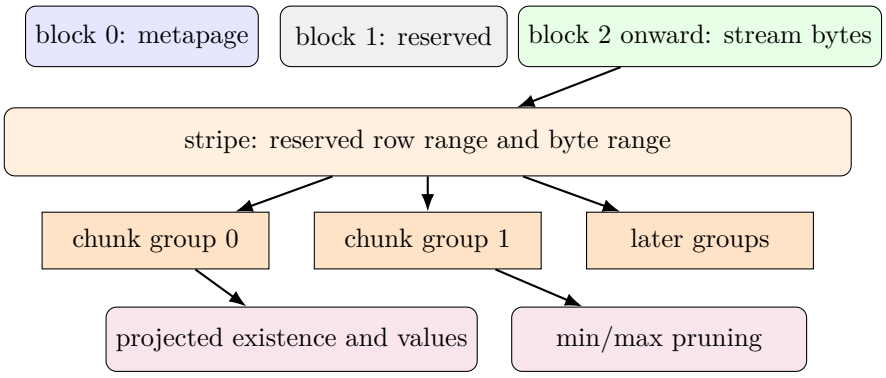


Figure 10.1: Column-store hierarchy. Published metadata maps stripes and chunks to the logical byte space. Projection and pruning determine which streams are decoded.

During a flush, offsets are calculated before any stream is copied. For one column, the physical order is its existence buffers followed by its value buffers. The completed metadata describes stripe and chunk-group row counts, stream offsets, compressed and uncompressed lengths, deletion masks, and compression identifiers. This metadata also supports columns added after an older stripe was written. Such a column is reconstructed from its PostgreSQL default or as NULL.

The metadata is divided by scope. A relation-options record contains the stripe size, chunk-group size, compression method, and compression level. A stripe record maps one storage identity and stripe number to its row interval and logical byte interval. Chunk-group records contain row and deletion counts. Per-column chunk records contain stream locations, value counts, compression details, and optional extrema. Row-mask records represent deleted logical rows without rewriting the immutable stream data. The sequence shard registry is separate from these ordinary column-store records because its key must also identify the parent column, length bucket, and fragment shard. Keeping these scopes distinct allows a narrow metadata change to be recorded without treating a fragment address as column-stream metadata.

The writer maintains min and max values for ordinary columns when PostgreSQL provides an appropriate comparison procedure. These extrema support chunk elimination. Sequence columns are excluded. A locator byte pattern has no biological ordering, and the lexicographic order of packed symbols does not provide a useful range predicate for alignment. A neighboring length, date, or sample category remains eligible for pruning.

Several limits prevent a chunk from becoming an unbounded allocation. A single aligned serialized value of 256 MiB or more is rejected. A stripe is flushed before a value buffer approaches its approximately 1.024 GiB bound. These limits apply to ordinary column chunks. Large biological sequences use the fragment layout described next, so their logical length is not restricted to one chunk buffer.

10.3. Fragmented and packed sequence payloads

10.3.1. Locator and fragment geometry

An externalized sequence is represented in its parent row by a normal `bioseq.seq` header followed by a 40-byte locator payload. The complete stub occupies 56 bytes. The header marks the value as a locator while retaining the alphabet identifier, packed storage width, and padding of the final fragment. The current locator format is version 3. It records the format version, length bucket, shard, fragment count, logical sequence length, first fragment row, and source epoch.

The source epoch identifies an immutable content version. A row address alone states where bytes were written, but it cannot show whether those bytes still belong to the same logical sequence after a storage lifecycle change. Readers therefore require a non-zero epoch and compare it with the registered source. They also recompute the expected fragment count and final padding from the logical length and packed representation. Invalid versions, reserved bytes, geometry, or epochs are reported as data corruption rather than followed as addresses.

The physical unit is alphabet-generic. For a packed storage unit of B bytes that represents L letters, one fragment contains

$$U_f = 1024 \text{ units}, \qquad S_f = 1024B \text{ bytes}, \qquad N_f = 1024L \text{ letters}.$$

For DNA4 stored in 64-bit units, a full fragment occupies 8 KiB and represents 32,768 bases. Other alphabets change L, and another packed unit changes B, without creating a DNA-only fragment path. Only the last fragment may contain padding.

10.3.2. Buckets, shards, and the canonical registry

Known-size insertions select one of 35 length buckets from the bit width of the final sequence length. The first bucket groups the smallest lengths, the middle buckets distinguish bit widths 9 through 41, and the last bucket covers larger values through the 64-bit range. A streaming insertion cannot know its final size when writing starts, so it uses the largest bucket. Bucketing is an operational choice rather than a biological classification. It prevents short and very long values from competing for the same active shard.

A bucket can contain several shards. All fragments of one sequence occupy one shard and receive contiguous row numbers. If r_0 is the first row, the row for fragment i is

$$r_i = r_0 + i, \qquad 0 \le i < n_f.$$

Known-size insertion estimates the required bytes and opens a new shard when the active shard lacks sufficient configured capacity. Streaming insertion checks the per-sequence cap while fragments arrive. A sequence is never split across shards. If it exceeds the configured shard limit, insertion fails with an error instead of creating a locator with ambiguous ownership.

The shard registry maps a parent storage identity, sequence column, bucket, and shard to the corresponding fragment relation and attachment relation. Its identity includes the database, tablespace, parent physical identity, column, bucket, and shard. This scope prevents an address in one database or tablespace from being accepted in another. The attachment relation can refer to derived state, while canonical persistent minimizer generations remain separate from the primary fragment rows.

Figure 10.2 follows one locator from the parent row to a bounded algorithm request.

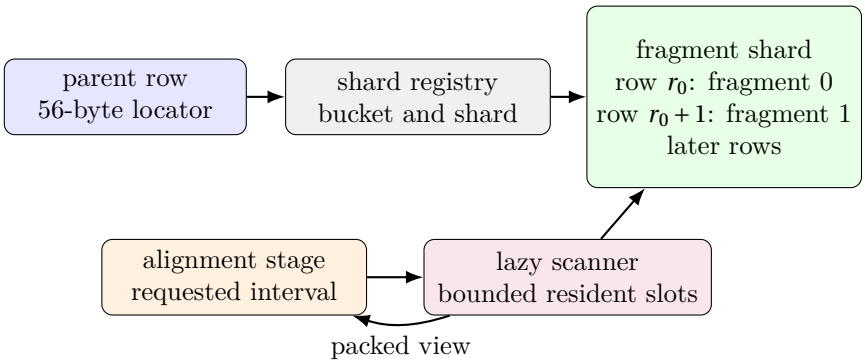


Figure 10.2: Sequence indirection and bounded acquisition. The scanner resolves the validated locator and reads only fragments needed by the requested interval.

10.3.3. Insertion, concurrency, and cleanup

Fragment allocation and writing use transaction-scoped advisory locks. One key coordinates allocation for a parent column and bucket. A second key also contains the shard and serializes writes to that internal resource. The keys are compact coordination identifiers, not full catalog identities. A hash collision can delay an unrelated writer, but it cannot make locators interchangeable because subsequent catalog and epoch checks use the complete identity.

Registry rows become authoritative only through the transaction that creates or updates the storage resources. Byte deltas accumulated during writing are applied when write states close. A reader holds a PostgreSQL snapshot and uses normal relation ownership rules, so it cannot treat an uncommitted fragment set as a complete source. Parent deletion and relation lifecycle handling are also explicit. Fragment relations are registered storage resources rather than permanent members of the extension installation. Parent-table drop processing removes associated resources, and orphan cleanup covers mismatches between registry rows and physical relations.

This separation matters because storage relations have a different lifetime from SQL extension objects. It also avoids changing extension membership from nested database execution contexts. The user-visible rule is simple. A committed parent row either resolves to a validated fragment source or fails with a storage error. It never silently reads a relation found only by a reused physical address.

10.4. Projection, pruning, and lazy reconstruction

A scan begins with the requested columns and predicates. For each visible stripe, the reader obtains its chunk metadata and determines which chunk groups can satisfy eligible scalar predicates. It then loads only selected streams for projected columns. The processing order is:

1. read stripe and chunk metadata under the scan snapshot
2. reject chunk groups contradicted by scalar min/max bounds
3. read existence and value streams for the remaining projected chunks
4. decompress when necessary and reconstruct aligned PostgreSQL values

5. apply deletion masks and residual predicates

6. resolve sequence locators only for rows and operators that need them

Consider a relation containing a packed sequence, an identifier, collection date, length, and sample attributes. A date predicate can reject a chunk group without loading the sequence locator stream. A projection of identifiers and lengths need not enter fragment storage at all. If an alignment operator needs the sequence, it receives a lazy packed range rather than a concatenated allocation.

The PostgreSQL-backed scanner retains a small set of open shard states for one execution. Fragment data are copied into scanner-owned memory with 16 fixed slots. With DNA4 and 64-bit units, the maximum full-fragment payload in those slots is

$$16 \times 1024 \times 8 = 131{,}072 \text{ bytes.}$$

A request contained in a resident fragment can return a direct packed view. A request crossing a boundary is copied into bounded scratch space. Slot replacement therefore decouples the resident sequence payload from the total logical sequence length.

This bound applies only to fragment acquisition. Minimizer anchors, chaining state, persistent index segments, and dynamic-programming matrices have their own memory budgets. Fragmentation prevents an implicit whole-sequence copy, but it does not make every alignment operation constant-memory.

The same column reader can publish scalar tuple slots or batch slots. A batch contains up to one chunk group's rows, projected vectors, NULL masks, a keep mask, and source row numbers. Supported comparisons and Boolean expressions can update the keep mask before scalar row materialization. Supported metadata aggregates may consume the batch interface. This use of the term vectorized describes batched executor calls and does not imply that every transition is a hardware SIMD instruction.

The planner uses the batch path only for a restricted single-relation plan whose child can publish the required slot type. DISTINCT aggregates, aggregate filters, alignment-owned projections, joins, and unsupported types retain the ordinary scalar executor. Empty and all-NULL inputs for some integer and date aggregates are a current limitation because their vector initial states do not yet match PostgreSQL's NULL results. The optimization must therefore be interpreted according to its stated eligibility and semantic limits.

Two local caches may reduce repeated work during these scans. Per-execution fragment state keeps open shard relations and bounded fragment slots. The decompression cache stores decoded compressed column chunks for one PostgreSQL backend. Its key includes relation, stripe, chunk group, and column. It has an explicit byte ceiling, skips chunks currently in use during eviction, and returns borrowed buffers whose lifetime remains tied to the cache entry. Neither mechanism coordinates work between sessions.

10.5. Coordination of repeated query requests

Shared reuse of a completed SQL result is separate from fragment storage and column decompression. The optional query-result cache can coalesce concurrent executions with the same execution identity. Its identity includes the analyzed query shape, parameter values, execution and security context, and a dependency version vector. Normalized SQL text alone is insufficient because two statements with the same text can run with different parameters or visibility rules. The cache must be loaded when the PostgreSQL server starts because its registry and memory areas are shared across backends.

The cache has three user-visible modes. In `disabled` mode, execution does not create a cache claim. In `exact_query` mode, the first executor of an exact identity becomes the producer. Concurrent identical executors wait, and later identical requests may replay the sealed ordered multiset while its dependencies and lifetime remain valid. In `reusable_fragments` mode, exact coalescing remains available. Broader reuse at an expensive operator boundary requires a provider that can prove compatible coverage and retain the lineage needed for residual predicates. Without that proof, behavior reduces to exact reuse. The current BioSeqDB installation does not supply a production coverage provider, so broader operator-level reuse remains unavailable.

The first exact claimant produces the result. A result becomes reusable only after every cache row group and its manifest have been sealed and checked against the schema and dependency versions. Producer failure or invalidation leaves no partial result that another query can replay. A canceled waiter releases its claim without canceling the producer, while a failed producer allows waiting queries to run normally or establish a new generation.

Completed results use checked columnar cache row groups. Memory and file admission have explicit ceilings. If neither tier can accept another row group, the original query continues without publishing a reusable result. Expiration is measured from successful publication, and cached results are cleared across a server restart because they are derived state rather than durable table data.

10.6. Resulting storage contract

The storage design gives each layer a defined responsibility. The Citus-derived column engine manages pages, stripes, chunk groups, scalar projection, compression, deletion masks, and min/max pruning. Sequence indirection adds alphabet-aware packed fragments, length buckets, shard capacity, locator validation, and source epochs. Lazy scanners translate bounded interval requests into fragment access. Shared query-result reuse has its own identity, publication protocol, resource admission, and failure behavior.

These boundaries support database execution without presenting a sequence as an ordinary column-store value. Metadata filters can reduce the candidate set before any sequence read. Alignment stages can request only the intervals they need. Concurrent writers reserve independent physical ranges, while readers observe only committed metadata and validated fragment sources. Repeated queries may share a sealed result, but an incomplete or stale generation is never exposed as complete. The result is a PostgreSQL storage model in which logical sequence length, scan projection, and retained memory are related by explicit contracts rather than by eager materialization.

Chapter 11

Persistent sequence indexes

11.1. Persistent indexes as derived database state

A minimizer index avoids sketching and organizing the same target for every alignment query. In a standalone mapper, the index can be a file that the user rebuilds when the reference changes. A current versioned database index has a stricter identity. It belongs to a physical relation, a sequence column, and a particular version of the source data. Its descriptor records the alphabet, minimizer parameters, hash scheme, occurrence policy, position width, storage format, and any execution interface that consumes the stored shape directly. Older asynchronous payloads have a narrower compatibility identity, as described in Section 11.4.

BioSeqDB therefore treats a persistent sequence index as reproducible database state, rather than as an opaque cache. The source sequence remains authoritative. Replacing or deleting an index does not change it. Conversely, an index built for an earlier source epoch is not eligible after the sequence changes, even if the relation and row identifiers still exist. This rule prevents an old payload from being attached to new sequence bytes by name alone.

The index has a control plane and a data plane. The control plane identifies a visible version and its compatibility contract. The data plane contains hash controls, positions, and navigation metadata. PostgreSQL stores the authoritative descriptor and, in the general path, the payload chunks. A grouped fragment index may additionally have an immutable sealed image for direct reading. That image remains subordinate to the PostgreSQL descriptor. It cannot become current without successful construction and publication of the corresponding generation.

Figure 11.1 summarizes the resulting lifecycle. Readers resolve the current descriptor under their snapshot. A writer builds a new generation without modifying the version already visible to those readers. Only a complete and compatible generation can replace the current descriptor.

11.2. Logical organization of minimizer indexes

BioSeqDB retains two logical minimizer organizations because they optimize different stages of the workload. Minimizers per fragment, abbreviated MPF, matches the fragmented representation of long sequences. Minimizers per sequence, abbreviated MPS, reorganizes the same information by hash range for selective loading. The registered index policy also includes compact symbol counts, which support planning and filtering but do not provide seed positions. The implemented persistent MPF and MPS policies admit the profile $k = 15, w = 10$.

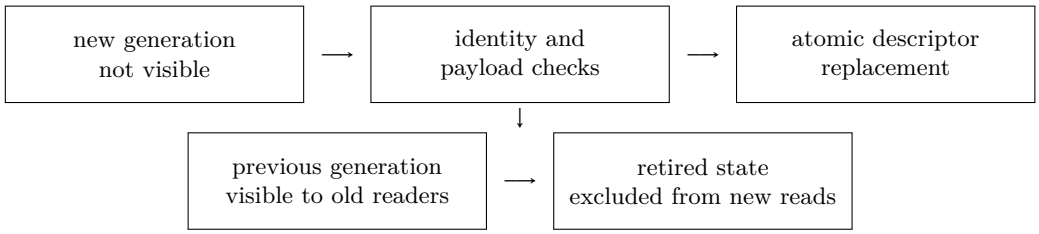


Figure 11.1: Publication of an immutable index generation. A failed build may leave removable unpublished data, but it cannot make an incomplete index visible. The legacy storage mode uses replacement within one relation and does not provide this generation lifecycle.

Other minimizer parameters can be used by transient execution, but these persistent layouts do not store them.

11.2.1. Minimizers per fragment

An MPF object is built for one source fragment. Its main components are a hash table and a contiguous positions array. Each occupied hash entry stores an offset and a count into that array. An occurrence encodes its local target position together with strand information. Readers rebase positions when assembling a queryable view.

Each fragment can be sketched independently. This bounds one construction unit and lets an insertion policy update affected index objects without sorting every minimizer in the logical sequence. A query over a long target must inspect or assemble several fragment images. The default path accepts this cost because it follows the existing storage and ingestion unit.

The grouped MPF representation, called Format 5, stores controls and positions in a specified array layout rather than serializing a language-specific hash container. The layout can be checked before use and can be adapted to a CPU or accelerator view. A sealed MPF5 image offers a direct read path for this grouped layout. It is an immutable reader representation, not an independently managed index provider.

11.2.2. Minimizers per sequence

MPS performs more work during construction so that a query can avoid loading unrelated target hashes. It combines fragment sketches into sequence-wide, hash-ordered buckets. A compact directory maps each stored hash interval to the bucket or stream region that contains its controls and positions. The reader searches this directory with the distinct query hashes, then admits only the intersecting regions.

The selective Format 6 organization separates three logical streams. The directory describes hash ranges and authorized spans. Control blocks store hashes and occurrence counts. Postings extents store positions for hashes that pass the occurrence policy. This order matters. The reader can reject a highly repetitive hash from its count without allocating or reading its full position list. The current Format 6 path requires DNA4 symbols, four-byte hashes, and the registered $k = 15, w = 10$ profile. Other combinations are not interpreted as this selective format.

Table 11.1 compares these organizations with the standalone minimap2 index discussed later in the chapter.

Neither BioSeqDB organization is always preferable. MPF avoids a global merge and follows incremental ingestion. MPS can reduce cold input when the query touches a small part

Table 11.1: Logical roles of persistent minimizer organizations.

Property	BioSeqDB MPF	BioSeqDB MPS	minimap2 `.mmi`
Construction unit	Source fragment	Logical sequence with bounded external merge	Reference index file
Lookup shape	Hash controls with a shared positions array	Hash-range directory, selected controls, and postings	Bucket-local hash tables and position arrays
Cold read	Fragment images needed by the target	Query-selected hash regions	Complete index reconstructed by the loader
Update model	Legacy insertion maintenance or explicit rebuild	Normally an explicit rebuild	User replaces the file
Identity	Relation, column, source epoch, algorithm, and generation	Same database identity plus directory and stream metadata	File parameters under the mapper's load contract
Main use	Default fragment-native path	Selective access for large targets	Reuse by a standalone mapping process

of the target hash space. The relevant factors are source fragmentation, update frequency, query selectivity, construction cost, and the memory available for transient assembly.

11.3. Bounded construction and publication

A sequence-wide index cannot safely be built by appending every $(hash, position)$ pair to one vector and sorting it in memory. The input may contain billions of bases, and one frequent minimizer may have a position list much larger than an ordinary work buffer. BioSeqDB instead uses bounded sorted runs. Pairs enter an admitted memory region. A full run is sorted and either retained under a residency permit or written to temporary database storage. The merge limits its fan-in, so the number of open readers and merge buffers is independent of the total number of runs.

The source is scanned in ranges with the halo required by minimizer extraction. Local positions are converted to global sequence coordinates before entering a run. The final merge groups equal hashes and writes storage units near a target size. A frequent-hash accumulator can also spill, which prevents one posting list from bypassing the general memory bound.

Algorithm 21 gives the high-level construction procedure. It omits physical row sizes and backend-specific calls because they do not change the algorithmic contract.

Algorithm 21 Memory-bounded construction of a sequence-wide minimizer index

Input. Source identity I, the registered minimizer profile $k = 15, w = 10$, and memory budget M.

1. Reserve a build identity and capture the source epoch.
2. Scan bounded source ranges with the required overlap.
3. Extract minimizers and append globally positioned $(hash, position, strand)$ records to the active run.
4. When a run reaches its permit, sort it and retain it only if memory remains available. Otherwise spill it.
5. Merge runs with bounded fan-in until one hash-ordered stream remains.
6. Group equal hashes, spilling a large posting list when necessary.
7. Write buckets or streams with lengths, checksums, and compatibility metadata.
8. Write the navigation directory after all referenced data is complete.
9. Check the unpublished object against I and its manifest.
10. Publish its descriptor atomically, or discard it on failure.

Writing the directory last is a basic completeness rule. A reader cannot infer a usable sequence-wide index from a prefix of buckets because there is no final directory that authorizes them. Immutable generation mode strengthens this rule. All rows and optional sealed artifacts belong to a fresh generation. The writer checks the complete object before replacing the small descriptor that names the current version. Readers that already resolved the previous descriptor may continue until their snapshot is no longer visible.

The older storage mode writes chunked objects in ordinary sidecar relations and replaces their contents within a transaction. It remains the default configuration and provides broad support for the registered index layouts. Grouped immutable generations are an explicit mode. They give stronger copy-on-publication behavior but impose additional manifest, retirement, and artifact management. The thesis distinguishes these modes because directory-last completion in one mutable namespace is not equivalent to snapshot-safe generation replacement.

Grouped derived index relations may be unlogged because their bytes can be rebuilt from durable sequences. This reduces duplicated write traffic, but it also defines a recovery obligation. After an unlogged reset, the prior index must not be advertised as ready. It must be rebuilt and published again.

The sealed direct-reader image follows the same authority rule. Construction first produces an authenticated grouped generation. The image is then written under a private name, made immutable, synchronized, and published without overwriting an existing generation. The database descriptor becomes current only after the final artifact has the expected identity. Retired descriptors are excluded from new reads. This sequence keeps filesystem publication consistent with transaction visibility without treating the image as durable source data.

Index updates are governed separately from reads. An index can be maintained on insertion or rebuilt only on request. MPF is the default minimizer index and is suited to insertion-time work. MPS is normally manual because construction requires a sequence-wide merge. Deferred persistence transfers stable relation identity and parameters to a database worker. It does not transfer live snapshots, relation handles, or memory owners. The worker opens the source under its own database context and rechecks the epoch and policy before writing.

11.4. Compatibility and integrity

A valid byte sequence is not necessarily a usable index. Query and target minimizers must be produced by the same hash scheme and parameter set. The reader also needs a position width capable of representing the source, a known control layout, and occurrence counts with the expected meaning. A direct accelerator view adds an execution interface requirement because its kernels interpret the stored shape without a general conversion step.

For current versioned MPF and MPS formats, the compatibility identity includes at least the following fields:

- source relation, sequence column, source epoch, and generation
- alphabet and encoded symbol width
- minimizer k, window w, hash scheme, and scheme version
- position width, occurrence policy, and persistent layout
- producer identity, and accelerator identity for directly consumed persisted outputs

Legacy asynchronous payloads do not carry the complete scheme identity. They are admitted only through their narrower layout contract. In particular, bounded ntHash reads are excluded from this legacy persisted path.

Manifests bind this identity to section lengths, ordering, checksums, and address ranges. Readers check arithmetic before constructing spans, so a corrupt count cannot wrap around and designate an apparently small allocation. Reserved fields and format versions are checked rather than ignored. Relation chunks are checked at the granularity in which they are admitted. A sealed image checks its fixed header and identity when opened, while complete section checks are part of publication and full inspection.

The SQL read requirement determines the response to absence or mismatch. A request that requires a compatible persisted index fails if no matching index is visible. It does not silently sketch the source. A permissive request may recompute a transient index. A request that requires absence bypasses persisted reads and forces transient recomputation even when stored data exists. These policies separate correct fallback behavior from measurements or operations that depend on a particular representation.

Compatibility checking occurs before a CPU or accelerator view is reused. A resident object is keyed by the persisted assembly identity and relevant algorithm parameters. Matching SQL types are insufficient because two payloads with the same row schema can use different hash schemes or control layouts.

11.5. Selective cold loading and memory accounting

Serialized size does not describe the peak memory of an index read. A backend may simultaneously own a directory, selected controls, postings, decoded positions, a transient lookup, query occurrence counts, input buffers, and an accelerator upload. BioSeqDB uses explicit permits for these significant owners. A component obtains admission before allocating or retaining its destination, and the permit remains held for the lifetime of cached or device-resident data.

Figure 11.2 shows the cold MPS path. The directory is small enough to read and check as a complete navigation object. Distinct query hashes identify the relevant control blocks.

Counts are then compared with the occurrence threshold. Only postings for surviving hashes are authorized, read, and clipped to the requested target interval.

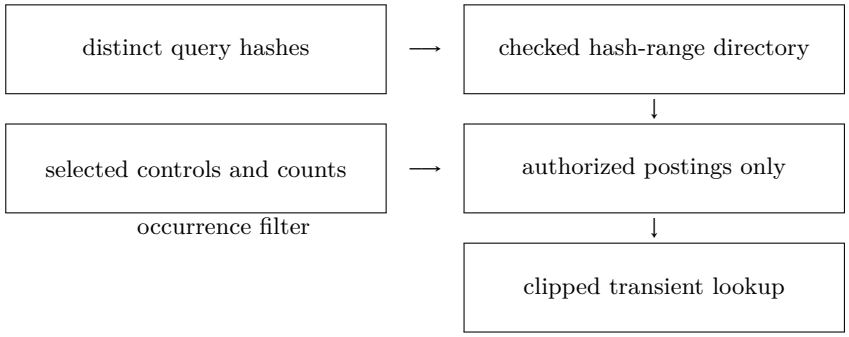


Figure 11.2: Query-selective cold loading. Navigation and counts are checked before position extents are admitted. Unselected and highly repetitive postings do not enter the backend's decoded working set.

Physical access must preserve this logical selectivity. Adjacent authorized ranges are coalesced into bounded read slabs, rather than fetched as many small objects. Checksums are evaluated at the addressed unit. For MPF, the direct sealed-image route avoids reconstructing a large immutable image through a large number of relation tuples. The relation generation remains authoritative for lifecycle and identity even when the read data plane is mapped.

Memory accounting distinguishes mapped address space from owned working data. Opening a large immutable image does not allocate an equally large decoded container, but operating-system page residency is not controlled by the backend's memory permit. The permit covers decoded arrays, assembled lookups, resident chunks, and device allocations. A memory-bound claim must state which of these domains are included. A process-cold measurement must also consider filesystem cache state separately.

Mapped-image lifetime is registered with database resource ownership so normal completion and error handling close it. Backend-local caching is bounded by entry and byte limits. The active relation reader can retain checked chunks, while runtime-specific caches may retain compatible derived views. Transaction abort and relation invalidation remove entries that may no longer be reused. These caches reduce repeated work within one backend, but they do not define visibility across backends. PostgreSQL snapshots and the published descriptor remain the source of coherence.

11.6. Runtime index selection

The alignment dispatcher resolves an index policy for each source sequence column. Presence alone is not enough. The dispatcher combines the requested index name with the read requirement, visible source epoch, compatibility identity, available reader route, memory admission, and execution backend. Algorithm 22 states the decision.

Algorithm 22 Runtime selection of a persistent minimizer index

Input. Source identity I, query minimizers Q, and read requirement R.

1. Resolve the active index policy for the source column.
2. If stored reads are permitted, resolve the visible descriptor.
3. If a descriptor exists, check its source epoch, algorithm identity, layout, lengths, and addressed checksums.
4. For MPF, read compatible fragment images or the configured sealed image, then obtain an admitted runtime view.
5. For MPS, search the directory with Q, load selected controls and postings, and assemble a clipped lookup.
6. Reuse a compatible resident view when its complete identity matches.
7. If no usable index exists and R requires persistence, fail.
8. Otherwise build a transient index and, when policy permits, schedule persistence using stable database identity.
9. Retain the selected descriptor and runtime view until the batch ends.

The default active minimizer path is MPF. When both deferred MPF and MPS policies could write an index, selection is deterministic and prefers MPF. This avoids presenting two incompatible background products as one implicit choice. MPS is selected explicitly when its construction cost is justified by selective reads. A strict persisted requirement turns an absent or incompatible object into an error. A permissive query can continue with recomputation.

Runtime selection also separates persistence from the final execution shape. The stored representation may be adapted into a CPU lookup or a compatible accelerator view. Those derived views have their own memory lifetime and cache identity. Device dispatch does not weaken host-side compatibility checks.

11.7. Differences from minimap2 `.mmi`

The minimap2 index is an appropriate algorithmic reference because BioSeqDB uses the same general family of minimizer seed lookup.[1] A `.mmi` file records minimizer parameters, reference metadata, bucket data, and optionally the packed reference sequence. Each bucket has a hash lookup and a positions array. A singleton position can be stored directly in the hash value, while a repeated minimizer points to a range in the positions array. The loader reads the file and reconstructs the complete in-memory bucket representation.

This design fits a command-line mapper. The caller chooses the file and replaces it when the reference changes. Stored parameters are interpreted by the mapper's own load contract. There is no need for a database snapshot to keep an older file version visible while another transaction publishes a replacement.

BioSeqDB retains the useful offset-and-count lookup idea, but changes the surrounding contract. Sequence-wide layouts use global coordinates, while MPF stores fragment-local positions because long sequence values are stored in fragments. MPS and the selective stream format can read only query-relevant hash ranges. Neither organization is wire-compatible with `.mmi`.

[1] Li, "Minimap2: pairwise alignment for nucleotide sequences"; Li, minimap2.

The larger difference concerns ownership. A versioned BioSeqDB index is bound to a relation, column, source epoch, algorithm identity, and published generation. Legacy updates can participate in insertion policy or an explicit rebuild. A new immutable generation is checked before publication, and the predecessor remains available to older snapshots. Cold loading is subject to backend memory admission, and cached CPU or device views inherit the same compatibility identity. These mechanisms are unnecessary for a standalone `.mmi` file, but they are required when index reuse is part of transactional SQL.

The direct sealed-image route does not remove this distinction. Although both systems can read an ordinary file efficiently, BioSeqDB's image is selected by an authoritative database descriptor, and its retirement is recorded transactionally. It is derived from the database generation and may be regenerated. It is not a user-selected replacement for PostgreSQL index state.

11.8. Operational limits and conclusions

Persistent minimizer storage reduces repeated construction, but it adds recovery and administration requirements. Unlogged payload relations must be rebuilt after reset. Sealed images must either be included in backup and replication procedures with their descriptors or be regenerated from the authoritative source. Operating-system page cache residency lies outside backend memory permits, so warm and cold measurements require separate interpretation.

The two retained organizations address different workload shapes. MPF follows fragment-native ingestion and remains the default. MPS pays for a bounded sequence-wide merge to gain query-selective loading. The versioned forms of both organizations include the source epoch and minimizer semantics in their compatibility identity. MPS construction uses admitted sorted runs and bounded spill. Immutable mode checks an unpublished generation before atomic descriptor replacement.

The resulting design differs from a database wrapper around a minimap2 file. It defines a transaction-aware index lifecycle with explicit logical organization, physical representation, publication, memory admission, and runtime selection. These properties allow persistent seed lookup to coexist with lazy sequence storage, concurrent SQL readers, and multiple execution backends.

Chapter 12

Bounded sequence ingestion and integrity limits

12.1. Bounded sequence ingestion and integrity limits

Alignment and search operate on compact sequences, whereas public archives usually distribute FASTA or FASTQ text. A FASTA record contains a header and a sequence body. FASTQ adds a separator and a quality string whose length must match the sequence. Records may span many input buffers, and one record may be larger than a PostgreSQL value. Ingestion must therefore combine record framing, compact encoding, metadata extraction, relational typing, and transactional storage.

BioSeqDB separates these concerns into a reusable input layer and a PostgreSQL-specific sink. The input layer reads bytes, selects the record grammar, parses incrementally, and emits headers with compact sequence chunks. It does not access PostgreSQL relations, tuples, or memory contexts. The sink selects the representation declared by the destination sequence column, converts header fields to SQL values, writes sequence fragments, and creates the parent row that refers to them. It evaluates defaults and the executor row constraints available to this bulk path. This boundary keeps file processing independent of database internals while retaining transaction visibility and rollback.

The resulting contract has four main properties. Multiline FASTA and FASTQ records are framed across arbitrary refill boundaries. Sequence symbols are packed directly into the destination representation. Packed sequence delivery is bounded even for very long FASTA records. A failed load does not commit a successful prefix as if the file had been loaded completely. FASTQ quality storage has a narrower guarantee, stated in Section 12.7.

12.2. Layered input pipeline

The lowest layer follows a small byte-source contract. A source fills a caller-owned span and reports the number of bytes read, end of input, or the first error. The same parser can consequently consume a mapped plain file, a sequential file descriptor, a zlib stream, or bytes supplied by another provider. The parser is specialized for its source, so refill does not require a virtual dispatch for every buffer.

Record format and compression are selected independently. The final non-compression suffix selects the FASTA or FASTQ grammar. Common FASTA and FASTQ suffixes are accepted case-insensitively, with an optional gzip suffix. The opened file is then inspected for the gzip magic bytes defined by RFC 1952 Deutsch, GZIP File Format Specification Version 4.3.

A compressed file can therefore be decompressed even when its name omits a gzip suffix. A plain file with such a suffix is not sent through a decompressor. By contrast, the parser does not guess FASTA or FASTQ from the first record. An unrecognized or misleading format suffix produces a grammar or format error instead of a heuristic reinterpretation.

For plain input, a mapped view is preferred when it is available and suitable for the file size. Otherwise, sequential reads refill a reusable slab. The SQL loader uses zlib for gzip input. Figure 12.1 shows where the portable input work ends and database work begins.

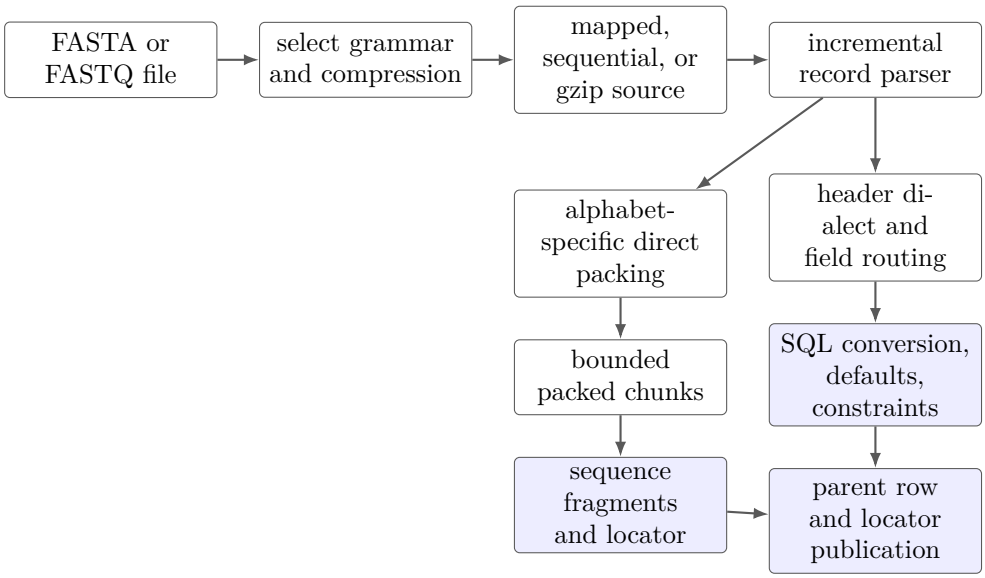


Figure 12.1: Sequence and header data follow separate paths after record framing. Packed sequence chunks are bounded. PostgreSQL operations are confined to the shaded boxes.

12.3. Incremental parsing and direct compact encoding

The parser is a finite-state machine over spans supplied by the selected source. A streamed source refills a recyclable input slab. When a record crosses a refill boundary, only its unread suffix is retained. A mapped source can expose input bytes directly. The general batch reader may enlarge its slab for an unusually large record, up to a configured hard limit. The SQL streaming path instead drains complete packed units while the body is being parsed. It does not first construct an ASCII sequence and then encode it, nor does it require the complete packed body to be contiguous in memory.

For FASTA, the states distinguish the header marker, header text, sequence body, and record end. Both LF and CRLF line endings are accepted, and the body may contain arbitrary line wrapping. On the common DNA4 path, one vectorized loop locates record boundaries and converts bases into two-bit storage words. Other registered alphabets use the same state machine with their corresponding packing operation. At record completion, the encoder flushes any partial word and records its padding. The represented base count can then exclude unused positions in the final storage unit.

FASTQ needs stricter framing because a quality line may begin with `@`, `+`, or `>`. Searching for a marker at the start of each physical line would therefore split valid records. The parser instead performs the following steps:

1. read one `@`-prefixed header

2. collect one or more sequence lines until a `+`-prefixed separator
3. skip the rest of the separator line
4. collect quality bytes until their newline-free length equals the raw sequence length

The final comparison uses the raw number of sequence bytes, not the number of symbols retained by the alphabet encoder. This matters because the current policy can discard an unknown symbol, while its corresponding quality byte is still present. A truncated quality field is a terminal input error. Excess quality bytes are rejected after a terminated line, but extra bytes in an unterminated final line are currently accepted. Multiline sequence and quality fields otherwise remain synchronized.

The destination sequence column controls representation. Before reading the first record, the loader requires exactly one parameterized sequence column and decodes its alphabet and storage-unit width. It then selects one compiled combination of alphabet, unit width, and FASTA or FASTQ grammar. Available layouts come from a compiled registry and can include DNA4, DNA5, and several storage-unit widths. Unsupported combinations fail during schema inspection rather than after a partial load. Each emitted chunk already uses the selected compact layout, so the database sink appends its bytes without a second conversion.

12.4. Bounded delivery and backpressure

The SQL path emits a begin-record event, a sequence of packed chunks, and an end-record event. Only complete storage units are emitted during a record. The last chunk carries any partial unit and the adjusted base count. A chunk view remains valid for its callback, allowing the database sink to copy it directly into fragment storage. This protocol bounds the packed body by the input slab and output window instead of the total sequence length. A multi-gigabase FASTA sequence can thus be stored without creating one multi-gigabyte database value.

PostgreSQL calls remain on the backend thread. The default path invokes the sink directly, which gives deterministic order and avoids cross-thread ownership. An optional overlap mode places owned begin, chunk, and end events in a fixed-capacity first-in, first-out queue. A producer performs only PostgreSQL-independent parsing. The backend thread consumes events and writes relations. A full queue blocks the producer, which supplies explicit backpressure. Closing the queue wakes both sides after an error, and the first consumer failure remains the reported cause. The overlap mode is disabled by default.

12.5. Header-to-column mapping

Archive headers often combine an identifier with structured metadata. UniProt, RefSeq, GenBank, NCBI, GISAID, and IMGT records use different positional fields, delimiters, and tagged values. BioSeqDB can preserve the entire header as text or extract recognized fields into SQL columns. The parser supports an explicit dialect, a plain mode, and automatic detection.

Automatic detection is bounded and occurs once per input file. The loader reads a limited decompressed prefix and samples a limited number of headers. FASTQ sampling follows logical sequence and quality lengths, so an `@` byte in quality data is not treated as a new header. Recognized dialects vote across the sample with deterministic tie handling.

If the sample is unstructured and no metadata mapping requires further parsing, the loader uses plain-header behavior. With metadata columns or user mappings, it retains automatic mode and applies generic per-record parsing.

Mapping is also resolved once. Explicit user mappings take priority. Remaining known fields are matched to normalized destination names, after which the loader chooses an identifier column and routes any generic metadata. Name normalization ignores ASCII case and common word separators. The resulting routing table maps positional or tagged header slices to destination columns. Per-record parsing then scans delimiters and passes raw byte ranges to the database boundary without knowing PostgreSQL types.

At that boundary, each destination's input conversion and typmod are cached. Unconstrained text can be constructed directly. Other scalar, range, or multirange values use PostgreSQL's normal type input rules, including session settings that affect dates and timestamps. Defaults are prepared once but evaluated for every row, so volatile defaults retain their SQL meaning. The completed tuple is checked with the executor's row-constraint step before it is written. This direct bulk path does not execute ordinary `INSERT` triggers or the secondary-index paths used for foreign-key, unique, and exclusion enforcement. Per-row memory is reset only after referenced values have been copied into database-owned storage.

The public operation can be summarized as `bioseq.load(target_table, file_path, header_format, columns)`. The optional header format defaults to automatic detection, and the optional mapping array can override inferred routes. The return value is the number of loaded records. Ambiguous mappings, unsupported sequence layouts, generated target columns, multiple sequence columns, and an unmapped required column without a default are rejected before streaming begins.

12.6. SQL publication and error behavior

Sequence data is stored outside the small parent tuple. For each record, the sink begins a sequence source, appends bounded packed chunks, and finishes the source to obtain a pending locator. It then parses the header, evaluates defaults, applies constraints, and writes the parent tuple containing that locator. Publication occurs only after the parent row has copied the tuple. This order avoids making an out-of-line sequence visible before a relational row refers to it. Publication does not commit the row. Transaction visibility still governs access to both objects.

The write is not equivalent to an ordinary `INSERT`. The loader writes through its bulk storage path after evaluating defaults and executor row constraints. It does not run normal insert triggers, maintain secondary indexes, or invoke the usual foreign-key, unique, and exclusion paths. A target relation must therefore be chosen with these limits in mind.

Input and parser failures are carried to the PostgreSQL boundary with an owned diagnostic. The boundary assigns an appropriate SQLSTATE and raises an SQL error. PostgreSQL conversion, default, and constraint failures keep their native messages. Error context adds the record number, the destination column when known, and a bounded prefix of the header. Limiting the header excerpt prevents a malformed record from creating an unbounded diagnostic.

Cleanup must preserve the earliest terminal error. Reader shutdown, queue closure, and relation cleanup cannot replace it with a later secondary failure. The target relation remains protected by a row-exclusive lock while the loader owns its executor, tuple slot, and write states. On success, both parent and fragment write states are flushed before newly loaded rows are used for optional index construction. These resources are also released on the error

```
inspect target and select format, alphabet, and unit width
detect the header dialect and resolve column routes

for each record
    begin an out-of-line sequence source
    append each bounded packed chunk
    finish the source and obtain a pending locator
    convert mapped header fields to SQL values
    evaluate unset defaults and enforce executor row constraints
    write the parent tuple with the locator
    publish the sequence source

flush parent and fragment writes
make the new rows visible to subsequent work in the transaction
optionally build configured sequence indexes
return the record count
```

Figure 12.2: Simplified loading procedure. Representation and routing decisions are made once, while row conversion and constraints are applied per record.

path. On failure, PostgreSQL aborts the enclosing transaction or subtransaction. Rows and sequence fragments written before the fault are not committed as a successful prefix. Within a larger explicit transaction, the caller can roll back to a savepoint established before the load.

This design does not introduce a second transaction protocol in the input layer. Locks, write-ahead logging, resource ownership, command visibility, and rollback remain PostgreSQL responsibilities. The reusable parser reports records and errors, while the database sink determines whether their effects become visible.

12.7. Integrity limits

The bounded claim applies to packed FASTA sequence bodies. The FASTQ adapter currently retains quality bytes until a record is complete, so its quality scratch space grows with record length. Sequence packing is still incremental, but total FASTQ memory is not independent of the largest record.

Alphabet checking is also permissive. Unknown FASTA and FASTQ symbols are currently dropped by the encoder. A strict-alphabet option exists in the reader interface but is not applied by the parser. The selected alphabet therefore determines representation, not complete validation of every input byte. Applications that require rejection of unknown symbols must validate them before relying on the current loader.

The SQL gzip path uses zlib's stream validation. A decompression error after output has begun is terminal because restarting with another decoder would duplicate the already delivered prefix.

Finally, the filename suffix remains authoritative for choosing FASTA or FASTQ. Magic-byte inspection identifies compression only. This separation keeps parser dispatch deterministic, but a mislabeled biological format is reported as an error rather than detected automatically.

12.8. Conclusion

BioSeqDB ingestion joins an incremental biological file parser to PostgreSQL without materializing a complete large sequence. The parser frames records and packs symbols into bounded native chunks. Header routing supplies typed column values, and the database sink coordinates fragment storage with parent-row publication and transaction rollback. The stated FASTQ and alphabet-validation limits define where this guarantee does not yet apply.

Chapter 13

External sequence providers

13.1. Remote databases as relations

Public sequence databases differ in identifiers, response formats, and retrieval models. NCBI separates taxonomy, assembly, gene, and sequence services National Center for Biotechnology Information, NCBI Datasets v2 REST API documentation. UCSC provides assembly and interval retrieval UCSC Genome Browser Project, UCSC Genome Browser REST API data interface. UniProt serves protein records and annotations UniProt Consortium, Programmatic access to UniProt, while EBI supports cross-domain discovery European Bioinformatics Institute, EBI Search REST API documentation.

BioSeqDB defines six remote relation shapes in the `bioseq_external` schema, but the generated table definitions currently omit the required relation-entity option. An installation must add that option before the custom scan can execute them. They cover taxonomy, provider catalogs, genes, locations, complete sequences, and bounded sequence chunks.

When configured, these relations have no local heap data. PostgreSQL replaces a physical scan with a custom scan PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Writing a Custom Scan Provider. Every qualification remains a local PostgreSQL check. Supported equality and array predicates are also lowered into a provider-neutral query that constrains the remote request. The scan selects a provider adapter and emits ordinary typed tuples. A remote record can therefore participate in a projection, filter, or join without a separate download script.

Table 13.1: External relation groups and their resource contracts.

Relation group	Result	Resource behavior
Taxonomy and catalog	Normalized metadata rows	Provider response is decoded before row production
Gene and location	One row per nested item	PostgreSQL applies residual predicates after normalization
Sequences	One packed sequence value	Memory may depend on record length
Sequence chunks	Ordered packed values	Accession-based NCBI retrieval uses a bounded queue

A custom scan retains state between tuple requests, postpones network access until execution, and reports the chosen provider through `EXPLAIN`. Ordinary `EXPLAIN` performs no network request, while `EXPLAIN ANALYZE` consumes the remote stream.

13.2. Provider and execution boundaries

Provider integration combines a capability descriptor, a generated typed HTTPS client, and a handwritten adapter. The client handles validation, encoding, size limits, and transport. The adapter maps provider fields to a canonical query and message model, keeping provider-specific formats outside PostgreSQL planning.

The adapters have different capabilities. NCBI covers metadata and nucleotide retrieval, UCSC covers catalogs and intervals, UniProt supplies protein metadata, and EBI supports discovery. Protein sequence materialization is not available through the nucleotide-oriented sequence type. A request that no selected provider can serve fails explicitly. In a multi-provider scan, an adapter error can be suppressed when another adapter returns rows, so such a result may be partial.

At scan initialization, provider predicates select adapters, while supported equality and constant array predicates constrain remote requests. Array predicate expansion is currently capped at 64 equality-query combinations. Excess combinations are omitted with a warning, so callers that require complete results must remain below this limit. At execution time, the scan opens a provider session on first demand. Metadata queries can collect a bounded set of owning messages. Large NCBI nucleotide queries instead use an incremental FASTA source and a queue bounded by both bytes and item count. Only the PostgreSQL backend thread creates `bioseq.seq` values and writes tuple slots. Network workers exchange ordinary owning values and never call the PostgreSQL API.

Algorithm 23 External relation scan

Require: relation kind, predicates, requested columns
1: retain all predicates locally and lower supported predicates remotely
2: select the requested or default adapter
3: open the provider session when the first row is requested
4: while the executor requests another row do
5: obtain the next provider message and convert required sequence values
6: place typed fields in the tuple slot for residual SQL processing
7: end while
8: cancel and close the session on rescan, error, or query completion

The streaming transport verifies its HTTPS destination, restricts redirects, and applies connection and progress time limits. The archive path also applies explicit wire-size and decoded-size limits. Incremental NCBI FASTA retrieval bounds queue and carry memory but does not impose a total response-size limit. The NCBI streaming scan checks PostgreSQL cancellation between bounded waits. Retry is allowed only before the first row is emitted, which prevents duplicated visible chunks.

Whole-record retrieval still requires record-sized memory, and archive retrieval requires bounded buffering before decoding. Requests are not currently rate-coordinated across backends. Chunked retrieval is preferable for large records.

Chapter 14

Source-driven SQL code generation

14.1. One source for the PostgreSQL boundary

The preceding chapter described clients generated from provider-owned OpenAPI descriptions. The generator considered here serves a different boundary. It derives the extension's PostgreSQL interface from BioSeqDB declarations.

A PostgreSQL extension repeats the same contract in several forms. SQL needs a routine name, argument types, result type, volatility, and permissions. The shared library needs a matching C entry point. C++ code must decode the same arguments and construct the declared result. Tables, operators, casts, configuration variables, and comments introduce similar duplication. A local change can therefore leave individually valid artifacts that disagree at the boundary.

BioSeqDB makes typed C++ declarations the source of these facts. Attributes describe only information that cannot be inferred from the declaration, such as an SQL type binding or a PostgreSQL volatility category. Function names, parameter order, and C++ types remain ordinary language constructs. Repeated annotations and conflicting defaults are rejected. This keeps annotations from becoming another complete copy of the interface.

The generator performs structural compiler analysis rather than searching source text. It observes active declarations, attributes, parameter types, initializers, documentation, and module ownership under the configured build profile. These records are normalized into a closed intermediate representation. Each SQL entity receives a stable identity and explicit dependencies. Tarjan's method classifies strongly connected components, then a stable topological traversal emits the component graph in deterministic order Tarjan, "Depth-First Search and Linear Graph Algorithms."

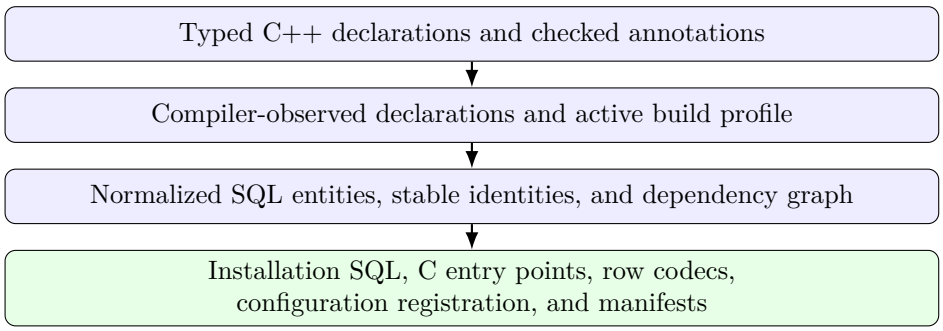


Figure 14.1: Source-driven generation of the PostgreSQL extension boundary.

14.2. Generated artifacts and safety

One declaration can produce several related artifacts. Functions normally yield installation statements, generated function-manager entry points, conversions, documentation, and manifest records. A row declaration can also yield tuple ordinals and codecs. An annotated configuration value yields registration code with a typed default, range, context, and description. A closed type registry rejects unsupported C++ and SQL combinations rather than falling back to an untyped `Datum` convention.

Dependencies determine a stable generation order. Generated outputs are written to the build tree and used directly as extension artifacts.

Error handling requires an additional boundary. PostgreSQL can report a recoverable error by a non-local jump that does not run intervening C++ destructors PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Using C++ for Extensibility. Conversely, a C++ exception must not cross a PostgreSQL callback. Generated wrappers therefore translate both directions through a common boundary. Diagnostics are copied to plain state, C++ owners are destroyed, and only then is the PostgreSQL error raised. Code that calls PostgreSQL below the wrapper still needs a bridge or a region without live nontrivial C++ objects.

The model is deliberately closed. Adding a new PostgreSQL object category requires a typed representation, validation, dependency rules, and a renderer. Compiler-based collection also adds build cost. In return, SQL declarations, C symbols, converters, tuple layouts, and configuration metadata are derived from the same program rather than maintained as independent inventories.

Chapter 15

Conclusions

BioSeqDB demonstrates that sequence search can be represented as a PostgreSQL execution operation rather than as a scalar call to an external mapper. The system stores packed biological sequences, constructs persistent minimizer indexes, and runs a minimap2-derived pipeline for seeding, chaining, filtering, and final extension. Planner support and custom scans preserve the batch structure required by SIMD and GPU implementations while presenting typed SQL results that can participate in filters, projections, and joins.

The first research question concerned a shared representation for the algorithmic stages. Alphabet-generic compact ranges cover resident values, while lazy sequence views represent fragmented, sliced, and composed sources. Algorithms read the required region without first converting the complete value to a byte array. The second question concerned heterogeneous execution. Minimizer extraction and chaining provide regular work for the implemented SIMD and GPU routes. Dispatch follows explicit algorithm settings, input compatibility, index form, and resource availability. Final alignment uses banded or unbanded KSW2, adaptive Block Aligner, and GPU routes that may return specialized scripts before device dynamic programming. Optional prefilters may reject unsuitable candidates but do not replace alignment.

The third question concerned database execution. A custom scan owns the search expression, batches input rows, and substitutes match fields into planned expressions. This design also permits multiple alignments for one input row. The fourth question concerned the surrounding data system. Physical sequence storage, persistent indexes, bounded ingestion, external-provider relation definitions, cache coordination, and source-driven SQL generation give the kernels explicit storage and lifecycle contracts.

The remaining limits are operational and algorithmic. Some remote retrieval paths still require record-sized or archive-sized memory. Provider quotas are not coordinated across all PostgreSQL backends. Hardware-specific routes have batch-size thresholds and require broader evaluation on additional devices. GPU chaining may examine a bounded predecessor set, and seed and alignment backends retain route-specific contracts. The present mapper is centered on linear references rather than pangenome graphs. A relational database can nevertheless plan and execute batched sequence analysis while retaining typed data, bounded resources, persistent derived state, and explicit algorithm selection.

References

Alser, Mohammed, Hasan Hassan, et al. “GateKeeper: a new hardware architecture for accelerating pre-alignment in DNA short read mapping.” In: Bioinformatics 33.21 (2017), pp. 3355–3363. doi: 10.1093/bioinformatics/btx342. arXiv: 1604.01789 [q-bio.GN]. url: https://doi.org/10.1093/bioinformatics/btx342.

Alser, Mohammed, Taha Shahroodi, et al. “SneakySnake: a fast and accurate universal genome pre-alignment filter for CPUs, GPUs and FPGAs.” In: Bioinformatics 36.22-23 (2020), pp. 5282–5290. doi: 10.1093/bioinformatics/btaa1015. arXiv: 1910.09020 [q-bio.GN]. url: https://doi.org/10.1093/bioinformatics/btaa1015.

Bingöl, Zülal et al. “GateKeeper-GPU: Fast and Accurate Pre-Alignment Filtering in Short Read Mapping.” In: IEEE Transactions on Computers 73.5 (2024), pp. 1206–1218. doi: 10.1109/TC.2024.3365931. arXiv: 2103.14978 [q-bio.GN]. url: https://doi.org/10.1109/TC.2024.3365931.

Citus Data contributors. Citus Columnar storage implementation. Version 14.0.0. AGPL-3.0. Feb. 17, 2026. url: https://github.com/citusdata/citus (visited on 07/28/2026).

Crick, Francis. “Central Dogma of Molecular Biology.” In: Nature 227.5258 (Aug. 8, 1970), pp. 561–563. doi: 10.1038/227561a0. url: https://doi.org/10.1038/227561a0.

Deutsch, Peter. GZIP File Format Specification Version 4.3. RFC 1952. Internet Engineering Task Force, May 1996. doi: 10.17487/RFC1952. url: https://www.rfc-editor.org/rfc/rfc1952.html.

European Bioinformatics Institute. EBI Search REST API documentation. Undated online source. url: https://www.ebi.ac.uk/ebisearch/documentation/rest-api (visited on 07/28/2026).

Feintzeig, Jake. SIMD + Parallelism: Ray Tracer Optimization Part 3. Undated online source. url: https://www.jakef.science/posts/simd-parallelism/ (visited on 08/19/2026).

Gotoh, Osamu. “An improved algorithm for matching biological sequences.” In: Journal of Molecular Biology 162.3 (1982), pp. 705–708. doi: 10.1016/0022-2836(82)90398-9. url: https://doi.org/10.1016/0022-2836(82)90398-9.

Groot Koerkamp, Ragnar and Igor Martayan. “SimdMinimizers: Computing Random Minimizers, fast.” In: 23rd International Symposium on Experimental Algorithms. Vol. 338. Leibniz International Proceedings in Informatics. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 20:1–20:19. doi: 10.4230/LIPIcs.SEA.2025.20. url: https://doi.org/10.4230/LIPIcs.SEA.2025.20.

Han, Yeejoo et al. “G^{3}SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment.” In: Proceedings of the 39th ACM International Conference on Supercomputing. ICS ’25.

Salt Lake City, Utah, USA: Association for Computing Machinery, 2025, pp. 173–188. doi: 10.1145/3721145.3729516. url: https://doi.org/10.1145/3721145.3729516.

Institute, National Human Genome Research. The Cost of Sequencing a Human Genome. Version 17. Undated online source. url: https://www.genome.gov/about-genomics/fact-sheets/Sequencing-Human-Genome-cost (visited on 07/28/2026).

Juliusz K Mateusz C, Szymon D and Damian W. "Master thesis: An integrated and practical database solution for storage and efficient processing of large genomic datasets." In: MIM UW 1.1 (2022). url: https://mimuw.edu.pl/en/.

Knuth, Donald E. The Art of Computer Programming, Volume 3: Sorting and Searching. 2nd ed. Reading, Massachusetts: Addison-Wesley, 1998. isbn: 0-201-89685-0. url: https://cs.stanford.edu/~knuth/taocp.html.

Li, Heng. minimap2. Version 2.30. url: https://github.com/lh3/minimap2/tree/79c9cc186b95f50bd899f69b48eba995ced810c6 (visited on 07/28/2026).

— "Minimap2: pairwise alignment for nucleotide sequences." In: Bioinformatics 34.18 (2018), pp. 3094–3100. doi: 10.1093/bioinformatics/bty191. arXiv: 1708.01492 [q-bio.GN]. url: https://doi.org/10.1093/bioinformatics/bty191.

Liu, Daniel and Martin Steinegger. "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices." In: Bioinformatics 39.8, btad487 (2023). doi: 10.1093/bioinformatics/btad487. url: https://doi.org/10.1093/bioinformatics/btad487.

National Center for Biotechnology Information. NCBI Datasets v2 REST API documentation. Undated online source. url: https://www.ncbi.nlm.nih.gov/datasets/docs/v2/api/ (visited on 07/28/2026).

Needleman, Saul B. and Christian D. Wunsch. "A general method applicable to the search for similarities in the amino acid sequence of two proteins." In: Journal of Molecular Biology 48.3 (1970), pp. 443–453. doi: 10.1016/0022-2836(70)90057-4. url: https://doi.org/10.1016/0022-2836(70)90057-4.

PostgreSQL Global Development Group. PlaceHolderVar in the PostgreSQL 17.10 source tree. Version REL_17_10. Undated online source. url: https://github.com/postgres/postgres/tree/REL_17_10 (visited on 07/28/2026).

— PostgreSQL 17 Documentation: Function Optimization Information. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/xfunc-optimization.html (visited on 07/28/2026).

— PostgreSQL 17 Documentation: TOAST. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/storage-toast.html (visited on 07/28/2026).

— PostgreSQL 17 Documentation: Using C++ for Extensibility. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/xfunc-c.html (visited on 07/28/2026).

— PostgreSQL 17 Documentation: Writing a Custom Scan Provider. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/custom-scan.html (visited on 07/28/2026).

Roberts, Michael et al. "Reducing storage requirements for biological sequence comparison." In: Bioinformatics 20.18 (2004), pp. 3363–3369. doi: 10.1093/bioinformatics/bth408. url: https://doi.org/10.1093/bioinformatics/bth408.

Sadasivan H Maric M, Dawson E et all.
mm2-gpu: Accelerating Minimap2 for Accurate Long Read Alignment on GPUs. Undated online source. doi: https://doi.org/10.26502/jbb.2642-91280067. url: https://github.com/Minimap2onGPU/mm2-gb (visited on 08/19/2026).

SAM/BAM Format Specification Working Group.
Sequence Alignment/Map Format Specification. Version 1.6. Undated online source. url: https://samtools.github.io/hts-specs/SAMv1.pdf (visited on 07/28/2026).

Saurabh Kalikar Chirag Jain, Md Vasimuddin et all.
mm2-fast: Accelerating minimap2 for long-read sequencing applications on modern CPUs. Undated online source. doi: https://doi.org/10.1038/s43588-022-00201-8. url: https://github.com/bwa-mem2/mm2-fast (visited on 08/19/2026).

Shen, Wei. LexicMap FAQs. Undated online source. url: https://bioinf.shenwei.me/LexicMap/faqs/ (visited on 08/19/2026).

Shur, Arseny. On Minimizers of Minimum Density. June 5, 2025. doi: 10.48550/arXiv.2506.05277. arXiv: 2506.05277 [cs.DS]. url: https://arxiv.org/abs/2506.05277 (visited on 08/31/2026).

Smith, Temple F. and Michael S. Waterman. "Identification of common molecular subsequences." In: Journal of Molecular Biology 147.1 (1981), pp. 195–197. doi: 10.1016/0022-2836(81)90087-5. url: https://doi.org/10.1016/0022-2836(81)90087-5.

Suzuki, Hajime and Masahiro Kasahara. "Introducing difference recurrence relations for faster semi-global alignment of long sequences." In: BMC Bioinformatics 19, 45 (2018). doi: 10.1186/s12859-018-2014-8. url: https://doi.org/10.1186/s12859-018-2014-8.

Sweden, NBIS - National Bioinformatics Infrastructure.
The Cost of Sequencing a Human Genome. Version 1. Undated online source. url: https://github.com/NBISweden/GAAS/blob/master/annotation/knowledge/cigar.md (visited on 07/28/2026).

Tarjan, Robert Endre. "Depth-First Search and Linear Graph Algorithms." In: SIAM Journal on Computing 1.2 (1972), pp. 146–160. doi: 10.1137/0201010. url: https://doi.org/10.1137/0201010.

UCSC Genome Browser Project. UCSC Genome Browser REST API data interface. Undated online source. url: https://genome.ucsc.edu/goldenPath/help/api.html (visited on 07/28/2026).

UniProt Consortium. Programmatic access to UniProt. Nov. 27, 2024. url: https://www.uniprot.org/help/programmatic_access (visited on 07/28/2026).

Youngblut, Nick. MMseqs2 Github issue 338. Undated online source. url: https://github.com/soedinglab/MMseqs2/issues/338 (visited on 08/19/2026).

Zhan, Xiaowei. MMseqs2 Github issue 517. Undated online source. url: https://github.com/soedinglab/MMseqs2/issues/517 (visited on 08/19/2026).